

OfferExchange

nano

October 1, 2026

Contents

1	Abstract offer-exchange model	1
1.1	Isabelle/HOL Setup	1
1.2	Price	2
1.3	Offers	2
1.4	The ledger	3
1.5	Maximum unrestricted exchange	4
1.6	Valid offers	4
1.7	Trades	5
1.8	Posting an offer	5
1.9	Crossing a posted sell-wheat offer	6
1.10	Properties	11
2	Exchange arithmetic	15
2.1	The model and its main properties	15
2.1.1	Fixed-width C++ arithmetic	16
2.1.2	Wide multiplication	17
2.1.3	Checked division	17
2.1.4	Price error bound	18
2.1.5	Price-error application	19
2.1.6	Offer value	21
2.1.7	Protocol-29 amount examples	24
2.1.8	Exchange before price-error thresholds	25
2.1.9	Full exchange	30
2.1.10	Protocol-facing exchange interface	30
2.1.11	Main properties	33
2.2	Wide multiplication	35
2.3	Checked division	35
2.4	Price error bound	39
2.5	Price-error application	41
2.6	Offer value	45
2.7	Exchange before price-error thresholds	46
2.8	Full exchange	59

3	Offer adjustment and liabilities	67
3.1	Offer adjustment and offer liabilities	67
3.2	Protocol compatibility of the unlimited-cap projections	70
3.3	Reusable normalization and replay results	74
4	Minimal implementation posting slice	82
4.1	Posting-state invariant facts	86
4.2	Repaired adjustment arithmetic	87
4.3	Posting refinement against the specification	94
5	Layered model of the wide arithmetic helpers	101
5.1	Definitions	101
5.1.1	Examples	102
5.2	The 128-bit family	102
5.2.1	Examples	104
5.3	Wide multiplication	104
5.3.1	Examples	104
5.4	Reading the word arithmetic as integer arithmetic	105
5.5	Characterizations	106
5.6	Agreement with the collapsed model	107
5.7	Wide multiplication agrees with the collapsed model	107
5.8	The 128-bit family: word arithmetic as integer arithmetic	108
5.9	The 128-bit family: characterizations	109
5.10	The 128-bit family agrees with the collapsed model	110

1 Abstract offer-exchange model

```

theory Offer-Exchange-Specification
  imports
    Main
    HOL.Rat
    HOL-Library.Monad-Syntax
begin

```

In this theory we formalize the offer exchange process with the goal of maximum simplicity, as opposed to following the C++ implementation. Ideally, we will later modify the C++ implementation to follow this specification.

1.1 Isabelle/HOL Setup

In this section we do a little technical setup that can be ignored if you mostly care about the semantics of the offer exchange and not Isabelle/HOL technicalities.

Longer ordered validation chains use the guarded conditional notation below. The first true guard determines the result, and the final otherwise clause is the fallback. This is input syntax only: Isabelle immediately translates it to ordinary nested conditionals.

nonterminal *cond-cases*

syntax

```
-cond :: cond-cases ⇒ 'a
  ((cond {/(2 -)/}) [12] 62)
-cond-when :: [bool, 'a, cond-cases] ⇒ cond-cases
  ((when - ⇒/ -/ | -) [0, 13, 12] 12)
-cond-otherwise :: 'a ⇒ cond-cases
  ((otherwise ⇒/ -) [13] 12)
```

translations

```
-cond (-cond-when condition result rest)
  ↪ CONST If condition result (-cond rest)
-cond (-cond-otherwise result)
  ↪ result
```

This theory enables Isabelle’s standard coercion from *int* (unbounded integer) to *rat* (unbounded-precision rational). An integer-valued ledger amount or price component used in a rational expression is therefore promoted automatically. This keeps exact prices written as ordinary quotients instead of obscuring the formulas with explicit conversion functions.

declare $[[coercion-enabled]]$

declare $[[coercion\ of\ int :: int \Rightarrow rat]]$

1.2 Price

datatype *ledger-price* =

Ledger-Price (*price-n*: *int*) (*price-d*: *int*)

definition *rat-price* :: *ledger-price* ⇒ *rat*

where

rat-price price = *price-n price* / *price-d price*

declare $[[coercion\ rat-price]]$ — We implicitly coerce *ledger-price* to rational.

definition *ledger-price-well-formed* :: *ledger-price* ⇒ *bool*

where

ledger-price-well-formed price $\longleftrightarrow 0 < price-n\ price \wedge 0 < price-d\ price$

ledger-price is the price representation stored in the ledger. Its coercion through *rat-price* gives the corresponding exact rational value whenever a rational expression requires it.

1.3 Offers

record *offer-attributes* =
 is-passive :: *bool*

record *sell-wheat-offer* = *offer-attributes* +
 wheat-amount :: *int*
 sheep-per-wheat :: *ledger-price*

record *sell-sheep-offer* = *offer-attributes* +
 sheep-amount :: *int*
 wheat-per-sheep :: *ledger-price*

definition *sell-wheat-offer-well-formed* :: *sell-wheat-offer* $\Rightarrow$ *bool*
 where
 sell-wheat-offer-well-formed offer $\longleftrightarrow$
 $0 < \text{wheat-amount } offer$
 $\wedge \text{ledger-price-well-formed } (\text{sheep-per-wheat } offer)$

definition *sell-sheep-offer-well-formed* :: *sell-sheep-offer* $\Rightarrow$ *bool*
 where
 sell-sheep-offer-well-formed offer $\longleftrightarrow$
 $0 < \text{sheep-amount } offer$
 $\wedge \text{ledger-price-well-formed } (\text{wheat-per-sheep } offer)$

The maker sells wheat and buys sheep, while the taker sells sheep and buys wheat. Both roles therefore have balances for both assets, while their selling liabilities, buying liabilities, and trustline limits follow their respective directions.

1.4 The ledger

definition *int64-max* :: *int*
 where
 int64-max = $2^{63} - 1$

We implicitly assume that all non-computed integer values are *int64-max* at most.

record *balances* =
 wheat-balance :: *int*
 sheep-balance :: *int*

record *maker-state* = *balances* +
 wheat-selling-liabilities :: *int*
 sheep-limit :: *int*
 sheep-buying-liabilities :: *int*

record *taker-state* = *balances* +
 sheep-selling-liabilities :: *int*

wheat-limit :: int
wheat-buying-liabilities :: int

record *ledger* =
maker-state :: *maker-state*
taker-state :: *taker-state*

definition *maker-state-well-formed* :: *maker-state* $\Rightarrow$ bool
where
maker-state-well-formed maker $\longleftrightarrow$
 $0 \leq \text{wheat-selling-liabilities } maker$
 $\wedge \text{wheat-selling-liabilities } maker \leq \text{wheat-balance } maker$
 $\wedge 0 \leq \text{sheep-balance } maker$
 $\wedge 0 \leq \text{sheep-buying-liabilities } maker$
 $\wedge \text{sheep-balance } maker + \text{sheep-buying-liabilities } maker \leq \text{sheep-limit } maker$

definition *maker-available-wheat* :: *maker-state* $\Rightarrow$ int
where
maker-available-wheat maker =
 $\text{wheat-balance } maker - \text{wheat-selling-liabilities } maker$

definition *maker-sheep-headroom* :: *maker-state* $\Rightarrow$ int
where
maker-sheep-headroom maker =
 $\text{sheep-limit } maker - \text{sheep-balance } maker -$
 $\text{sheep-buying-liabilities } maker$

1.5 Maximum unrestricted exchange

record *exchanged-amounts* =
wheat-to-taker :: int
sheep-to-maker :: int

definition *max-unrestricted-exchange* **where**
max-unrestricted-exchange offer = do {
 let $p = \text{sheep-per-wheat } offer$;
 let *feasible* = { *wheat* . $\exists$ *sheep* .
 $\text{wheat} = \lceil \text{sheep} / p \rceil \wedge \text{wheat} \leq \text{wheat-amount } offer$
 $\wedge \text{sheep} = \lfloor \text{wheat} * p \rfloor$
 $\wedge 0 < \text{wheat} \wedge \text{wheat} \leq \text{int64-max}$
 $\wedge 0 < \text{sheep} \wedge \text{sheep} \leq \text{int64-max}$
 $\wedge \text{wheat} * p \leq \text{int64-max}$ — NOTE: seems weird
 };
 let *max-wheat* = *Max feasible*;
 if *feasible* = {} then ($\text{wheat-to-taker} = 0, \text{sheep-to-maker} = 0$)
 else ($\text{wheat-to-taker} = \text{max-wheat}, \text{sheep-to-maker} = \lfloor \text{max-wheat} * p \rfloor$)

max-unrestricted-exchange corresponds to the maximum trade possible, measured in wheat, assuming an unrestricted counterparty.

1.6 Valid offers

TODO: is this enforced upstream of `OfferExchange.cpp`?

definition *valid-offer* **where**

```

valid-offer offer ledger  $\longleftrightarrow$  do {
  let max-exch = max-unrestricted-exchange offer;
  let maker-state = maker-state ledger;

  if (max-exch = (wheat-to-taker = 0, sheep-to-maker = 0))
  then
    False
  else
    wheat-balance maker-state  $\geq$ 
      wheat-to-taker max-exch + wheat-selling-liabilities maker-state
     $\wedge$  sheep-limit maker-state  $\geq$ 
      sheep-balance maker-state + sheep-to-maker max-exch
      + sheep-buying-liabilities maker-state}

```

An offer is valid when its maximum unrestricted exchange is nonzero and the maker can reserve the corresponding selling and buying liabilities.

1.7 Trades

TODO: do we need this record? Why not not just use *exchanged-amounts*?

record *trade* = *exchanged-amounts* +
resting-offer-stays :: *bool*

A *trade* records a positive whole-unit transfer and whether the resting offer has greater capped capacity (meaning it is not fully consumed and stays on the book).

definition *effective-sheep-per-wheat* :: 'a *exchanged-amounts-scheme* $\Rightarrow$ *rat*

where

```

effective-sheep-per-wheat exchange =
  sheep-to-maker exchange / wheat-to-taker exchange

```

For a trade with a positive wheat amount, *effective-sheep-per-wheat* is the realized number of sheep sent per wheat received.

definition *price-error-within-bound* ::

```

ledger-price  $\Rightarrow$  'a exchanged-amounts-scheme  $\Rightarrow$  bool

```

where

```

price-error-within-bound price exchange  $\longleftrightarrow$ 
  abs (effective-sheep-per-wheat exchange - price)  $\leq$  (price :: rat) / 100

```

For a positive whole-unit trade, *price-error-within-bound* compares its effective sheep-per-wheat price with the posted ledger price. It accepts the trade when their absolute difference is at most one percent of the posted price. A trade that fails this test is rejected.

1.8 Posting an offer

definition *reserve-sell-wheat-offer-liabilities* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *maker-state*

where

reserve-sell-wheat-offer-liabilities offer maker = *do* {
let amounts = *max-unrestricted-exchange offer*;
maker(
wheat-selling-liabilities :=
wheat-selling-liabilities maker + *wheat-to-taker amounts*,
sheep-buying-liabilities :=
sheep-buying-liabilities maker + *sheep-to-maker amounts*) } }

datatype *post-outcome* =

Malformed
| *Line-Full*
| *Underfunded*
| *No-Offer*
| *Created sell-wheat-offer maker-state*

definition *post-sell-wheat-offer* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *post-outcome*

where

post-sell-wheat-offer offer maker = *do* {
let amounts = *max-unrestricted-exchange offer*;
cond {
when $\neg$ *ledger-price-well-formed* (*sheep-per-wheat offer*) $\vee$ *wheat-amount*
offer < 0 $\Rightarrow$
Malformed
| *when* *maker-sheep-headroom maker* $\leq$ 0 $\vee$
maker-sheep-headroom maker < *sheep-to-maker amounts* $\Rightarrow$
Line-Full
| *when* *maker-available-wheat maker* < *wheat-to-taker amounts* $\Rightarrow$
Underfunded
| *when* *wheat-amount offer* = 0 $\vee$ *wheat-to-taker amounts* = 0 $\Rightarrow$
No-Offer
| *when* $\neg$ *price-error-within-bound*
(*sheep-per-wheat offer*) *amounts* $\Rightarrow$
No-Offer
| *otherwise* $\Rightarrow$ *do* {
let posted = *offer*(*wheat-amount* := *wheat-to-taker amounts*);
Created posted (*reserve-sell-wheat-offer-liabilities posted maker*) } } }

The line-full branch also rejects nonpositive sheep headroom, matching the immediate C++ check `maxWheatReceive == 0` after capacities are clamped. The maximum unrestricted exchange is the full execution selected against an unrestricted active taker, for which the resting offer does not stay. Applying the price-error threshold before reserving liabilities ensures that every *Created* offer admits that execution. Failure is *No-Offer* because the offer is

syntactically well formed but has no acceptable full unrestricted execution at whole-unit precision.

1.9 Crossing a posted sell-wheat offer

definition *taker-state-well-formed* :: *taker-state* $\Rightarrow$ *bool*

where

taker-state-well-formed taker $\longleftrightarrow$
 $0 \leq \text{sheep-selling-liabilities taker}$
 $\wedge \text{sheep-selling-liabilities taker} \leq \text{sheep-balance taker}$
 $\wedge 0 \leq \text{wheat-balance taker}$
 $\wedge 0 \leq \text{wheat-buying-liabilities taker}$
 $\wedge \text{wheat-balance taker} + \text{wheat-buying-liabilities taker} \leq \text{wheat-limit taker}$

definition *taker-available-sheep* :: *taker-state* $\Rightarrow$ *int*

where

taker-available-sheep taker =
 $\text{sheep-balance taker} - \text{sheep-selling-liabilities taker}$

definition *taker-wheat-headroom* :: *taker-state* $\Rightarrow$ *int*

where

taker-wheat-headroom taker =
 $\text{wheat-limit taker} - \text{wheat-balance taker} - \text{wheat-buying-liabilities taker}$

definition *offers-cross* ::

sell-wheat-offer $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *bool*

where

offers-cross resting incoming $\longleftrightarrow$ do {
 $\text{let sheep-per-wheat-resting} = (\text{sheep-per-wheat resting} :: \text{rat});$
 $\text{let sheep-per-wheat-incoming-limit} = 1 / (\text{wheat-per-sheep incoming} :: \text{rat});$
 $\text{if is-passive incoming}$
 $\text{then sheep-per-wheat-resting} < \text{sheep-per-wheat-incoming-limit}$
 $\text{else sheep-per-wheat-resting} \leq \text{sheep-per-wheat-incoming-limit}$
}

definition *release-sell-wheat-offer-liabilities* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *maker-state*

where

release-sell-wheat-offer-liabilities offer maker =
 $(\text{let amounts} = \text{max-unrestricted-exchange offer}$
 $\text{in maker}(\text{wheat-selling-liabilities} :=$
 $\text{wheat-selling-liabilities maker} - \text{wheat-to-taker amounts},$
 $\text{sheep-buying-liabilities} :=$
 $\text{sheep-buying-liabilities maker} - \text{sheep-to-maker amounts}))$

record *exchange-caps* =

maker-wheat-send-cap :: *int*
taker-wheat-receive-cap :: *int*

taker-sheep-send-cap :: *int*
maker-sheep-receive-cap :: *int*

exchange-caps groups both parties' upper bounds on the two transferred assets.

definition *exchange-caps* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *exchange-caps*

where

exchange-caps *resting* *maker* *incoming* *taker* =

(*let* *amounts* = *max-unrestricted-exchange* *resting*

in *let* *released-maker* =

release-sell-wheat-offer-liabilities *resting* *maker*

in (*maker-wheat-send-cap* =

min (*wheat-to-taker* *amounts*)

(*maker-available-wheat* *released-maker*), — TODO: Isn't this

guaranteed to be the first arg since we just released the liabilities?

taker-wheat-receive-cap = *taker-wheat-headroom* *taker*,

taker-sheep-send-cap =

min (*sheep-amount* *incoming*) (*taker-available-sheep* *taker*),

maker-sheep-receive-cap =

min (*sheep-to-maker* *amounts*) — NOTE: C++ just uses just the

headroom...

(*maker-sheep-headroom* *released-maker*)))

exchange-caps combines the resting offer's released reservation with the incoming offer's amount and the parties' current ledger capacities. By construction, it caps the resting side by its reservation even after releasing that reservation. Consequently, unrelated capacity acquired later cannot cause the crossing to consume more than the resting offer had reserved. The incoming offer's own price is deliberately absent: it decides whether the offers cross, while a selected crossing executes at the resting price.

definition *resting-capacity-in-wheat* ::

ledger-price $\Rightarrow$ *exchange-caps* $\Rightarrow$ *rat*

where

resting-capacity-in-wheat *price* *caps* =

min (*maker-wheat-send-cap* *caps*)

(*maker-sheep-receive-cap* *caps* / *price*)

definition *incoming-capacity-in-wheat* ::

ledger-price $\Rightarrow$ *exchange-caps* $\Rightarrow$ *rat*

where

incoming-capacity-in-wheat *price* *caps* =

min (*taker-wheat-receive-cap* *caps*)

(*taker-sheep-send-cap* *caps* / *price*)

resting-capacity-in-wheat and *incoming-capacity-in-wheat* express both sides' exact capacities in the common unit of wheat at the resting price. They are rational quantities so the comparison happens before whole-unit rounding.

definition *resting-offer-has-greater-capacity* ::

ledger-price $\Rightarrow$ *exchange-caps* $\Rightarrow$ *bool*

where

resting-offer-has-greater-capacity price caps $\longleftrightarrow$

resting-capacity-in-wheat price caps $>$ *incoming-capacity-in-wheat price caps*

definition *resting-offer-stays-at-crossing* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *bool*

— TODO: does this actually implies it will stay after adjustment?

where

resting-offer-stays-at-crossing resting maker incoming taker $\longleftrightarrow$

(*let caps* = *exchange-caps resting maker incoming taker*

in resting-offer-has-greater-capacity (sheep-per-wheat resting) caps)

resting-offer-has-greater-capacity uses a strict comparison: the resting offer stays only when its exact wheat capacity is greater. Equal capacities therefore do not leave a remainder of the resting offer. *resting-offer-stays-at-crossing* instantiates that comparison with the live caps and the resting execution price.

The pre-threshold selector below mirrors the liability calculation: it describes every positive whole-unit crossing allowed by the four live caps and the appropriate rounding direction, then chooses the greatest feasible wheat transfer. An empty feasible set denotes the absence of a trade.

definition *trade-before-error-threshold* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *trade option*

— TODO: shouldn't we just use the posted amount for maker capacity? It's supposed to be an invariant that liabilities are covered.

where

trade-before-error-threshold resting maker incoming taker =

(*let price* = *sheep-per-wheat resting*

in let caps =

exchange-caps resting maker incoming taker

in let stays =

resting-offer-has-greater-capacity price caps

in let feasible = {*wheat*. $\exists$ *sheep*.

$0 < \textit{wheat}$

$\wedge \textit{wheat} \leq \textit{maker-wheat-send-cap caps}$

$\wedge \textit{wheat} \leq \textit{taker-wheat-receive-cap caps}$

$\wedge \textit{wheat} \leq \textit{int64-max}$

$\wedge 0 < \textit{sheep}$

$\wedge \textit{sheep} \leq \textit{taker-sheep-send-cap caps}$

$\wedge \textit{sheep} \leq \textit{maker-sheep-receive-cap caps}$

$\wedge \textit{sheep} \leq \textit{int64-max}$

$\wedge \textit{wheat} * \textit{price} \leq \textit{int64-max}$

$\wedge \textit{sheep} / \textit{price} \leq \textit{int64-max}$

$\wedge$ (*if stays*

then sheep = $\lceil \textit{wheat} * \textit{price} \rceil$

$\wedge \textit{wheat}$ = $\lfloor \textit{sheep} / \textit{price} \rfloor$)

```

      else  $sheep = \lfloor wheat * price \rfloor$ 
            $\wedge wheat = \lceil sheep / price \rceil$ 
    in if feasible = {} then None
    else
      (let wheat = Max feasible
       in let sheep =
          (if stays then  $\lceil wheat * price \rceil$ 
           else  $\lfloor wheat * price \rfloor$ )
       in Some
          ( $\lfloor wheat-to-taker = wheat,$ 
            $sheep-to-maker = sheep,$ 
            $resting-offer-stays = stays \rfloor$ ))

```

trade-before-error-threshold selects amounts before applying the price-error filter. When the resting offer stays, the rounding favors its seller; otherwise the opposite rounding favors the incoming seller. The mutually rounded equations rule out transfers made artificially favorable by more than whole-unit indivisibility requires.

definition *trade* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *trade option*

where

```

trade resting maker incoming taker =
  (case trade-before-error-threshold
   resting maker incoming taker of
    None  $\Rightarrow$  None
  | Some selected  $\Rightarrow$ 
    if price-error-within-bound
      ( $sheep-per-wheat$  resting) selected
    then Some selected else None)

```

trade applies the one-percent price-error bound to the maximal pre-threshold trade. Rejection produces no trade.

definition *cross-sell-wheat-offer* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *trade option*

where

```

cross-sell-wheat-offer resting maker incoming taker =
  (if offers-cross resting incoming
   then trade resting maker incoming taker
   else None)

```

cross-sell-wheat-offer first checks price compatibility and then delegates to the declarative selector. Well-formed offers and party states, together with coverage of the resting offer's reservation, are domain assumptions for later crossing theorems rather than additional result variants of this pure selection function.

definition *unrestricted-sell-sheep-offer* ::

sell-wheat-offer $\Rightarrow$ *sell-sheep-offer*

where

```

unrestricted-sell-sheep-offer resting =
  (is-passive = False,
   sheep-amount = int64-max,
   wheat-per-sheep =
     Ledger-Price
     (price-d (sheep-per-wheat resting))
     (price-n (sheep-per-wheat resting)))

```

definition *unrestricted-taker-state* :: *taker-state*

where

```

unrestricted-taker-state =
  (wheat-balance = 0,
   sheep-balance = int64-max,
   sheep-selling-liabilities = 0,
   wheat-limit = int64-max,
   wheat-buying-liabilities = 0)

```

unrestricted-sell-sheep-offer is an active incoming offer at the reciprocal resting price with the largest ledger amount. Together with *unrestricted-taker-state*, which has maximal sheep balance and wheat headroom and no liabilities, it provides the canonical counterparty used to state that a successfully posted offer is executable.

Refinement note: *exchange-caps* caps the maker's sheep-receive capacity by both its live headroom and the buying liability reserved for the resting offer. In contrast, C++ `crossOfferV10` releases that liability and passes the full live headroom returned by `canBuyAtMost` to `exchangeV10`. Extra headroom can therefore change which offer stays and select a different rounding branch; for small amounts, an exchange admitted by this specification can become a zero exchange in C++. This mismatch remains when both exact-cap protocol choices are enabled because they do not change the initial stays comparison.

1.10 Properties

A resting offer successfully produced by *post-sell-wheat-offer* carries a full-fill pair given by *max-unrestricted-exchange*. The property below says that any price-compatible incoming offer whose net send and receive capacities cover that pair consumes the resting offer completely. The crossing-time maker state is otherwise arbitrary: well-formedness and coverage of the resting offer's two booked liabilities are the only maker requirements. In particular, unrelated extra maker balance or headroom must not change the transferred amounts or which offer stays.

lemma *max-unrestricted-exchange-all-zero-or-none*:

assumes *sell-wheat-offer-well-formed offer*

defines *amounts* $\equiv$ *max-unrestricted-exchange offer*

shows $wheat\text{-}to\text{-}taker\ amounts = 0 \iff sheep\text{-}to\text{-}maker\ amounts = 0$

Proof sketch. The feasible wheat amounts form a finite set because they lie in an integer interval. If the set is empty, the definition assigns zero to both amounts. Otherwise its maximum belongs to the set; feasibility says that the maximum wheat amount and its corresponding rounded sheep amount are both positive. Hence neither amount is zero.

Proof sketch. Reaching the *Created* branch means that every earlier posting guard was false. The negated guards give price and amount validity, a nonzero unrestricted amount, acceptance of that exchange by the price-error threshold, and sufficient maker capacity. Injectivity of *Created* identifies the returned state with the state produced by reservation.

lemma *created-sell-wheat-offer-facts*:

assumes *posted*:

post-sell-wheat-offer offer maker = Created offer posted-maker

defines $amounts \equiv max\text{-}unrestricted\text{-}exchange\ offer$

defines $unrestricted\text{-}trade \equiv$

$(wheat\text{-}to\text{-}taker = wheat\text{-}to\text{-}taker\ amounts,$
 $sheep\text{-}to\text{-}maker = sheep\text{-}to\text{-}maker\ amounts,$
 $resting\text{-}offer\text{-}stays = False)$

shows $ledger\text{-}price\text{-}well\text{-}formed\ (sheep\text{-}per\text{-}wheat\ offer)$

$\wedge 0 < wheat\text{-}amount\ offer$
 $\wedge wheat\text{-}to\text{-}taker\ amounts \neq 0$
 $\wedge price\text{-}error\text{-}within\text{-}bound$
 $\quad (sheep\text{-}per\text{-}wheat\ offer)\ unrestricted\text{-}trade$
 $\wedge sheep\text{-}to\text{-}maker\ amounts$
 $\quad \leq maker\text{-}sheep\text{-}headroom\ maker$
 $\wedge wheat\text{-}to\text{-}taker\ amounts$
 $\quad \leq maker\text{-}available\text{-}wheat\ maker$
 $\wedge posted\text{-}maker =$
 $\quad reserve\text{-}sell\text{-}wheat\text{-}offer\text{-}liabilities\ offer\ maker$

Proof sketch. A nonzero wheat amount rules out the empty feasible set, because the empty branch assigns zero. The set is finite since all its wheat amounts lie in a finite integer interval. Its maximum therefore belongs to the set, and expanding the nonempty branch transfers every feasibility condition to the two stored amounts.

lemma *nonzero-max-unrestricted-exchange-is-feasible*:

assumes *nonzero*:

wheat-to-taker

$(max\text{-}unrestricted\text{-}exchange\ offer) \neq 0$

defines $amounts \equiv max\text{-}unrestricted\text{-}exchange\ offer$

shows

$wheat\text{-}to\text{-}taker\ amounts =$
 $\lceil sheep\text{-}to\text{-}maker\ amounts /$
 $\quad sheep\text{-}per\text{-}wheat\ offer \rceil$

$$\begin{aligned}
& \wedge \text{wheat-to-taker amounts} \leq \text{wheat-amount offer} \\
& \wedge \text{sheep-to-maker amounts} = \\
& \quad \lfloor \text{wheat-to-taker amounts} * \\
& \quad \quad \text{sheep-per-wheat offer} \rfloor \\
& \wedge 0 < \text{wheat-to-taker amounts} \\
& \wedge \text{wheat-to-taker amounts} \leq \text{int64-max} \\
& \wedge \text{sheep-to-maker amounts} / \\
& \quad \text{sheep-per-wheat offer} \leq \text{int64-max} \\
& \wedge 0 < \text{sheep-to-maker amounts} \\
& \wedge \text{sheep-to-maker amounts} \leq \text{int64-max} \\
& \wedge \text{wheat-to-taker amounts} * \\
& \quad \text{sheep-per-wheat offer} \leq \text{int64-max}
\end{aligned}$$

Proof sketch. Reserving and then releasing this offer cancels exactly the two liability additions. The successful posting guards say the original maker capacities cover the liabilities, so both maker-side minima select the liabilities. The canonical taker contributes *int64-max* on both of its sides.

lemma *unrestricted-caps-after-created:*

assumes *posted:*

post-sell-wheat-offer offer maker = Created offer posted-maker

defines *amounts* $\equiv$ *max-unrestricted-exchange offer*

shows

exchange-caps
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
(maker-wheat-send-cap =
wheat-to-taker amounts,
taker-wheat-receive-cap = int64-max,
taker-sheep-send-cap = int64-max,
maker-sheep-receive-cap =
sheep-to-maker amounts)

Proof sketch. The canonical incoming offer is active and stores the reciprocal ledger price. Taking its reciprocal recovers the resting price, so the non-strict active-offer crossing test holds with equality.

lemma *unrestricted-offer-crosses:*

offers-cross offer (unrestricted-sell-sheep-offer offer)

Proof sketch. The two resting caps are the unrestricted exchange amounts, bounds both by *int64-max*. Positivity of the posted price preserves the sheep-cap inequality when converting it to wheat. Monotonicity of *min* therefore makes resting capacity no greater than canonical incoming capacity, so the strict stays comparison is false.

lemma *unrestricted-resting-offer-does-not-stay-after-created:*

assumes *posted*:
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
shows
 $\neg$ *resting-offer-has-greater-capacity*
(sheep-per-wheat offer)
(exchange-caps
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state)

Proof sketch. The unrestricted exchange is feasible for the pre-threshold selector under the canonical caps and the non-stays rounding direction. Conversely, every selector-feasible wheat amount is bounded by the unrestricted wheat amount. Hence that amount is exactly the maximum. Its rounded sheep amount is the unrestricted sheep amount, so the selected record is that exchange.

lemma *unrestricted-prethreshold-exchange-after-created*:
assumes *posted*:
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
defines *unrestricted-trade* $\equiv$
(wheat-to-taker = wheat-to-taker amounts,
sheep-to-maker = sheep-to-maker amounts,
resting-offer-stays = False)
shows
trade-before-error-threshold
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
Some unrestricted-trade

Proof sketch. The previous lemma identifies the pre-threshold selection. The additional posting guard records exactly that this unrestricted exchange satisfies the price-error bound, so the thresholded selector preserves it.

lemma *unrestricted-selected-exchange-after-created*:
assumes *posted*:
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
defines *unrestricted-trade* $\equiv$
(wheat-to-taker = wheat-to-taker amounts,
sheep-to-maker = sheep-to-maker amounts,
resting-offer-stays = False)
shows
trade
offer posted-maker

(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
Some unrestricted-trade

Proof sketch. The canonical incoming offer crosses at equality because it is active and reciprocal. The thresholded selector returns the unrestricted exchange by the preceding lemma, so unfolding the outer crossing check yields that same successful trade.

theorem *successfully-posted-offer-has-unrestricted-taker:*

assumes *posted:*

post-sell-wheat-offer offer maker = Created offer posted-maker

defines *amounts ≡ max-unrestricted-exchange offer*

shows

cross-sell-wheat-offer
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
Some
(|wheat-to-taker =
wheat-to-taker amounts,
sheep-to-maker =
sheep-to-maker amounts,
resting-offer-stays = False|)

end

theory *Offer-Exchange-Arithmetic*

imports *HOL–Library.Word HOL–Library.Monad-Syntax*

begin

2 Exchange arithmetic

The export check of the Isabelle tooling audits, alongside the code equations of the exported model, every fact collected under *export-audit*: the theorems that connect the exported code-level definitions to what was proved about them must depend on no oracle and on no project axiom that is not a definition. Theories add their refinement and equivalence theorems to it.

named-theorems *export-audit*

facts the export check audits together with the exported code equations

This theory gives an executable, bit-precise model of the arithmetic used by *exchangeV10*. It first presents the modeled operations and their main properties, then proves those properties in the same order so that the definitions and their verification can be read side by side.

2.1 The model and its main properties

This first section presents the model in full and, at its end, states the properties that the remaining sections prove. Nothing is proved here, so the model can be read on its own and compared line by line with *OfferExchange.cpp*.

The definitions stay close to the C++ they model: the same branch structure, the same order of calls, and bit-precise arithmetic on 32-, 64-, and 128-bit words. The error monad below stands for the C++ control flow that a failed assertion, a detected overflow, or an explicit *throw* interrupts. Alongside each modeled function there is a specification phrased over unbounded integers; the proofs then show that the two agree.

2.1.1 Fixed-width C++ arithmetic

type-synonym *int32* = 32 word

type-synonym *int64* = 64 word

type-synonym *uint128* = 128 word

abbreviation *int64-max-int* :: int

— C++: *INT64-MAX*

where *int64-max-int* $\equiv$ 9223372036854775807

int64-max-int is the value of the C++ constant *INT64-MAX* as an unbounded integer, that is $2^{63} - 1$. It is an abbreviation rather than a definition so that every arithmetic step still sees the plain numeral. The companion constant *int64-max* defined below is the signed 64-bit word with that bit pattern.

datatype *cxx-error* =

Cxx-Assertion-Failed

| *Cxx-Overflow*

| *Cxx-Runtime-Error*

datatype 'a *cxx-result* =

Cxx-Ok 'a

| *Cxx-Err* *cxx-error*

fun *cxx-bind* ::

'a *cxx-result* $\Rightarrow$ ('a $\Rightarrow$ 'b *cxx-result*) $\Rightarrow$ 'b *cxx-result*

where

cxx-bind (*Cxx-Ok* x) f = f x

| *cxx-bind* (*Cxx-Err* e) f = *Cxx-Err* e

adhoc-overloading

Monad-Syntax.bind $\hat{=}$ *cxx-bind*

The operation ($\gg=$) is an executable exception-style bind. A value built with *Cxx-Err* is propagated unchanged, while a value built with *Cxx-Ok*

is passed to the next computation. The overload enables `do` notation in executable definitions below.

lemma *cxx-bind-assoc*:

cxx-bind (*cxx-bind* *m f*) *g* = *cxx-bind* *m* ($\lambda x. \text{cxx-bind } (f\ x)\ g$)

— Sequencing is associative, so a computation may be repackaged as a helper that performs several steps without changing its result.

Proof sketch: an error in *m* is propagated by both sides, and an *Cxx-Ok* value is passed to the same continuation by both sides, so a case distinction on *m* closes the goal.

2.1.2 Wide multiplication

This models the C++ function *bigMultiply* from *util/numeric.cpp*. Its arguments have the bit patterns of C++ *int64_t* values, so they are interpreted with *sint*. The implementation asserts that both arguments are non-negative, widens them to unsigned 128-bit values, and multiplies them. Two non-negative signed 64-bit values have a product below 2^{126} , so the 128-bit multiplication cannot wrap.

definition *big-multiply* ::

int64 $\Rightarrow$ *int64* $\Rightarrow$ *uint128 cxx-result*

— C++: *bigMultiply* (*util/numeric.cpp*)

where

big-multiply *a b* =

(if *sint a* < 0 $\vee$ *sint b* < 0 then

Cxx-Err Cxx-Assertion-Failed

else

Cxx-Ok (*word-of-int* (*sint a* * *sint b*)))

2.1.3 Checked division

datatype *cxx-rounding* =

Cxx-Round-Down

| *Cxx-Round-Up*

fun *rounded-quotient* ::

int $\Rightarrow$ *int* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ *int*

where

rounded-quotient *numerator divisor Cxx-Round-Down* =
numerator div divisor

| *rounded-quotient* *numerator divisor Cxx-Round-Up* =
(*numerator* + *divisor* - 1) *div divisor*

The two definitions below model the throwing public helpers *bigDivide-OrThrow* and *bigDivideOrThrow128* from *util/numeric.cpp*. The ordinary helper widens a product of signed 64-bit operands. The 128-bit helper ad-

ditionally preserves the explicit guard that prevents the numerator used for round-up from wrapping before division.

definition *big-divide-or-throw* ::

int64 $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$
int64 cxx-result

— C++: *bigDivideOrThrow* (*util/numeric.cpp*)

where

big-divide-or-throw *a b c rounding* =
 (if *sint a* < 0 $\vee$ *sint b* < 0 $\vee$ *sint c* $\leq$ 0 then
 Cxx-Err Cxx-Assertion-Failed
 else
 let *quotient* =
 rounded-quotient (*sint a* * *sint b*) (*sint c*) *rounding*
 in if *quotient* > *int64-max-int* then
 Cxx-Err Cxx-Overflow
 else
 Cxx-Ok (*word-of-int quotient*))

definition *big-divide-or-throw128* ::

uint128 $\Rightarrow$ *int64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ *int64 cxx-result*

— C++: *bigDivideOrThrow128* (*util/numeric.cpp*)

where

big-divide-or-throw128 *a b rounding* =
 (if *sint b* $\leq$ 0 then
 Cxx-Err Cxx-Assertion-Failed
 else if *rounding* = *Cxx-Round-Up* $\wedge$
 uint a > (*2 :: int*) $^$ 128 – 1 – (*sint b* – 1) then
 Cxx-Err Cxx-Overflow
 else
 let *quotient* = *rounded-quotient* (*uint a*) (*sint b*) *rounding*
 in if *quotient* > *int64-max-int* then
 Cxx-Err Cxx-Overflow
 else
 Cxx-Ok (*word-of-int quotient*))

2.1.4 Price error bound

The definition below is the direct model of *checkPriceErrorBound* (*OfferExchange.cpp*). The C++ function first scales each signed 32-bit price component by one hundred in signed 64-bit arithmetic. It then compares two unsigned 128-bit products. If the caller permits unbounded error in favor of wheat, the favorable direction returns immediately; otherwise the absolute difference must be no greater than the unscaled wheat value.

definition *check-price-error-bound* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
bool $\Rightarrow$ *bool cxx-result*

— C++: *checkPriceErrorBound* (*OfferExchange.cpp*)

where

```

check-price-error-bound price-n price-d wheat-receive sheep-send
  can-favor-wheat = do {
    lhs ← big-multiply
      (word-of-int (100 * sint price-n)) wheat-receive;
    rhs ← big-multiply
      (word-of-int (100 * sint price-d)) sheep-send;
    if can-favor-wheat ∧ rhs > lhs then
      Cxx-Ok True
    else
      let abs-diff = (if lhs > rhs then lhs - rhs else rhs - lhs)
      in do {
        cap ← big-multiply (scast price-n) wheat-receive;
        Cxx-Ok (abs-diff ≤ cap)
      }
  }

```

definition *price-error-bound-spec* ::

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ bool ⇒ bool

where

```

price-error-bound-spec price-n price-d wheat-receive sheep-send
  can-favor-wheat =
    (let lhs = 100 * sint price-n * sint wheat-receive;
     rhs = 100 * sint price-d * sint sheep-send;
     cap = sint price-n * sint wheat-receive
     in (can-favor-wheat ∧ rhs > lhs) ∨ abs (lhs - rhs) ≤ cap)

```

price-error-bound-spec is the intended mathematical reading of the bit-precise test above, stated over the exact integers denoted by the arguments rather than over 128-bit words.

2.1.5 Price-error application

datatype *exchange-rounding* =

```

  Exchange-Normal
| Exchange-Strict-Send
| Exchange-Strict-Receive

```

record *exchange-options* =

```

  exact-receive-cap :: bool
  symmetric-exact-receive-cap :: bool

```

exchange-options groups protocol-gated choices passed through the exchange model. A record literal such as (*exact-receive-cap* = *True*, *symmetric-exact-receive-cap* = *False*) makes the selected behavior explicit at each call site and leaves room for additional options without adding more positional Boolean arguments. The *exact-receive-cap* field models the C++ parameter *exactReceiveCap*, which repairs the sheep-stays, wheat-more-valuable normal branch; the *symmetric-exact-receive-cap* field models the C++ parameter *symmetricEx-*

actReceiveCap, which applies the mirror-image repair to the wheat-stays, sheep-more-valuable normal branch.

The three constructors of *exchange-rounding* correspond to *NORMAL*, *PATH-PAYMENT-STRICT-S* and *PATH-PAYMENT-STRICT-RECEIVE*. The result record retains the two signed 64-bit amount bit patterns and the unchanged decision about which offer remains.

```
record exchange-result-v10 =
  num-wheat-received :: int64
  num-sheep-send :: int64
  result-wheat-stays :: bool
```

```
definition make-exchange-result ::
  int64 ⇒ int64 ⇒ bool ⇒ exchange-result-v10
where
  make-exchange-result wheat-receive sheep-send wheat-stays =
    (num-wheat-received = wheat-receive,
     num-sheep-send = sheep-send,
     result-wheat-stays = wheat-stays)
```

```
definition favored-seller-ok ::
  int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ bool ⇒ bool
where
  favored-seller-ok price-n price-d wheat-receive sheep-send wheat-stays =
    (let wheat-value = sint wheat-receive * sint price-n;
     sheep-value = sint sheep-send * sint price-d
     in if wheat-stays
        then wheat-value ≤ sheep-value
        else sheep-value ≤ wheat-value)
```

This definition follows the branch and mutation order of *applyPriceErrorThresholds*. In particular, arithmetic assertions are reached only when both amounts are positive, and the non-positive strict-send branch tests the raw sheep word for zero exactly as C++ does.

```
definition apply-price-error-thresholds ::
  int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ bool ⇒
  exchange-rounding ⇒ exchange-result-v10 cxx-result
— C++: applyPriceErrorThresholds (OfferExchange.cpp)
where
  apply-price-error-thresholds price-n price-d wheat-receive sheep-send
  wheat-stays rounding =
    (if 0 < sint wheat-receive ∧ 0 < sint sheep-send then do {
      wheat-value ← big-multiply wheat-receive (scast price-n);
      sheep-value ← big-multiply sheep-send (scast price-d);
      if (wheat-stays ∧ sheep-value < wheat-value) ∨
        (¬ wheat-stays ∧ sheep-value > wheat-value)
      then Cxx-Err Cxx-Runtime-Error
      else if rounding = Exchange-Normal then do {
```

```

    within-bound  $\leftarrow$ 
      check-price-error-bound price-n price-d wheat-receive sheep-send
      False;
    if within-bound
    then Cxx-Ok
      (make-exchange-result wheat-receive sheep-send wheat-stays)
    else Cxx-Ok (make-exchange-result 0 0 wheat-stays)
  } else do {
    within-bound  $\leftarrow$ 
      check-price-error-bound price-n price-d wheat-receive sheep-send
      True;
    if within-bound
    then Cxx-Ok
      (make-exchange-result wheat-receive sheep-send wheat-stays)
    else Cxx-Err Cxx-Runtime-Error
  }
} else if rounding = Exchange-Strict-Send then
  if sheep-send = 0
  then Cxx-Err Cxx-Runtime-Error
  else Cxx-Ok
    (make-exchange-result wheat-receive sheep-send wheat-stays)
else Cxx-Ok (make-exchange-result 0 0 wheat-stays))

```

definition *apply-price-error-thresholds-spec* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *bool* $\Rightarrow$
exchange-rounding $\Rightarrow$ *exchange-result-v10* *cxx-result*

where

```

apply-price-error-thresholds-spec price-n price-d wheat-receive sheep-send
  wheat-stays rounding =
    (let original =
      make-exchange-result wheat-receive sheep-send wheat-stays;
      zero = make-exchange-result 0 0 wheat-stays
    in if  $\neg$  ( $0 < \text{sint } \text{wheat-receive} \wedge 0 < \text{sint } \text{sheep-send}$ ) then
      if rounding = Exchange-Strict-Send then
        if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
        else Cxx-Ok original
      else Cxx-Ok zero
    else if  $\text{sint } \text{price-n} < 0 \vee \text{sint } \text{price-d} < 0$  then
      Cxx-Err Cxx-Assertion-Failed
    else if  $\neg$  favored-seller-ok price-n price-d wheat-receive sheep-send
      wheat-stays
    then Cxx-Err Cxx-Runtime-Error
    else if rounding = Exchange-Normal then
      Cxx-Ok
        (if price-error-bound-spec price-n price-d wheat-receive
          sheep-send False
        then original else zero)
    else if price-error-bound-spec price-n price-d wheat-receive
      sheep-send True

```

```

    then Cxx-Ok original
    else Cxx-Err Cxx-Runtime-Error)

```

apply-price-error-thresholds-spec restates the same function as a single decision tree over exact integers, with no monadic sequencing left.

2.1.6 Offer value

This is the direct model of *calculateOfferValue*. The signed 32-bit prices are sign-extended to the signed 64-bit arguments expected by *big-multiply*. The final minimum uses the unsigned ordering on 128-bit words, matching C++ *std::min* on *uint128-t*.

definition *calculate-offer-value* ::

```

    int32 ⇒ int32 ⇒ int64 ⇒
    int64 ⇒ uint128 cxx-result

```

— C++: *calculateOfferValue* (*OfferExchange.cpp*)

where

```

    calculate-offer-value price-n price-d max-send max-receive = do {
        send-value ← big-multiply max-send (scast price-n);
        receive-value ← big-multiply max-receive (scast price-d);
        Cxx-Ok (min send-value receive-value)
    }

```

definition *calculate-offer-value-pre* ::

```

    int32 ⇒ int32 ⇒ int64 ⇒
    int64 ⇒ bool

```

where

```

    calculate-offer-value-pre price-n price-d max-send max-receive ⇔
    0 ≤ sint price-n ∧ 0 ≤ sint price-d ∧
    0 ≤ sint max-send ∧ 0 ≤ sint max-receive

```

definition *int64-max* :: *int64*

— C++: *INT64-MAX*

where *int64-max* = *word-of-int* ($2^{63} - 1$)

int64-max has the bit pattern of the C++ constant *INT64-MAX*.

definition *calculate-offer-value-with-exact-receive-cap* ::

```

    int32 ⇒ int32 ⇒ int64 ⇒
    int64 ⇒ uint128 cxx-result

```

— proof-only: the value stage of *calculateOfferAmountFromValue* (*OfferExchange.cpp*)

where

```

    calculate-offer-value-with-exact-receive-cap price-n price-d max-send
    max-receive = do {
        send-value ← big-multiply max-send (scast price-n);
        receive-product ← big-multiply max-receive (scast price-d);
        let receive-value = receive-product +
            ucast (scast (price-d - 1) :: int64);
        receive-cap ← big-multiply int64-max (scast price-d);
    }

```

```

    Cxx-Ok (min send-value (min receive-value receive-cap))
  }

```

This is the value stage of *calculateOfferAmountFromValue* (*OfferExchange.cpp*), split out for proofs. It is not itself a C++ function: the implementation-facing model is *calculate-offer-amount-from-value* below, which performs this calculation and the final round-down division in one helper. Compared with *calculate-offer-value*, the receive-side value is relaxed by *sint price-d* $- 1$ before the caller rounds down, so a receive cap that is not an exact multiple of the price still admits the last fractional lot. The addend models the C++ cast chain from *priceD* $- 1$ through *uint64-t* to *uint128-t*: 32-bit subtraction, sign extension to 64 bits, zero extension to 128 bits. The subsequent minimum with *big-multiply int64-max* (*scast price-d*) restores the plain cap when the receive cap is *INT64-MAX*, keeping such offers fixed points of adjustment.

definition *calculate-offer-amount-from-value* ::

```

  int32  $\Rightarrow$  int32  $\Rightarrow$  int64  $\Rightarrow$ 
  int64  $\Rightarrow$  int64 cxx-result

```

— C++: *calculateOfferAmountFromValue* (*OfferExchange.cpp*)

where

```

  calculate-offer-amount-from-value price-n price-d max-send
  max-receive = do {
    send-value  $\leftarrow$  big-multiply max-send (scast price-n);
    receive-product  $\leftarrow$  big-multiply max-receive (scast price-d);
    let receive-value = receive-product +
      ucast (scast (price-d - 1) :: int64);
    receive-cap  $\leftarrow$  big-multiply int64-max (scast price-d);
    let capped-receive-value = min receive-value receive-cap;
    let offer-value = min send-value capped-receive-value;
    big-divide-or-throw128 offer-value (scast price-n) Cxx-Round-Down
  }

```

This is the direct model of *calculateOfferAmountFromValue* (*OfferExchange.cpp*), the helper introduced for protocol 29. It keeps the C++ sequencing: the send-side product, the receive-side product relaxed by *priceD* $- 1$ through the cast chain *int32-t* to *uint64-t* to *uint128-t*, the saturation clamp by *big-multiply int64-max* (*scast price-d*), the minimum of the send and capped receive values, and the final round-down division by *sint price-n*.

Returning the amount rather than the value is the one structural change protocol 29 makes to this part of the arithmetic. It exists because the receive cap a resting offer imposes on itself is its own buying liability *price-n* $\ast$ *amount div price-d*, and dividing that by the price again rounds down a second time, which can lose one unit. Relaxing the receive product by *sint price-d* $- 1$ before the single round-down division compensates for exactly that lost unit, while the clamp keeps an *INT64-MAX* receive cap behaving as before.

lemma *calculate-offer-amount-from-value-eq-value-then-divide* [simp]:

$$\begin{aligned} & \text{calculate-offer-amount-from-value } \text{price-n } \text{price-d } \text{max-send } \text{max-receive} = \\ & (\text{calculate-offer-value-with-exact-receive-cap } \text{price-n } \text{price-d } \text{max-send} \\ & \quad \text{max-receive} \gg= \\ & \quad (\lambda \text{offer-value.} \\ & \quad \quad \text{big-divide-or-throw128 } \text{offer-value } (\text{scast } \text{price-n}) \text{ Cxx-Round-Down})) \end{aligned}$$

 — The protocol-29 helper is exactly the relaxed offer value followed by the round-down division the caller used to perform.

Proof sketch: both sides run the same three multiplications and form the same minimum. They differ only in where the division is sequenced, so unfolding the two definitions and reassociating the binds with *cxx-bind-assoc* makes them syntactically equal.

This equation is the bridge that carries the existing repaired-arithmetic results to the protocol-29 interface. Every proof phrased in terms of the relaxed value followed by a division applies verbatim to *calculate-offer-amount-from-value*, so the protocol-29 helper needs no separate development of the underlying arithmetic.

It is declared a simplification rule so that the existing repaired development continues to apply unchanged. The helper remains the implementation-facing definition, and the one exported for code generation, because it is what protocol 29 actually computes; the decomposition into a relaxed value and a division is a proof device only.

2.1.7 Protocol-29 amount examples

The examples below are the boundary cases cited by the protocol-29 change (stellar-core commit *cb257d2810*), evaluated in the model. Each pairs the protocol-29 helper with the protocol-28 calculation it replaces, so the intended change of behavior is explicit and executable.

A resting offer of 2999 units at price $3/2$ books a buying liability of $(3::'a) * (2999::'a) \text{ div } (2::'a)$, that is 4498. Re-deriving the amount from that liability is exactly the round trip protocol 29 repairs: protocol 28 returns 2998, silently shrinking an offer that nothing had consumed, while protocol 29 returns the stored 2999.

lemma *p29-amount-recovers-offer-at-its-own-buying-liability*:

$$\text{calculate-offer-amount-from-value } 3 \ 2 \ 2999 \ 4498 = \text{Cxx-Ok } 2999$$

lemma *p28-amount-loses-a-unit-at-its-own-buying-liability*:

$$\begin{aligned} & (\text{calculate-offer-value } 3 \ 2 \ 2999 \ 4498 \gg= \\ & \quad (\lambda \text{offer-value.} \\ & \quad \quad \text{big-divide-or-throw128 } \text{offer-value } 3 \text{ Cxx-Round-Down})) = \\ & \text{Cxx-Ok } 2998 \end{aligned}$$

The adjustment example uses price $7/3$ with a send cap of 428 and a receive cap of 998 . The plain send and receive values are 2996 and 2994 , so protocol 28 divides 2994 and returns 427 ; protocol 29 relaxes the receive product to 2996 and returns 428 .

lemma *p29-adjustment-keeps-the-full-send-cap:*

calculate-offer-amount-from-value 7 3 428 998 = Cxx-Ok 428

lemma *p28-adjustment-reduces-the-send-cap:*

*(calculate-offer-value 7 3 428 998 $\gg$
 (λ offer-value.
 big-divide-or-throw128 offer-value 7 Cxx-Round-Down)) =
 Cxx-Ok 427*

At an unlimited receive cap the retained clamp makes the two protocols agree, which is the property the liability definitions rely on.

lemma *p29-amount-agrees-with-p28-at-unlimited-receive-cap:*

*calculate-offer-amount-from-value 3 2 2999 int64-max = Cxx-Ok 2999
 (calculate-offer-value 3 2 2999 int64-max $\gg$
 (λ offer-value.
 big-divide-or-throw128 offer-value 3 Cxx-Round-Down)) =
 Cxx-Ok 2999*

2.1.8 Exchange before price-error thresholds

definition *signed-min64* :: *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64*

— C++: *std::min* on *int64-t*

where

signed-min64 a b = (if sint a $\leq$ sint b then a else b)

signed-min64 models C++ *std::min* on *int64-t* operands: the minimum of the two bit patterns in the signed order.

definition *exchange-v10-amounts* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *uint128* $\Rightarrow$ *uint128* $\Rightarrow$
int64 $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *bool* $\Rightarrow$
exchange-rounding $\Rightarrow$ *exchange-options* $\Rightarrow$
(int64 $\times$ *int64)* *cxx-result*

— C++: calculation block of *exchangeV10WithoutPriceErrorThresholds* (*Offer-Exchange.cpp*)

where

*exchange-v10-amounts price-n price-d wheat-value sheep-value
 max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
 wheat-stays rounding options =
 (if wheat-stays then
 if rounding = Exchange-Strict-Send then do {
 wheat-receive $\leftarrow$ big-divide-or-throw128 sheep-value
 (scast price-n) Cxx-Round-Down;*

```

    Cxx-Ok
    (wheat-receive, signed-min64 max-sheep-send max-sheep-receive)
  } else if sint price-n > sint price-d ∨
    rounding = Exchange-Strict-Receive
  then do {
    wheat-receive ← big-divide-or-throw128 sheep-value
    (scast price-n) Cxx-Round-Down;
    sheep-send ← big-divide-or-throw wheat-receive
    (scast price-n) (scast price-d) Cxx-Round-Up;
    Cxx-Ok (wheat-receive, sheep-send)
  } else do {
    sheep-send ←
      (if symmetric-exact-receive-cap options
       then calculate-offer-amount-from-value price-d price-n
         max-sheep-send max-wheat-receive
       else big-divide-or-throw128 sheep-value
         (scast price-d) Cxx-Round-Down);
    wheat-receive ← big-divide-or-throw sheep-send
    (scast price-d) (scast price-n) Cxx-Round-Down;
    Cxx-Ok (wheat-receive, sheep-send)
  }
else if sint price-n > sint price-d then do {
  wheat-receive ←
    (if exact-receive-cap options
     then calculate-offer-amount-from-value price-n price-d
       max-wheat-send max-sheep-receive
     else big-divide-or-throw128 wheat-value
       (scast price-n) Cxx-Round-Down);
  sheep-send ← big-divide-or-throw wheat-receive
  (scast price-n) (scast price-d) Cxx-Round-Down;
  Cxx-Ok (wheat-receive, sheep-send)
} else do {
  sheep-send ← big-divide-or-throw128 wheat-value
  (scast price-d) Cxx-Round-Down;
  wheat-receive ← big-divide-or-throw sheep-send
  (scast price-d) (scast price-n) Cxx-Round-Up;
  Cxx-Ok (wheat-receive, sheep-send)
})

```

exchange-v10-amounts is the *wheatReceive/sheepSend* calculation block of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*). Separating the tuple-valued block lets the top-level model retain the C++ sequencing while local proofs analyze each division branch independently. The final options argument carries the two C++ parameters *exactReceiveCap* and *symmetricExactReceiveCap*. When *exact-receive-cap* is set and the sheep offer stays with wheat the more valuable asset, the trade is valued by *calculate-offer-value-with-exact-receive-cap* applied to the resting wheat offer's send and receive caps instead of the plain wheat value, exactly as in the

C++ conditional expression. When *symmetric-exact-receive-cap* is set and the wheat offer stays with sheep at least as valuable, the mirror call *calculate-offer-value-with-exact-receive-cap price-d price-n max-sheep-send max-wheat-receive* replaces the plain sheep value, so the taker's wheat-receive cap is treated as an exact unit cap before the round-down division by *sint price-d*. Both branches keep *wheatStays* decided on the two unadjusted offer values.

definition *exchange-v10-without-price-error-thresholds-with-options* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *exchange-rounding* $\Rightarrow$ *exchange-options* $\Rightarrow$
exchange-result-v10 cxx-result

— C++: *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*)

where

exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options = do {
wheat-value $\leftarrow$ *calculate-offer-value price-n price-d*
max-wheat-send max-sheep-receive;
sheep-value $\leftarrow$ *calculate-offer-value price-d price-n*
max-sheep-send max-wheat-receive;
let wheat-stays = *wheat-value* > *sheep-value*;
amounts $\leftarrow$ *exchange-v10-amounts price-n price-d wheat-value sheep-value*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
wheat-stays rounding options;
let wheat-receive = *fst amounts*;
let sheep-send = *snd amounts*;
if sint wheat-receive < 0 $\vee$
sint wheat-receive >
min (sint max-wheat-receive) (sint max-wheat-send)
then Cxx-Err Cxx-Runtime-Error
else if sint sheep-send < 0 $\vee$
sint sheep-send >
min (sint max-sheep-receive) (sint max-sheep-send)
then Cxx-Err Cxx-Runtime-Error
else Cxx-Ok
(make-exchange-result wheat-receive sheep-send wheat-stays)
}

exchange-v10-without-price-error-thresholds-with-options is the direct model of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*): the two offer values, the retained-offer decision, the branch block modeled by *exchange-v10-amounts*, and the final bounds checks on both amounts.

definition *check-exchange-v10-result* ::

int64 $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *bool* $\Rightarrow$
(int64 $\times$ int64) $\Rightarrow$ exchange-result-v10 cxx-result

— C++: bounds checks of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*)

where

check-exchange-v10-result max-wheat-send max-wheat-receive max-sheep-send

```

    max-sheep-receive wheat-stays amounts =
  (let wheat-receive = fst amounts;
    sheep-send = snd amounts
  in if sint wheat-receive < 0 ∨
    sint wheat-receive >
      min (sint max-wheat-receive) (sint max-wheat-send)
    then Cxx-Err Cxx-Runtime-Error
    else if sint sheep-send < 0 ∨
      sint sheep-send >
        min (sint max-sheep-receive) (sint max-sheep-send)
      then Cxx-Err Cxx-Runtime-Error
      else Cxx-Ok
    (make-exchange-result wheat-receive sheep-send wheat-stays))

```

check-exchange-v10-result restates the final bounds checks of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*) as a separate function so that the specification below can reuse them verbatim.

definition *exchange-v10-without-price-error-thresholds-spec* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ int64 ⇒
int64 ⇒ exchange-rounding ⇒ exchange-options ⇒
exchange-result-v10 cxx-result

```

where

```

exchange-v10-without-price-error-thresholds-spec price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options =
  (if calculate-offer-value-pre price-n price-d max-wheat-send
    max-sheep-receive ∧
    calculate-offer-value-pre price-d price-n max-sheep-send
    max-wheat-receive
  then
    let wheat-value =
      min
        (word-of-int (sint max-wheat-send * sint price-n) ::
          uint128)
        (word-of-int (sint max-sheep-receive * sint price-d));
      sheep-value =
      min
        (word-of-int (sint max-sheep-send * sint price-d) ::
          uint128)
        (word-of-int (sint max-wheat-receive * sint price-n));
      wheat-stays = wheat-value > sheep-value
    in exchange-v10-amounts price-n price-d wheat-value sheep-value
      max-wheat-send max-wheat-receive max-sheep-send
      max-sheep-receive wheat-stays rounding options ≫=
      check-exchange-v10-result max-wheat-send max-wheat-receive
      max-sheep-send max-sheep-receive wheat-stays
    else Cxx-Err Cxx-Assertion-Failed)

```

The remaining definitions of this subsection describe the same calculation

at the level of unbounded integers.

definition *exchange-v10-pre* ::

$int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $int64 \Rightarrow bool$

where

exchange-v10-pre price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive $\longleftrightarrow$
 $0 < sint\ price-n \wedge 0 < sint\ price-d \wedge$
 $0 \leq sint\ max-wheat-send \wedge 0 \leq sint\ max-wheat-receive \wedge$
 $0 \leq sint\ max-sheep-send \wedge 0 \leq sint\ max-sheep-receive$

definition *exchange-wheat-value-int* ::

$int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int$

where

exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive =
 $min\ (sint\ max-wheat-send * sint\ price-n)$
 $(sint\ max-sheep-receive * sint\ price-d)$

definition *exchange-sheep-value-int* ::

$int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int$

where

exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive =
 $min\ (sint\ max-sheep-send * sint\ price-d)$
 $(sint\ max-wheat-receive * sint\ price-n)$

definition *exchange-v10-amounts-int* ::

$int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $int64 \Rightarrow exchange-rounding \Rightarrow int \times int$

where

exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding =
 $(let\ wheat-value =$
 $\quad exchange-wheat-value-int\ price-n\ price-d\ max-wheat-send$
 $\quad max-sheep-receive;$
 $\quad sheep-value =$
 $\quad exchange-sheep-value-int\ price-n\ price-d\ max-sheep-send$
 $\quad max-wheat-receive;$
 $\quad wheat-stays = wheat-value > sheep-value$
 $in\ if\ wheat-stays\ then$
 $\quad if\ rounding = Exchange-Strict-Send\ then$
 $\quad\quad (sheep-value\ div\ sint\ price-n,$
 $\quad\quad\quad min\ (sint\ max-sheep-send)\ (sint\ max-sheep-receive))$
 $\quad else\ if\ sint\ price-n > sint\ price-d \vee$
 $\quad\quad rounding = Exchange-Strict-Receive$
 $\quad then$
 $\quad\quad let\ wheat-receive = sheep-value\ div\ sint\ price-n$
 $\quad\quad in\ (wheat-receive,$

```

      (wheat-receive * sint price-n + sint price-d - 1)
      div sint price-d)
else
  let sheep-send = sheep-value div sint price-d
  in ((sheep-send * sint price-d) div sint price-n, sheep-send)
else if sint price-n > sint price-d then
  let wheat-receive = wheat-value div sint price-n
  in (wheat-receive,
      (wheat-receive * sint price-n) div sint price-d)
else
  let sheep-send = wheat-value div sint price-d
  in ((sheep-send * sint price-d + sint price-n - 1)
      div sint price-n,
      sheep-send))

```

exchange-v10-pre is the precondition under which the model is claimed to be free of assertions and overflow, and *exchange-v10-amounts-int* is the pair of amounts that the calculation is then claimed to produce.

2.1.9 Full exchange

definition *exchange-v10-with-options* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ int64 ⇒
int64 ⇒ exchange-rounding ⇒ exchange-options ⇒
exchange-result-v10 cxx-result

```

— C++: *exchangeV10* (*OfferExchange.cpp*)

where

```

exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding options = do {
before-thresholds ←
exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options;
apply-price-error-thresholds price-n price-d
(num-wheat-received before-thresholds)
(num-sheep-send before-thresholds)
(result-wheat-stays before-thresholds) rounding
}

```

exchange-v10-with-options is the direct model of *exchangeV10* (*OfferExchange.cpp*). Its monadic bind preserves the C++ call order and propagates every explicit error from the pre-threshold calculation before invoking *apply-price-error-thresholds*. The trailing options record carries the C++ parameter *exactReceiveCap* and is forwarded unchanged to the pre-threshold calculation; the threshold pass never inspects it.

2.1.10 Protocol-facing exchange interface

The definitions above are option-parametric proof kernels. They are useful because a proof about all four option settings covers both real protocols and two diagnostic mutants at once, but the settings are not themselves protocol versions. The layer below is the implementation-facing interface: it mirrors the C++ signatures, which take a ledger version, and maps that version onto exactly one of the two configurations the protocol can be in.

type-synonym *uint32* = 32 word

definition *protocol-version-starts-from* :: *uint32* $\Rightarrow$ *uint32* $\Rightarrow$ *bool*

— C++: *protocolVersionStartsFrom* (*util/ProtocolVersion.cpp*)

where

protocol-version-starts-from *protocol-version* *from-version* $\longleftrightarrow$
uint *from-version* $\leq$ *uint* *protocol-version*

The C++ ledger version is a *uint32-t* and the comparison *protocolVersion* $\geq$ *static-cast<uint32-t>(fromVersion)* is therefore unsigned. The model uses *uint*, not *sint*, so a version with the high bit set compares as a large version rather than a negative one, exactly as in C++.

definition *protocol-version-v29* :: *uint32*

— C++: *ProtocolVersion::V-29* (*util/ProtocolVersion.h*)

where *protocol-version-v29* = 29

definition *legacy-exchange-options* :: *exchange-options*

where

legacy-exchange-options =
(*exact-receive-cap* = *False*,
symmetric-exact-receive-cap = *False*)

definition *repaired-exchange-options* :: *exchange-options*

where

repaired-exchange-options =
(*exact-receive-cap* = *True*,
symmetric-exact-receive-cap = *True*)

definition *exchange-options-at-version* :: *uint32* $\Rightarrow$ *exchange-options*

where

exchange-options-at-version *ledger-version* =
(*if* *protocol-version-starts-from* *ledger-version* *protocol-version-v29*
then *repaired-exchange-options*
else *legacy-exchange-options*)

exchange-options-at-version is the whole protocol mapping. Ledger versions below 29 select the legacy consensus arithmetic; version 29 and later select the repaired arithmetic, in which both round-down branches use *calculate-offer-amount-from-value*. The two mixed configurations are proof mutants: they are reachable in the kernel but not through this mapping, as

recorded below.

definition *exchange-v10-without-price-error-thresholds* ::
 $uint32 \Rightarrow int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $int64 \Rightarrow int64 \Rightarrow exchange-rounding \Rightarrow$
 $exchange-result-v10\ cxx-result$
 — C++: *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*)
where
 $exchange-v10-without-price-error-thresholds\ ledger-version\ price-n$
 $price-d\ max-wheat-send\ max-wheat-receive\ max-sheep-send$
 $max-sheep-receive\ rounding =$
 $exchange-v10-without-price-error-thresholds-with-options\ price-n\ price-d$
 $max-wheat-send\ max-wheat-receive\ max-sheep-send\ max-sheep-receive$
 $rounding\ (exchange-options-at-version\ ledger-version)$

definition *exchange-v10* ::
 $uint32 \Rightarrow int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $int64 \Rightarrow int64 \Rightarrow exchange-rounding \Rightarrow$
 $exchange-result-v10\ cxx-result$
 — C++: *exchangeV10* (*OfferExchange.cpp*)
where
 $exchange-v10\ ledger-version\ price-n\ price-d\ max-wheat-send$
 $max-wheat-receive\ max-sheep-send\ max-sheep-receive\ rounding =$
 $exchange-v10-with-options\ price-n\ price-d\ max-wheat-send$
 $max-wheat-receive\ max-sheep-send\ max-sheep-receive\ rounding$
 $(exchange-options-at-version\ ledger-version)$

These two definitions have the argument lists of the C++ functions as protocol 29 leaves them: a leading *uint32-t* ledger version, then the price, the four caps, and the rounding mode. Everything below the interface is unchanged, so the reduction theorems that follow let every existing option-parametric result be specialized to a real protocol.

lemma *exchange-options-at-version-legacy* [simp]:
 $\neg\ protocol-version-starts-from\ ledger-version\ protocol-version-v29 \implies$
 $exchange-options-at-version\ ledger-version = legacy-exchange-options$

lemma *exchange-options-at-version-repaired* [simp]:
 $protocol-version-starts-from\ ledger-version\ protocol-version-v29 \implies$
 $exchange-options-at-version\ ledger-version = repaired-exchange-options$

lemma *exchange-v10-without-price-error-thresholds-legacy*:
 $\neg\ protocol-version-starts-from\ ledger-version\ protocol-version-v29 \implies$
 $exchange-v10-without-price-error-thresholds\ ledger-version\ price-n\ price-d$
 $max-wheat-send\ max-wheat-receive\ max-sheep-send\ max-sheep-receive$
 $rounding =$
 $exchange-v10-without-price-error-thresholds-with-options\ price-n\ price-d$
 $max-wheat-send\ max-wheat-receive\ max-sheep-send\ max-sheep-receive$
 $rounding\ legacy-exchange-options$

lemma *exchange-v10-without-price-error-thresholds-repaired:*
 $\text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10-without-price-error-thresholds ledger-version price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding} =$
 $\text{exchange-v10-without-price-error-thresholds-with-options price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding repaired-exchange-options}$

lemma *exchange-v10-legacy:*
 $\neg \text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10 ledger-version price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive rounding} =$
 $\text{exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive}$
 $\text{max-sheep-send max-sheep-receive rounding legacy-exchange-options}$

lemma *exchange-v10-repaired:*
 $\text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10 ledger-version price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive rounding} =$
 $\text{exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive}$
 $\text{max-sheep-send max-sheep-receive rounding repaired-exchange-options}$

The four lemmas above are the protocol reduction theorems. Protocol 28 and earlier reduce to the legacy kernel and protocol 29 and later to the repaired kernel, so protocol-28 preservation is a theorem about the public interface rather than an appeal to test evidence.

lemma *protocol-boundary-is-at-29:*
 $\text{exchange-options-at-version } 28 = \text{legacy-exchange-options}$
 $\text{exchange-options-at-version } 29 = \text{repaired-exchange-options}$
— The boundary sits exactly between ledger versions 28 and 29.

lemma *mixed-exchange-options-are-unreachable:*
 $\text{exchange-options-at-version ledger-version} \neq$
 $(\text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{False})$
 $\text{exchange-options-at-version ledger-version} \neq$
 $(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{True})$
— No ledger version selects a one-sided repair.

Proof sketch: the mapping returns one of only two records, and neither agrees with a mixed one on both fields, so unfolding the definitions and splitting on the version test closes both goals.

The two single-option configurations remain available to proofs as mutants and counterexample witnesses, but no ledger version reaches them. Any claim about protocol behavior must therefore be stated through *exchange-options-at-version*.

2.1.11 Main properties

This subsection collects the properties that the rest of the theory establishes about the model above. Each is stated here informally, together with the proposition that is actually proved; the proofs follow, one section per modeled function.

Faithfulness. Every modeled C++ function is shown equal, on all bit patterns, to a branch-free specification phrased in exact integers. The smallest instance is the offer value:

```
calculate-offer-value price-n price-d max-send max-receive =
(if calculate-offer-value-pre price-n price-d max-send max-receive
 then Cxx-Ok
  (min (word-of-int (sint max-send * sint price-n))
        (word-of-int (sint max-receive * sint price-d)))
 else Cxx-Err Cxx-Assertion-Failed)
```

The analogous statements for *big-divide-or-throw*, *big-divide-or-throw128*, *check-price-error-bound*, *apply-price-error-thresholds*, and *exchange-v10-without-price-error-thresholds* are proved in their own sections.

Absence of arithmetic failure. Under *exchange-v10-pre* the whole pre-threshold calculation is total: no assertion fails, no 128-bit numerator wraps, and no quotient escapes the signed 64-bit range. What it computes is exactly *exchange-v10-amounts-int*, evaluated in unbounded integers, so the model of *exchange-v10-with-options* reduces to

```
apply-price-error-thresholds price-n price-d
(word-of-int
 (fst (exchange-v10-amounts-int price-n price-d max-wheat-send
                                max-wheat-receive max-sheep-send max-sheep-receive rounding)))
(word-of-int
 (snd (exchange-v10-amounts-int price-n price-d max-wheat-send
                                max-wheat-receive max-sheep-send max-sheep-receive rounding)))
(exchange-sheep-value-int price-n price-d max-sheep-send max-wheat-receive
 < exchange-wheat-value-int price-n price-d max-wheat-send
   max-sheep-receive)
rounding
```

Result contract. Whenever *exchange-v10-with-options* succeeds with the plain receive cap, both returned amounts are non-negative and respect the four caps, and the retained-offer flag is decided by the two offer values:

```
exchange-v10-pre price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive ∧
exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding
(⟦exact-receive-cap = False, symmetric-exact-receive-cap = False⟧ =
Cxx-Ok exchange-result →
```

$$\begin{aligned}
& 0 \leq \text{sint } (\text{num-wheat-received exchange-result}) \wedge \\
& \text{sint } (\text{num-wheat-received exchange-result}) \\
& \leq \min (\text{sint max-wheat-receive}) (\text{sint max-wheat-send}) \wedge \\
& 0 \leq \text{sint } (\text{num-sheep-send exchange-result}) \wedge \\
& \text{sint } (\text{num-sheep-send exchange-result}) \\
& \leq \min (\text{sint max-sheep-receive}) (\text{sint max-sheep-send})
\end{aligned}$$

Contract of a positive trade. When the returned amounts are both positive, the party whose offer is kept is never disadvantaged, and the trade respects the price error bound appropriate to the rounding mode:

```


favored-seller-ok price-n price-d (num-wheat-received exchange-result)
(num-sheep-send exchange-result) (result-wheat-stays exchange-result) ∧
(if rounding = Exchange-Normal
then price-error-bound-spec price-n price-d
  (num-wheat-received exchange-result) (num-sheep-send exchange-result)
  False
else price-error-bound-spec price-n price-d
  (num-wheat-received exchange-result) (num-sheep-send exchange-result)
  True)


```

Outside strict-send mode the two amounts are moreover zero together, so a successful call either trades on both sides or trades nothing.

The properties of this subsection are stated and proved for the plain receive cap, matching the protocol versions the C++ has shipped with; their *exactReceiveCap* counterparts are future work.

2.2 Wide multiplication

The lemmas below characterize the outcomes of *big-multiply* and record that its successful result is the exact integer product.

lemma *big-multiply-nonnegative* [simp]:
assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$
shows *big-multiply* a $b =$
Cxx-Ok (*word-of-int* ($\text{sint } a * \text{sint } b$))

lemma *big-multiply-negative-left* [simp]:
assumes $\text{sint } a < 0$
shows *big-multiply* a $b = \text{Cxx-Err Cxx-Assertion-Failed}$

lemma *big-multiply-negative-right* [simp]:
assumes $\text{sint } b < 0$
shows *big-multiply* a $b = \text{Cxx-Err Cxx-Assertion-Failed}$

lemma *nonnegative-int64-product-fits-uint128*:
fixes a $b :: \text{int64}$
assumes *a-nonnegative*: $0 \leq \text{sint } a$

and *b-nonnegative*: $0 \leq \text{sint } b$
shows $0 \leq \text{sint } a * \text{sint } b$
and $\text{sint } a * \text{sint } b < (2 :: \text{int}) \wedge 128$

lemma *big-multiply-uint-value*:
fixes $a \ b :: \text{int64}$
assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$
shows $\text{uint } (\text{word-of-int } (\text{sint } a * \text{sint } b) :: \text{uint128}) =$
 $\text{sint } a * \text{sint } b$

lemma *sint-scalt-int32-int64 [simp]*:
 $\text{sint } (\text{scalt } x :: \text{int64}) = \text{sint } (x :: \text{int32})$

2.3 Checked division

The lemmas below determine, for every bit pattern, which of the three outcomes of *big-divide-or-throw* and *big-divide-or-throw128* occurs, and they identify the word returned on success with the exact rounded quotient.

lemma *rounded-quotient-nonnegative*:
assumes *numerator-nonnegative*: $0 \leq \text{numerator}$
and *divisor-positive*: $0 < \text{divisor}$
shows $0 \leq \text{rounded-quotient numerator divisor rounding}$

Proof sketch: split on the rounding constructor. The round-down numerator is non-negative directly; the round-up numerator remains non-negative because the positive divisor is at least one. Integer division by a positive value preserves non-negativity.

lemma *sint-word-of-int-nonnegative-int64*:
assumes *value-nonnegative*: $0 \leq \text{value}$
and *value-bounded*: $\text{value} \leq \text{int64-max-int}$
shows $\text{sint } (\text{word-of-int value} :: \text{int64}) = \text{value}$

Proof sketch: the assumptions place the value inside the non-negative half of the signed 64-bit range, so signed truncation is the identity.

lemma *nonnegative-int64-round-up-numerator-fits-uint128*:
fixes $a \ b \ c :: \text{int64}$
assumes *a-nonnegative*: $0 \leq \text{sint } a$
and *b-nonnegative*: $0 \leq \text{sint } b$
and *c-positive*: $0 < \text{sint } c$
shows $\text{sint } a * \text{sint } b + \text{sint } c - 1 < (2 :: \text{int}) \wedge 128$

Proof sketch: each non-negative multiplicand is below two to the sixty-third power, so their product is below two to the one-hundred-twenty-sixth power.

Adding a signed 64-bit divisor still leaves the round-up numerator strictly below the unsigned 128-bit limit.

lemma *uint128-round-up-numerator-fits*:

assumes *guard*:

$\text{uint } a \leq (2 :: \text{int}) ^{128} - 1 - (\text{sint } b - 1)$

shows $\text{uint } a + \text{sint } b - 1 < (2 :: \text{int}) ^{128}$

Proof sketch: rearranging the explicit C++ guard bounds the incremented numerator by the largest unsigned 128-bit integer, hence strictly below the modulus.

lemma *big-divide-or-throw-success*:

assumes *a-nonnegative*: $0 \leq \text{sint } a$

and *b-nonnegative*: $0 \leq \text{sint } b$

and *c-positive*: $0 < \text{sint } c$

and *quotient-bounded*:

$\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding}$
 $\leq \text{int64-max-int}$

shows *big-divide-or-throw* $a \ b \ c \ \text{rounding} =$

Cxx-Ok (*word-of-int*

$(\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding}))$

and *sint* (*word-of-int*

$(\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding}) ::$
 $\text{int64}) =$

$\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding}$

Proof sketch: valid operands make the exact numerator and quotient non-negative. The quotient bound selects the successful branch, and the signed-word range lemma shows that the returned bit pattern denotes the same mathematical quotient.

lemma *big-divide-or-throw-assertion-iff*:

big-divide-or-throw $a \ b \ c \ \text{rounding} =$

Cxx-Err Cxx-Assertion-Failed

$\longleftrightarrow \text{sint } a < 0 \vee \text{sint } b < 0 \vee \text{sint } c \leq 0$

Proof sketch: unfold the checked helper. Only its initial validity test constructs an assertion error; later branches construct either overflow or a successful word.

lemma *big-divide-or-throw-overflow-iff*:

assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$ **and** $0 < \text{sint } c$

shows *big-divide-or-throw* *a b c rounding* = *Cxx-Err Cxx-Overflow*
 $\longleftrightarrow$ *rounded-quotient* (*sint a* * *sint b*) (*sint c*) *rounding*
 $>$ *int64-max-int*

Proof sketch: under the validity conditions the assertion branch is impossible, leaving overflow exactly when the mathematical quotient does not fit in a signed 64-bit result.

theorem *big-divide-or-throw-success-iff*:
big-divide-or-throw *a b c rounding* = *Cxx-Ok result*
 $\longleftrightarrow$ $0 \leq \text{sint } a \wedge 0 \leq \text{sint } b \wedge 0 < \text{sint } c \wedge$
 $\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding}$
 $\leq \text{int64-max-int} \wedge$
 $\text{result} = \text{word-of-int}$
 $(\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding})$

Proof sketch: unfold both checked branches. Equality with a successful result forces valid operands and a bounded quotient; conversely those conditions select the successful constructor with the exact word value.

lemma *big-divide-or-throw128-success*:
assumes *divisor-positive*: $0 < \text{sint } b$
and *no-round-up-overflow*:
 $\neg (\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{uint } a > (2 :: \text{int}) \wedge 128 - 1 - (\text{sint } b - 1))$
and *quotient-bounded*:
 $\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \text{ rounding}$
 $\leq \text{int64-max-int}$
shows *big-divide-or-throw128* *a b rounding* =
 $\text{Cxx-Ok } (\text{word-of-int}$
 $(\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \text{ rounding}))$
and *sint* (*word-of-int*
 $(\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \text{ rounding}) ::$
 $\text{int64}) =$
 $\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \text{ rounding}$

Proof sketch: a word has a non-negative unsigned value and the divisor is positive, so the exact quotient is non-negative. The two remaining assumptions bypass both overflow checks, and the signed-word range lemma identifies the returned word with that quotient.

lemma *big-divide-or-throw128-assertion-iff*:
big-divide-or-throw128 *a b rounding* =
 $\text{Cxx-Err Cxx-Assertion-Failed}$
 $\longleftrightarrow \text{sint } b \leq 0$

Proof sketch: the divisor check is the only assertion-producing branch; the explicit numerator guard and result-range check both report overflow.

lemma *big-divide-or-throw128-overflow-iff*:

assumes $0 < \text{sint } b$

shows $\text{big-divide-or-throw128 } a \ b \ \text{rounding} = \text{Cxx-Err } \text{Cxx-Overflow}$

$\longleftrightarrow$

$(\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{uint } a > (2 :: \text{int}) ^{128} - 1 - (\text{sint } b - 1))$

$\vee$

$(\neg (\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{uint } a > (2 :: \text{int}) ^{128} - 1 - (\text{sint } b - 1)) \wedge$
 $\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \ \text{rounding}$
 $> \text{int64-max-int})$

Proof sketch: after the positive-divisor check, overflow arises either from the explicit round-up increment guard or, when that guard is clear, from narrowing a quotient above the signed 64-bit maximum.

theorem *big-divide-or-throw128-success-iff*:

$\text{big-divide-or-throw128 } a \ b \ \text{rounding} = \text{Cxx-Ok } \text{result}$

$\longleftrightarrow 0 < \text{sint } b \wedge$

$\neg (\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{uint } a > (2 :: \text{int}) ^{128} - 1 - (\text{sint } b - 1)) \wedge$
 $\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \ \text{rounding}$
 $\leq \text{int64-max-int} \wedge$
 $\text{result} = \text{word-of-int}$
 $(\text{rounded-quotient } (\text{uint } a) (\text{sint } b) \ \text{rounding})$

Proof sketch: unfolding the definition shows that success is equivalent to passing the divisor assertion, the round-up increment guard, and the signed result bound, after which the returned word is fixed uniquely.

2.4 Price error bound

The lemmas below show that *check-price-error-bound* fails an assertion exactly when one of its four numeric arguments is negative, and that it otherwise returns the mathematical predicate *price-error-bound-spec*.

lemma *sint-price-scale*:

fixes $\text{price} :: \text{int32}$

shows $\text{sint } (\text{word-of-int } (100 * \text{sint } \text{price}) :: \text{int64}) =$
 $100 * \text{sint } \text{price}$

Proof sketch: the signed range of a 32-bit word, after multiplication by one hundred, is strictly inside the signed 64-bit range. Consequently signed truncation to 64 bits is the identity.

lemma *uint-word-abs-diff*:
fixes $a\ b :: \text{uint128}$
shows $\text{uint } (if\ a > b\ then\ a - b\ else\ b - a) =$
 $\text{abs } (\text{uint } a - \text{uint } b)$

Proof sketch: split on the unsigned ordering of the words. In each branch the larger value is the minuend, so unsigned subtraction does not wrap and agrees with the corresponding non-negative integer difference.

lemma *nonnegative-price-error-product-fits-uint128*:
fixes $\text{price} :: \text{int32}$ **and** $\text{amount} :: \text{int64}$
assumes $\text{price-nonnegative}: 0 \leq \text{sint price}$
and $\text{amount-nonnegative}: 0 \leq \text{sint amount}$
shows $0 \leq 100 * \text{sint price} * \text{sint amount}$
and $100 * \text{sint price} * \text{sint amount} < (2 :: \text{int}) \wedge 128$

Proof sketch: the scaled price is an exact, non-negative signed 64-bit value by *sint-price-scale*. The previously proved bound for multiplying two non-negative signed 64-bit words then supplies both claims.

lemma *price-error-product-uint-value*:
fixes $\text{price} :: \text{int32}$ **and** $\text{amount} :: \text{int64}$
assumes $\text{price-nonnegative}: 0 \leq \text{sint price}$
and $\text{amount-nonnegative}: 0 \leq \text{sint amount}$
shows $\text{uint } (\text{word-of-int } (100 * \text{sint price} * \text{sint amount})) ::$
 $\text{uint128} = 100 * \text{sint price} * \text{sint amount}$

Proof sketch: interpret the scaled price as a signed 64-bit word and apply the exact unsigned value theorem for *big-multiply*; the scaling lemma then rewrites the result to mathematical integers.

lemma *check-price-error-bound-success-words*:
assumes $\text{price-n-nonnegative}: 0 \leq \text{sint price-n}$
and $\text{price-d-nonnegative}: 0 \leq \text{sint price-d}$
and $\text{wheat-nonnegative}: 0 \leq \text{sint wheat-receive}$
and $\text{sheep-nonnegative}: 0 \leq \text{sint sheep-send}$
shows $\text{check-price-error-bound price-n price-d wheat-receive sheep-send}$
 $\text{can-favor-wheat} =$
 Cxx-Ok
 $(\text{let lhs} = \text{word-of-int}$
 $(100 * \text{sint price-n} * \text{sint wheat-receive}) :: \text{uint128};$
 $\text{rhs} = \text{word-of-int}$
 $(100 * \text{sint price-d} * \text{sint sheep-send}) :: \text{uint128};$

```

    cap = word-of-int
    (sint price-n * sint wheat-receive) :: uint128;
    abs-diff = (if lhs > rhs then lhs - rhs else rhs - lhs)
    in (can-favor-wheat ∧ rhs > lhs) ∨ abs-diff ≤ cap)

```

Proof sketch: non-negativity makes all three calls of *big-multiply* succeed. Substitute their exact word results into the monadic computation; the early return is equivalent to disjoining its condition with the ordinary bound check.

theorem *check-price-error-bound-integer-characterization:*

```

assumes price-n-nonnegative: 0 ≤ sint price-n
and price-d-nonnegative: 0 ≤ sint price-d
and wheat-nonnegative: 0 ≤ sint wheat-receive
and sheep-nonnegative: 0 ≤ sint sheep-send
shows check-price-error-bound price-n price-d wheat-receive sheep-send
    can-favor-wheat =
    Cxx-Ok
    (price-error-bound-spec price-n price-d wheat-receive sheep-send
    can-favor-wheat)

```

Proof sketch: use the word-level success lemma, then replace the unsigned values of the two scaled products and the cap by their exact integer values. The unsigned absolute difference is exact because the model always subtracts the smaller word from the larger one.

theorem *check-price-error-bound-negative:*

```

assumes negative:
    sint price-n < 0 ∨ sint price-d < 0 ∨
    sint wheat-receive < 0 ∨ sint sheep-send < 0
shows check-price-error-bound price-n price-d wheat-receive sheep-send
    can-favor-wheat = Cxx-Err Cxx-Assertion-Failed

```

Proof sketch: exact signed scaling preserves the sign of each price component. Follow the monadic evaluation order: the first negative operand encountered by *big-multiply* produces the assertion error, which the bind propagates unchanged.

theorem *check-price-error-bound-characterization:*

```

check-price-error-bound price-n price-d wheat-receive sheep-send
    can-favor-wheat =
    (if 0 ≤ sint price-n ∧ 0 ≤ sint price-d ∧
    0 ≤ sint wheat-receive ∧ 0 ≤ sint sheep-send
    then Cxx-Ok
    (price-error-bound-spec price-n price-d wheat-receive sheep-send

```

can-favor-wheat)
else Cxx-Err Cxx-Assertion-Failed)

Proof sketch: split on the weakest successful-input condition. The non-negative branch is the exact integer characterization; negating that conjunction exposes a negative operand, so the assertion theorem applies.

2.5 Price-error application

The lemmas below replace the monadic branch structure of *apply-price-error-thresholds* by the flat decision tree *apply-price-error-thresholds-spec*, and derive what a successful call guarantees about the record it returns.

lemma *apply-price-error-thresholds-nonpositive*:

assumes *nonpositive*:

$\neg (0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send})$

shows

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding =
(if rounding = Exchange-Strict-Send then
if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
else Cxx-Ok
(make-exchange-result wheat-receive sheep-send wheat-stays)
else Cxx-Ok (make-exchange-result 0 0 wheat-stays))

Proof sketch: the failed positivity guard selects the final C++ branch; simplifying only that guard leaves the strict-send zero test unchanged.

lemma *apply-price-error-thresholds-negative-price*:

assumes *amounts-positive*:

$0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$

and *negative-price*: $\text{sint price-n} < 0 \vee \text{sint price-d} < 0$

shows

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Err Cxx-Assertion-Failed

Proof sketch: positivity reaches both multiplications. Sign extension preserves each price sign, so the first negative price reached by *big-multiply* produces the modeled assertion and monadic bind propagates it.

theorem *apply-price-error-thresholds-positive-characterization*:

assumes *amounts-positive*:

$0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$

and *price-n-nonnegative*: $0 \leq \text{sint price-n}$

and *price-d-nonnegative*: $0 \leq \text{sint price-d}$

shows

```

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
  wheat-stays rounding =
  (let original =
    make-exchange-result wheat-receive sheep-send wheat-stays;
    zero = make-exchange-result 0 0 wheat-stays
    in if ¬ favored-seller-ok price-n price-d wheat-receive sheep-send
      wheat-stays
      then Cxx-Err Cxx-Runtime-Error
      else if rounding = Exchange-Normal then
        Cxx-Ok
        (if price-error-bound-spec price-n price-d wheat-receive
          sheep-send False
          then original else zero)
      else if price-error-bound-spec price-n price-d wheat-receive
        sheep-send True
        then Cxx-Ok original
        else Cxx-Err Cxx-Runtime-Error)

```

Proof sketch: the non-negative multiplication contracts replace both 128-bit words by their exact integer products. Unsigned word comparisons then become the favored-seller inequality, and the already-proved price-error contract replaces each checked bound call.

theorem *apply-price-error-thresholds-characterization:*

```

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
  wheat-stays rounding =
  apply-price-error-thresholds-spec price-n price-d wheat-receive sheep-send
  wheat-stays rounding

```

Proof sketch: partition all bit patterns first by the amount-positivity guard and then by price signs. The preceding three branch theorems cover the non-positive, assertion, and fully non-negative cases respectively.

theorem *apply-price-error-thresholds-preserves-wheat-stays:*

assumes *result:*

```

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
  wheat-stays rounding = Cxx-Ok exchange-result

```

shows *result-wheat-stays exchange-result = wheat-stays*

Proof sketch: rewrite the executable function to its total branch characterization. Every successful constructor in that specification uses the input stay flag.

theorem *apply-price-error-thresholds-normal-result:*

assumes *result:*

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays Exchange-Normal = Cxx-Ok exchange-result
shows
exchange-result =
make-exchange-result wheat-receive sheep-send wheat-stays ∨
exchange-result = make-exchange-result 0 0 wheat-stays

Proof sketch: after rewriting with the total characterization, all normal success branches select either the original record or the explicit zero-amount record.

theorem *apply-price-error-thresholds-positive-path-result:*
assumes *amounts-positive:*
 $0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$
and *path-mode:* $\text{rounding} \neq \text{Exchange-Normal}$
and *result:*
apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result
shows
exchange-result =
make-exchange-result wheat-receive sheep-send wheat-stays

Proof sketch: under the positive guard, every non-normal successful branch of the total characterization returns the original amount record; all alternatives are explicit errors.

theorem *apply-price-error-thresholds-positive-success-favored:*
assumes *amounts-positive:*
 $0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$
and *result:*
apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result
shows
favored-seller-ok price-n price-d wheat-receive sheep-send wheat-stays

Proof sketch: a positive call can return successfully only after passing the exact favored-seller comparison in the total characterization. Negative prices and the opposite direction are error branches.

theorem *apply-price-error-thresholds-positive-trade-bound:*
assumes *amounts-positive:*
 $0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$
and *result:*
apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result

and *result-positive*:
 $0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \wedge$
 $0 < \text{sint } (\text{num-sheep-send } \text{exchange-result})$
shows
if *rounding* = *Exchange-Normal*
then *price-error-bound-spec price-n price-d wheat-receive sheep-send* *False*
else *price-error-bound-spec price-n price-d wheat-receive sheep-send* *True*

Proof sketch: normal mode may evade its bound only by returning the zero-amount record, which the positive-result assumption excludes. Non-normal success already requires the one-sided bound.

2.6 Offer value

The lemmas below show that *calculate-offer-value* succeeds exactly under *calculate-offer-value-pre*, and that the unsigned 128-bit word it then returns denotes the exact integer $\min (\text{sint } \text{max-send} * \text{sint } \text{price-n}) (\text{sint } \text{max-receive} * \text{sint } \text{price-d})$.

lemma *calculate-offer-value-success*:

assumes *calculate-offer-value-pre price-n price-d max-send max-receive*

shows *calculate-offer-value price-n price-d max-send max-receive* =

Cxx-Ok

(*min*

(*word-of-int* (*sint max-send * sint price-n*) :: *uint128*)

(*word-of-int* (*sint max-receive * sint price-d*) :: *uint128*))

lemma *uint-min-word*:

uint (*min a b*) = *min* (*uint a*) (*uint b*)

for *a b* :: '*a*::*len* *word*

theorem *calculate-offer-value-integer-characterization*:

assumes *pre*:

calculate-offer-value-pre price-n price-d max-send max-receive

obtains *v* **where**

calculate-offer-value price-n price-d max-send max-receive = *Cxx-Ok v*

and

uint v =

min (*sint max-send * sint price-n*)

(*sint max-receive * sint price-d*)

lemma *calculate-offer-value-example*:

calculate-offer-value

(*3* :: *int32*) (*2* :: *int32*)

(*10* :: *int64*) (*10* :: *int64*) =

Cxx-Ok (*20* :: *uint128*)

theorem *calculate-offer-value-characterization:*

```

calculate-offer-value price-n price-d max-send max-receive =
  (if calculate-offer-value-pre price-n price-d max-send max-receive
    then Cxx-Ok
    (min
      (word-of-int (sint max-send * sint price-n) :: uint128)
      (word-of-int (sint max-receive * sint price-d) :: uint128))
    else Cxx-Err Cxx-Assertion-Failed)

```

Proof sketch: unfold both multiplications. The conjunction in *calculate-offer-value-pre* is exactly the condition under which neither sequential *big-multiply* call asserts; on that branch both exact product words reach the unsigned minimum.

2.7 Exchange before price-error thresholds

The lemmas below show that, under *exchange-v10-pre*, no assertion, overflow, or runtime error can arise in *exchange-v10-without-price-error-thresholds-with-options*, and that the pair of amounts it returns is exactly *exchange-v10-amounts-int* evaluated in unbounded integer arithmetic. This is the longest development in the theory: each of the five division branches is bounded separately before the branches are recombined.

lemma *exchange-v10-without-as-checked-amounts:*

```

exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options = do {
  wheat-value ← calculate-offer-value price-n price-d
  max-wheat-send max-sheep-receive;
  sheep-value ← calculate-offer-value price-d price-n
  max-sheep-send max-wheat-receive;
  let wheat-stays = wheat-value > sheep-value;
  amounts ← exchange-v10-amounts price-n price-d wheat-value sheep-value
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  wheat-stays rounding options;
  check-exchange-v10-result max-wheat-send max-wheat-receive
  max-sheep-send max-sheep-receive wheat-stays amounts
}

```

Proof sketch: unfold the top-level model and the factored final checker. The tuple projections and two ordered bounds tests are definitionally the same as the explicit tail of the executable model.

theorem *exchange-v10-without-price-error-thresholds-characterization:*

```

exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options =
  exchange-v10-without-price-error-thresholds-spec price-n price-d

```

max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options

Proof sketch: replace both sequential offer-value calls with their total characterizations. A failed precondition produces the first reached assertion; otherwise monadic simplification leaves precisely the two exact value words, the amount block, and the final checker in the specification.

lemma *int-div-upper-bound*:
fixes *numerator divisor bound :: int*
assumes *divisor-positive*: $0 < \text{divisor}$
and *numerator-bound*: $\text{numerator} \leq \text{bound} * \text{divisor}$
shows $\text{numerator} \text{ div } \text{divisor} \leq \text{bound}$

Proof sketch: integer division by a positive divisor is monotone in the numerator. Compare with $\text{bound} * \text{divisor}$ and cancel the exact multiple.

lemma *int-ceiling-div-upper-bound*:
fixes *numerator divisor bound :: int*
assumes *divisor-positive*: $0 < \text{divisor}$
and *numerator-nonnegative*: $0 \leq \text{numerator}$
and *numerator-bound*: $\text{numerator} \leq \text{bound} * \text{divisor}$
shows $(\text{numerator} + \text{divisor} - 1) \text{ div } \text{divisor} \leq \text{bound}$

Proof sketch: the assumptions force a non-negative bound. Add $\text{divisor} - 1$ to the numerator inequality, use division monotonicity, and split the exact multiple from the remainder.

lemma *int-ceiling-div-nonnegative*:
fixes *numerator divisor :: int*
assumes $0 \leq \text{numerator}$ **and** $0 < \text{divisor}$
shows $0 \leq (\text{numerator} + \text{divisor} - 1) \text{ div } \text{divisor}$

Proof sketch: this expression is *rounded-quotient* in round-up mode, whose non-negativity theorem applies directly.

lemma *int-div-mult-le*:
fixes *numerator divisor :: int*
assumes *divisor-positive*: $0 < \text{divisor}$
shows $\text{numerator} \text{ div } \text{divisor} * \text{divisor} \leq \text{numerator}$

Proof sketch: decompose the numerator into its quotient multiple and remainder. A positive divisor makes the integer remainder non-negative.

lemma *sint64-upper-bound*:
 $\text{sint } (x :: \text{int64}) \leq \text{int64-max-int}$

Proof sketch: instantiate the generic strict upper bound for signed words at width 64 and convert the strict power-of-two bound to the largest signed 64-bit integer.

lemma *signed-min64-word*:
 $\text{signed-min64 } a \ b =$
 $\text{word-of-int } (\min (\text{sint } a) (\text{sint } b))$

Proof sketch: split on the signed comparison used by the definition. Re-encoding the signed interpretation of either selected word returns the original bit pattern.

lemma *big-divide-or-throw128-down-bounded*:
assumes *divisor-positive*: $0 < \text{sint divisor}$
and *value-bound*: $\text{uint value} \leq \text{bound} * \text{sint divisor}$
and *bound-bounded*: $\text{bound} \leq \text{int64-max-int}$
shows *big-divide-or-throw128 value divisor Cxx-Round-Down* =
 $\text{Cxx-Ok } (\text{word-of-int } (\text{uint value div sint divisor}))$
and *sint*
 $(\text{word-of-int } (\text{uint value div sint divisor}) :: \text{int64}) =$
 $\text{uint value div sint divisor}$

Proof sketch: monotonic division turns the supplied product bound into a signed-64 quotient bound. Round-down makes the 128-bit increment guard false, so the existing checked-division success theorem yields both the exact returned word and its signed interpretation.

lemma *exchange-wheat-value-int-bounds*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
 $\text{max-wheat-receive max-sheep-send max-sheep-receive}$
shows $0 \leq \text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 max-sheep-receive
and *exchange-wheat-value-int price-n price-d max-wheat-send*
 $\text{max-sheep-receive} \leq \text{sint max-wheat-send} * \text{sint price-n}$
and *exchange-wheat-value-int price-n price-d max-wheat-send*
 $\text{max-sheep-receive} \leq \text{sint max-sheep-receive} * \text{sint price-d}$

Proof sketch: well-formedness makes both exact products non-negative, and their mathematical minimum is non-negative and below each operand.

lemma *exchange-sheep-value-int-bounds:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

shows $0 \leq \text{exchange-sheep-value-int price-n price-d max-sheep-send max-wheat-receive}$

and *exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive $\leq$ sint max-sheep-send * sint price-d*

and *exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive $\leq$ sint max-wheat-receive * sint price-n*

Proof sketch: this is the symmetric minimum argument for the reversed offer value.

lemma *exchange-wheat-value-word:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

defines *value $\equiv$*

min

*(word-of-int (sint max-wheat-send * sint price-n) :: uint128)
(word-of-int (sint max-sheep-receive * sint price-d))*

shows *uint value =*

exchange-wheat-value-int price-n price-d max-wheat-send max-sheep-receive

Proof sketch: the established multiplication range theorem identifies each 128-bit word with its exact non-negative product; unsigned minimum then commutes with *uint*.

lemma *exchange-sheep-value-word:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

defines *value $\equiv$*

min

*(word-of-int (sint max-sheep-send * sint price-d) :: uint128)
(word-of-int (sint max-wheat-receive * sint price-n))*

shows *uint value =*

exchange-sheep-value-int price-n price-d max-sheep-send max-wheat-receive

Proof sketch: apply the same exact-product and unsigned-minimum argument to the price and maxima in their reversed order.

lemma *exchange-amounts-strict-send-bounds:*

```

fixes price-n sheep-value wheat-value max-wheat-send max-wheat-receive
      max-sheep-send max-sheep-receive :: int
assumes price-positive: 0 < price-n
      and sheep-nonnegative: 0 ≤ sheep-value
      and sheep-receive-bound:
        sheep-value ≤ max-wheat-receive * price-n
      and wheat-stays: sheep-value < wheat-value
      and wheat-send-bound: wheat-value ≤ max-wheat-send * price-n
      and sheep-send-nonnegative: 0 ≤ max-sheep-send
      and sheep-receive-nonnegative: 0 ≤ max-sheep-receive
shows 0 ≤ sheep-value div price-n
      and sheep-value div price-n ≤
        min max-wheat-receive max-wheat-send
      and 0 ≤ min max-sheep-send max-sheep-receive
      and min max-sheep-send max-sheep-receive ≤
        min max-sheep-receive max-sheep-send

```

Proof sketch: floor division preserves non-negativity and each value bound yields one wheat maximum. The strict value ordering supplies the other; the sheep output is the signed minimum directly.

```

lemma exchange-amounts-wheat-stays-up-bounds:
  fixes price-n price-d sheep-value wheat-value max-wheat-send
      max-wheat-receive max-sheep-send max-sheep-receive wheat-receive
      sheep-send :: int
  assumes price-n-positive: 0 < price-n
      and price-d-positive: 0 < price-d
      and sheep-nonnegative: 0 ≤ sheep-value
      and sheep-wheat-receive-bound:
        sheep-value ≤ max-wheat-receive * price-n
      and sheep-sheep-send-bound:
        sheep-value ≤ max-sheep-send * price-d
      and wheat-stays: sheep-value < wheat-value
      and wheat-wheat-send-bound:
        wheat-value ≤ max-wheat-send * price-n
      and wheat-sheep-receive-bound:
        wheat-value ≤ max-sheep-receive * price-d
  defines wheat-receive ≡ sheep-value div price-n
      and sheep-send ≡
        (wheat-receive * price-n + price-d - 1) div price-d
  shows 0 ≤ wheat-receive
      and wheat-receive ≤ min max-wheat-receive max-wheat-send
      and 0 ≤ sheep-send
      and sheep-send ≤ min max-sheep-receive max-sheep-send

```

Proof sketch: floor the sheep value to obtain wheat, then bound its product by the original value. The ceiling bound transfers both sheep maxima; the strict stay ordering transfers the two opposite-offer maxima.

lemma *exchange-amounts-wheat-stays-down-bounds:*
fixes *price-n price-d sheep-value wheat-value max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive wheat-receive
sheep-send :: int
assumes *price-n-positive: 0 < price-n*
and *price-d-positive: 0 < price-d*
and *sheep-nonnegative: 0 ≤ sheep-value*
and *sheep-wheat-receive-bound:*
*sheep-value ≤ max-wheat-receive * price-n*
and *sheep-sheep-send-bound:*
*sheep-value ≤ max-sheep-send * price-d*
and *wheat-stays: sheep-value < wheat-value*
and *wheat-wheat-send-bound:*
*wheat-value ≤ max-wheat-send * price-n*
and *wheat-sheep-receive-bound:*
*wheat-value ≤ max-sheep-receive * price-d*
defines *sheep-send ≡ sheep-value div price-d*
and *wheat-receive ≡ sheep-send * price-d div price-n*
shows *0 ≤ wheat-receive*
and *wheat-receive ≤ min max-wheat-receive max-wheat-send*
and *0 ≤ sheep-send*
and *sheep-send ≤ min max-sheep-receive max-sheep-send*

Proof sketch: this branch floors first in sheep units and then in wheat units. Each quotient product is below its numerator, so the two value minima and the strict stay ordering provide all four maxima.

lemma *exchange-amounts-sheep-stays-wheat-more-bounds:*
fixes *price-n price-d wheat-value sheep-value max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive wheat-receive
sheep-send :: int
assumes *price-n-positive: 0 < price-n*
and *price-d-positive: 0 < price-d*
and *wheat-nonnegative: 0 ≤ wheat-value*
and *wheat-wheat-send-bound:*
*wheat-value ≤ max-wheat-send * price-n*
and *wheat-sheep-receive-bound:*
*wheat-value ≤ max-sheep-receive * price-d*
and *sheep-stays: wheat-value ≤ sheep-value*
and *sheep-wheat-receive-bound:*
*sheep-value ≤ max-wheat-receive * price-n*
and *sheep-sheep-send-bound:*
*sheep-value ≤ max-sheep-send * price-d*
defines *wheat-receive ≡ wheat-value div price-n*
and *sheep-send ≡ wheat-receive * price-n div price-d*
shows *0 ≤ wheat-receive*

and $wheat\text{-}receive \leq \min \max\text{-}wheat\text{-}receive \max\text{-}wheat\text{-}send$
and $0 \leq sheep\text{-}send$
and $sheep\text{-}send \leq \min \max\text{-}sheep\text{-}receive \max\text{-}sheep\text{-}send$

Proof sketch: floor the smaller wheat value twice. Its own two product bounds and its ordering below the sheep value transfer the four maxima.

lemma *exchange-amounts-sheep-stays-sheep-more-bounds*:
fixes $price\text{-}n \ price\text{-}d \ wheat\text{-}value \ sheep\text{-}value \ max\text{-}wheat\text{-}send$
 $\max\text{-}wheat\text{-}receive \ max\text{-}sheep\text{-}send \ max\text{-}sheep\text{-}receive \ wheat\text{-}receive$
 $sheep\text{-}send :: int$
assumes $price\text{-}n\text{-}positive: 0 < price\text{-}n$
and $price\text{-}d\text{-}positive: 0 < price\text{-}d$
and $wheat\text{-}nonnegative: 0 \leq wheat\text{-}value$
and $wheat\text{-}wheat\text{-}send\text{-}bound$:
 $wheat\text{-}value \leq \max\text{-}wheat\text{-}send * price\text{-}n$
and $wheat\text{-}sheep\text{-}receive\text{-}bound$:
 $wheat\text{-}value \leq \max\text{-}sheep\text{-}receive * price\text{-}d$
and $sheep\text{-}stays: wheat\text{-}value \leq sheep\text{-}value$
and $sheep\text{-}wheat\text{-}receive\text{-}bound$:
 $sheep\text{-}value \leq \max\text{-}wheat\text{-}receive * price\text{-}n$
and $sheep\text{-}sheep\text{-}send\text{-}bound$:
 $sheep\text{-}value \leq \max\text{-}sheep\text{-}send * price\text{-}d$
defines $sheep\text{-}send \equiv wheat\text{-}value \div price\text{-}d$
and $wheat\text{-}receive \equiv$
 $(sheep\text{-}send * price\text{-}d + price\text{-}n - 1) \div price\text{-}n$
shows $0 \leq wheat\text{-}receive$
and $wheat\text{-}receive \leq \min \max\text{-}wheat\text{-}receive \max\text{-}wheat\text{-}send$
and $0 \leq sheep\text{-}send$
and $sheep\text{-}send \leq \min \max\text{-}sheep\text{-}receive \max\text{-}sheep\text{-}send$

Proof sketch: floor the smaller wheat value to sheep, then apply the ceiling bound to the remaining conversion. The value ordering supplies the maxima originating in the other offer.

theorem *exchange-v10-amounts-int-bounds*:
fixes $amounts :: int \times int$
assumes $pre: exchange\text{-}v10\text{-}pre \ price\text{-}n \ price\text{-}d \ max\text{-}wheat\text{-}send$
 $\max\text{-}wheat\text{-}receive \ max\text{-}sheep\text{-}send \ max\text{-}sheep\text{-}receive$
and $amounts$:
 $amounts = exchange\text{-}v10\text{-}amounts\text{-}int \ price\text{-}n \ price\text{-}d \ max\text{-}wheat\text{-}send$
 $\max\text{-}wheat\text{-}receive \ max\text{-}sheep\text{-}send \ max\text{-}sheep\text{-}receive \ rounding$
shows $0 \leq fst \ amounts$
and $fst \ amounts \leq$
 $\min (sint \ max\text{-}wheat\text{-}receive) (sint \ max\text{-}wheat\text{-}send)$
and $0 \leq snd \ amounts$
and $snd \ amounts \leq$

$\min (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$

Proof sketch: split on which exact offer value is smaller, the rounding constructor, and the price ordering. Each of the five C++ calculation branches is discharged by its preceding arithmetic-bound lemma.

lemma *exchange-v10-amounts-wheat-stays-strict-send:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *sheep-value:*
uint sheep-word =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
Exchange-Strict-Send options =
Cxx-Ok
(word-of-int
(exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive div sint price-n),
word-of-int
(min (sint max-sheep-send) (sint max-sheep-receive)))

Proof sketch: the reversed offer value is bounded by *sint max-wheat-receive* * *sint price-n*, so its round-down division fits signed 64 bits. The second component is exactly the signed minimum.

lemma *exchange-v10-amounts-wheat-stays-up:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *sheep-value:*
uint sheep-word =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
and *not-strict-send: rounding ≠ Exchange-Strict-Send*
and *up-branch:*
sint price-n > sint price-d ∨
rounding = Exchange-Strict-Receive
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
rounding options =
(let wheat-receive =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive div sint price-n;
sheep-send =
*(wheat-receive * sint price-n + sint price-d - 1) div sint price-d*
in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))

Proof sketch: the first round-down quotient is bounded by the wheat-receive maximum. Its product remains below the sheep value, so the round-up quotient is bounded by the sheep-send maximum and both checked helpers return the exact integer formulas.

lemma *exchange-v10-amounts-wheat-stays-down:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *sheep-value:*
uint sheep-word =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
and *not-strict-send: rounding ≠ Exchange-Strict-Send*
and *down-branch:*
 $\neg (\text{sint price-n} > \text{sint price-d} \vee$
 $\text{rounding} = \text{Exchange-Strict-Receive})$
and *plain-cap: $\neg$ symmetric-exact-receive-cap options*
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
rounding options =
(let sheep-send =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive div sint price-d;
*wheat-receive = sheep-send * sint price-d div sint price-n*
in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))

Proof sketch: with the symmetric receive cap disabled the branch divides the plain sheep value. The first floor is bounded by the sheep-send maximum. Its exact product stays below the sheep value, so the second floor is bounded by the wheat-receive maximum and both checked divisions succeed exactly.

lemma *exchange-v10-amounts-sheep-stays-wheat-more:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *wheat-value:*
uint wheat-word =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive
and *wheat-more: sint price-n > sint price-d*
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
rounding ($\neg$ exact-receive-cap = False, symmetric-exact-receive-cap = False) =
(let wheat-receive =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive div sint price-n;
*sheep-send = wheat-receive * sint price-n div sint price-d*

in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))

Proof sketch: divide the wheat value down by the numerator, then divide its exact product down by the denominator. The two defining product bounds of the wheat value keep both narrowing operations in signed-64 range.

lemma *exchange-v10-amounts-sheep-stays-sheep-more:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *wheat-value:*
uint wheat-word =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive
and *sheep-more: $\neg$ sint price-n > sint price-d*
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
rounding options =
(let sheep-send =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive div sint price-d;
wheat-receive =
*(sheep-send * sint price-d + sint price-n - 1) div sint price-n*
in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))

Proof sketch: the first floor is bounded by the sheep-receive maximum. Its product remains below the wheat value, so the final ceiling is bounded by the wheat-send maximum and both checked divisions return exactly.

theorem *exchange-v10-amounts-integer-characterization:*
fixes *rounding :: exchange-rounding*
and *wheat-word sheep-word :: uint128*
and *amounts :: int $\times$ int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *wheat-word $\equiv$*
min
*(word-of-int (sint max-wheat-send * sint price-n) :: uint128)*
*(word-of-int (sint max-sheep-receive * sint price-d))*
and *sheep-word $\equiv$*
min
*(word-of-int (sint max-sheep-send * sint price-d) :: uint128)*
*(word-of-int (sint max-wheat-receive * sint price-n))*
and *amounts $\equiv$*
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive


```

      (wheat-word > sheep-word) rounding (exact-receive-cap = False, symmetric-exact-receive-cap = False) =
    Cxx-Ok
      (word-of-int (fst amounts), word-of-int (snd amounts))
    and sint (word-of-int (fst amounts) :: int64) = fst amounts
    and sint (word-of-int (snd amounts) :: int64) = snd amounts

```

Proof sketch: identify both 128-bit values with their exact integers, then split on the value comparison, rounding mode, and price ordering. The five branch equations above give the exact pair; the global bounds theorem makes both re-encoded result words faithful signed-64 values.

theorem *exchange-v10-without-price-error-thresholds-integer-characterization:*
fixes *rounding :: exchange-rounding and amounts :: int × int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts ≡*
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-without-price-error-thresholds-with-options price-n price-d*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding (exact-receive-cap = False, symmetric-exact-receive-cap = False) =
Cxx-Ok
(make-exchange-result
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive))

Proof sketch: both offer-value calls succeed with their exact 128-bit minima under *exchange-v10-pre*. Substitute the exact amount-block theorem, then use its signed interpretations and four bounds to prove that neither final runtime check is reachable.

lemma *int-ceiling-product-div-eq-zero-iff:*
fixes *x multiplier divisor :: int*
assumes *x-nonnegative: 0 ≤ x*
and *multiplier-positive: 0 < multiplier*
and *divisor-positive: 0 < divisor*
shows *((x * multiplier + divisor - 1) div divisor = 0) =*
(x = 0)

Proof sketch: the zero input simplifies directly. A positive integer input times a positive multiplier is at least one, so the augmented numerator is at least the divisor and monotonic division makes the quotient positive.

lemma *int-floor-product-div-eq-zero-iff*:
fixes x *multiplier divisor* :: *int*
assumes x -*nonnegative*: $0 \leq x$
and *multiplier-at-least-divisor*: $\text{divisor} \leq \text{multiplier}$
and *divisor-positive*: $0 < \text{divisor}$
shows $(x * \text{multiplier} \text{ div } \text{divisor} = 0) = (x = 0)$

Proof sketch: a positive integer input is at least one. Multiplying it by a non-negative multiplier no smaller than the divisor makes the numerator at least the divisor, hence the floor quotient is positive.

theorem *exchange-v10-amounts-zero-iff*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *not-strict-send*: *rounding* $\neq$ *Exchange-Strict-Send*
shows $(fst (exchange-v10-amounts-int price-n price-d max-wheat-send$
max-wheat-receive max-sheep-send max-sheep-receive rounding) = 0) =
 $(snd (exchange-v10-amounts-int price-n price-d max-wheat-send$
max-wheat-receive max-sheep-send max-sheep-receive rounding) = 0)

Proof sketch: split along the executable branch tree. Every normal or strict-receive branch relates the two amounts by either a positive ceiling conversion or a floor conversion whose multiplier is at least its divisor; the two preceding zero lemmas apply.

theorem *exchange-v10-without-price-error-thresholds-zero-iff*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *not-strict-send*: *rounding* $\neq$ *Exchange-Strict-Send*
and *result*:
exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*)
= *Cxx-Ok exchange-result*
shows $(num-wheat-received exchange-result = 0) =$
 $(num-sheep-send exchange-result = 0)$

Proof sketch: the exact top-level characterization fixes the returned record to the integer amount pair. Its signed-word equalities convert word zero to integer zero, where $\llbracket exchange-v10-pre ?price-n ?price-d ?max-wheat-send ?max-wheat-receive ?max-sheep-send ?max-sheep-receive; ?rounding \neq Exchange-Strict-Send \rrbracket \implies (fst (exchange-v10-amounts-int ?price-n ?price-d ?max-wheat-send ?max-wheat-receive ?max-sheep-send ?max-sheep-receive ?rounding) = 0) = (snd (exchange-v10-amounts-int ?price-n ?price-d ?max-wheat-send$

$?max\text{-}wheat\text{-}receive\ ?max\text{-}sheep\text{-}send\ ?max\text{-}sheep\text{-}receive\ ?rounding) = 0)$
applies.

lemma *int-nested-floor-product-positive-iff*:
fixes x *multiplier divisor* :: *int*
assumes $x\text{-nonnegative}$: $0 \leq x$
and $multiplier\text{-greater}$: $divisor < multiplier$
and $divisor\text{-positive}$: $0 < divisor$
shows $(0 < (x \text{ div } multiplier) * multiplier \text{ div } divisor) =$
 $(multiplier \leq x)$

Proof sketch: if the first quotient is positive, it is at least one and its product is above the smaller divisor. Conversely the first quotient is zero exactly when the value lies below the positive multiplier.

theorem *exchange-v10-strict-send-sheep-positive-iff*:
assumes pre : $exchange\text{-}v10\text{-pre}\ price\text{-}n\ price\text{-}d\ max\text{-}wheat\text{-}send$
 $max\text{-}wheat\text{-}receive\ max\text{-}sheep\text{-}send\ max\text{-}sheep\text{-}receive$
shows $(0 < snd$
 $(exchange\text{-}v10\text{-amounts}\text{-}int\ price\text{-}n\ price\text{-}d\ max\text{-}wheat\text{-}send$
 $max\text{-}wheat\text{-}receive\ max\text{-}sheep\text{-}send\ max\text{-}sheep\text{-}receive$
 $Exchange\text{-}Strict\text{-}Send)) =$
 $(let\ wheat\text{-}value =$
 $exchange\text{-}wheat\text{-}value\text{-}int\ price\text{-}n\ price\text{-}d\ max\text{-}wheat\text{-}send$
 $max\text{-}sheep\text{-}receive;$
 $sheep\text{-}value =$
 $exchange\text{-}sheep\text{-}value\text{-}int\ price\text{-}n\ price\text{-}d\ max\text{-}sheep\text{-}send$
 $max\text{-}wheat\text{-}receive$
in $if\ wheat\text{-}value > sheep\text{-}value$
 $then\ 0 < sint\ max\text{-}sheep\text{-}send \wedge 0 < sint\ max\text{-}sheep\text{-}receive$
 $else\ if\ sint\ price\text{-}n > sint\ price\text{-}d$
 $then\ sint\ price\text{-}n \leq wheat\text{-}value$
 $else\ sint\ price\text{-}d \leq wheat\text{-}value)$

Proof sketch: in the wheat-stays branch, sheep is the minimum of the two sheep maxima. Otherwise simplify the selected floor formula: when wheat is more valuable the nested quotient is positive exactly at one numerator unit, and in the other price order ordinary positive division has the same threshold.

theorem *exchange-v10-without-strict-send-sheep-positive-iff*:
assumes pre : $exchange\text{-}v10\text{-pre}\ price\text{-}n\ price\text{-}d\ max\text{-}wheat\text{-}send$
 $max\text{-}wheat\text{-}receive\ max\text{-}sheep\text{-}send\ max\text{-}sheep\text{-}receive$
and $result$:
 $exchange\text{-}v10\text{-without}\text{-}price\text{-}error\text{-}thresholds\text{-}with\text{-}options\ price\text{-}n\ price\text{-}d$

```

    max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
    Exchange-Strict-Send (exact-receive-cap = False, symmetric-exact-receive-cap
= False) = Cxx-Ok exchange-result
shows (0 < sint (num-sheep-send exchange-result)) =
  (let wheat-value =
    exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive;
    sheep-value =
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive
  in if wheat-value > sheep-value
    then 0 < sint max-sheep-send ∧ 0 < sint max-sheep-receive
    else if sint price-n > sint price-d
    then sint price-n ≤ wheat-value
    else sint price-d ≤ wheat-value)

```

Proof sketch: rewrite the successful result to the exact integer amount record and use the signed interpretation of its sheep word. The preceding integer equivalence is then exactly the desired minimal input condition.

theorem *exchange-v10-without-price-error-thresholds-result-contract:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and result:
exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding (exact-receive-cap = False, symmetric-exact-receive-cap = False)
= *Cxx-Ok exchange-result*
shows $0 \leq \text{sint } (\text{num-wheat-received } \text{exchange-result})$
and $\text{sint } (\text{num-wheat-received } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-wheat-receive}) (\text{sint } \text{max-wheat-send})$
and $0 \leq \text{sint } (\text{num-sheep-send } \text{exchange-result})$
and $\text{sint } (\text{num-sheep-send } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$
and *result-wheat-stays exchange-result =*
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)

Proof sketch: uniqueness of the successful constructor identifies the returned record with the exact integer characterization. Its signed-word equalities and the four amount bounds yield the range clauses, while the record constructor exposes the exact offer-value comparison.

2.8 Full exchange

This last section composes the results above into the contract of *exchange-v10-with-options* and then establishes the crossing property announced at the end of the model.

theorem *exchange-v10-characterization:*

exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding options =
(case exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options of
Cxx-Err error ⇒ Cxx-Err error
| Cxx-Ok before-thresholds ⇒
apply-price-error-thresholds price-n price-d
(num-wheat-received before-thresholds)
(num-sheep-send before-thresholds)
(result-wheat-stays before-thresholds) rounding)

Proof sketch: unfold the single monadic bind. Its two constructor cases are exactly propagation of the first error or the second C++ call with the successful record fields.

theorem *exchange-v10-integer-characterization:*

fixes *rounding :: exchange-rounding and amounts :: int × int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts ≡*
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive*
max-sheep-send max-sheep-receive rounding (exact-receive-cap = False, sym-
metric-exact-receive-cap = False) =
apply-price-error-thresholds price-n price-d
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)
rounding

Proof sketch: *exchange-v10-without-price-error-thresholds-integer-characterization* replaces the first call by its exact amount record under *exchange-v10-pre*. Simplifying the record selectors leaves the threshold call shown here.

theorem *apply-price-error-thresholds-result-choices:*

assumes *result:*

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result
shows *exchange-result =*
make-exchange-result wheat-receive sheep-send wheat-stays ∨
exchange-result = make-exchange-result 0 0 wheat-stays

Proof sketch: every successful branch of the total threshold characterization returns either the unchanged record or its explicit zero-amount counterpart. All remaining branches are errors.

theorem *exchange-v10-result-contract:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding (⊥exact-receive-cap
= False, symmetric-exact-receive-cap = False) =
Cxx-Ok exchange-result
shows $0 \leq \text{sint } (\text{num-wheat-received } \text{exchange-result})$
and $\text{sint } (\text{num-wheat-received } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-wheat-receive}) (\text{sint } \text{max-wheat-send})$
and $0 \leq \text{sint } (\text{num-sheep-send } \text{exchange-result})$
and $\text{sint } (\text{num-sheep-send } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$
and *result-wheat-stays exchange-result =*
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)

Proof sketch: the integer characterization fixes the input to the threshold caller. A successful threshold call returns either those bounded pre-threshold amounts or two zeros and always preserves the stay flag. The previously proved signed interpretations and four bounds therefore establish every clause.

lemma *int-le-ceiling-div-mult:*
fixes *numerator divisor :: int*
assumes *divisor-positive: 0 < divisor*
shows *numerator ≤*
 $((\text{numerator} + \text{divisor} - 1) \text{ div } \text{divisor}) * \text{divisor}$

Proof sketch: decompose the adjusted numerator into its quotient multiple and remainder. The remainder is strictly below the positive divisor, so cancellation of the common divisor increment gives the ceiling inequality.

lemma *exchange-strict-send-favored-int*:
fixes *wheat-value sheep-value price-n price-d max-sheep-send*
max-sheep-receive :: *int*
assumes *pn*: $0 < \text{price-n}$ **and** *pd*: $0 < \text{price-d}$
and *stays*: $\text{sheep-value} < \text{wheat-value}$
and *wheat-receive-bound*:
 $\text{wheat-value} \leq \text{max-sheep-receive} * \text{price-d}$
and *sheep-send-bound*:
 $\text{sheep-value} \leq \text{max-sheep-send} * \text{price-d}$
shows $(\text{sheep-value} \text{ div } \text{price-n}) * \text{price-n} \leq$
 $\min \text{max-sheep-send max-sheep-receive} * \text{price-d}$

Proof sketch: the rounded-down received amount has value at most the sheep offer value. That value is bounded by the send maximum directly and, because wheat stays, strictly by the receive maximum; split which maximum is the minimum.

lemma *exchange-v10-amounts-int-favored*:
fixes *rounding* :: *exchange-rounding* **and** *amounts* :: *int* $\times$ *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* $\equiv$
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *if* *exchange-wheat-value-int price-n price-d max-wheat-send*
max-sheep-receive $>$
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
then *fst amounts* $* \text{sint price-n} \leq \text{snd amounts} * \text{sint price-d}$
else *snd amounts* $* \text{sint price-d} \leq \text{fst amounts} * \text{sint price-n}$

Proof sketch: follow the five amount branches. Every floor conversion favors the output side by $0 < ?divisor \implies ?numerator \text{ div } ?divisor * ?divisor \leq ?numerator$, every ceiling conversion favors it by $0 < ?divisor \implies ?numerator \leq (?numerator + ?divisor - 1) \text{ div } ?divisor * ?divisor$, and the special strict-send branch is $\llbracket 0 < ?price-n; 0 < ?price-d; ?sheep-value < ?wheat-value; ?wheat-value \leq ?max-sheep-receive * ?price-d; ?sheep-value \leq ?max-sheep-send * ?price-d \rrbracket \implies ?sheep-value \text{ div } ?price-n * ?price-n \leq \min ?max-sheep-send ?max-sheep-receive * ?price-d$.

lemma *exchange-v10-before-favored*:
fixes *rounding* :: *exchange-rounding*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* $\equiv$
exchange-v10-amounts-int price-n price-d max-wheat-send

```

    max-wheat-receive max-sheep-send max-sheep-receive rounding
defines wheat-stays  $\equiv$ 
    exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive >
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive
shows favored-seller-ok price-n price-d
    (word-of-int (fst amounts)) (word-of-int (snd amounts)) wheat-stays

```

Proof sketch: the amount words denote the exact mathematical amounts under *exchange-v10-pre*. Substitute those signed interpretations into *favored-seller-ok* and apply the preceding branch theorem.

```

theorem exchange-v10-wellformed-characterization:
fixes rounding :: exchange-rounding and amounts :: int  $\times$  int
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
defines amounts  $\equiv$ 
    exchange-v10-amounts-int price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive rounding
defines wheat-stays  $\equiv$ 
    exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive >
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive
shows exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
    max-sheep-send max-sheep-receive rounding (exact-receive-cap = False, sym-
metric-exact-receive-cap = False) =
    (let wheat-receive = (word-of-int (fst amounts) :: int64) in
    let sheep-send = (word-of-int (snd amounts) :: int64) in
    let original =
        make-exchange-result wheat-receive sheep-send wheat-stays in
    let zero = make-exchange-result 0 0 wheat-stays in
    if  $\neg$  (0 < fst amounts  $\wedge$  0 < snd amounts) then
        if rounding = Exchange-Strict-Send then
            if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
            else Cxx-Ok original
        else Cxx-Ok zero
    else if rounding = Exchange-Normal then
        Cxx-Ok
        (if price-error-bound-spec price-n price-d wheat-receive
            sheep-send False
            then original else zero)
    else if price-error-bound-spec price-n price-d wheat-receive
        sheep-send True
    then Cxx-Ok original
    else Cxx-Err Cxx-Runtime-Error)

```


Proof sketch: substitute the exact pre-threshold amount record. Its signed interpretations replace the positivity guard, positive prices eliminate assertion outcomes, and *exchange-v10-pre* *?price-n* *?price-d* *?max-wheat-send* *?max-wheat-receive* *?max-sheep-send* *?max-sheep-receive* $\implies$ *avored-seller-ok* *?price-n* *?price-d* (*word-of-int* (*fst* (*exchange-v10-amounts-int* *?price-n* *?price-d* *?max-wheat-send* *?max-wheat-receive* *?max-sheep-send* *?max-sheep-receive* *?rounding*))) (*word-of-int* (*snd* (*exchange-v10-amounts-int* *?price-n* *?price-d* *?max-wheat-send* *?max-wheat-receive* *?max-sheep-send* *?max-sheep-receive* *?rounding*))) (*exchange-sheep-value-int* *?price-n* *?price-d* *?max-sheep-send* *?max-wheat-receive* $<$ *exchange-wheat-value-int* *?price-n* *?price-d* *?max-wheat-send* *?max-sheep-receive*) eliminates both favored-seller runtime failures. The remaining branches are exactly the normal and path price-bound behavior plus the strict-send zero test.

corollary *exchange-v10-normal-characterization:*

```

fixes amounts :: int  $\times$  int
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
             max-wheat-receive max-sheep-send max-sheep-receive
defines amounts  $\equiv$ 
             exchange-v10-amounts-int price-n price-d max-wheat-send
             max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal
shows exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
        max-sheep-send max-sheep-receive Exchange-Normal
        (exact-receive-cap = False, symmetric-exact-receive-cap = False) =
        (let wheat-stays =
            exchange-wheat-value-int price-n price-d max-wheat-send
            max-sheep-receive  $>$ 
            exchange-sheep-value-int price-n price-d max-sheep-send
            max-wheat-receive in
            let wheat-receive = (word-of-int (fst amounts) :: int64) in
            let sheep-send = (word-of-int (snd amounts) :: int64) in
            Cxx-Ok
            (if  $0 < \text{fst amounts} \wedge 0 < \text{snd amounts} \wedge$ 
                price-error-bound-spec price-n price-d wheat-receive
                sheep-send False
                then make-exchange-result wheat-receive sheep-send wheat-stays
                else make-exchange-result 0 0 wheat-stays)))

```

Proof sketch: specialize the well-formed characterization to normal mode. Every non-positive trade becomes the zero record; a positive trade retains its amounts exactly when the symmetric price-error predicate succeeds.

corollary *exchange-v10-path-characterization:*

```

fixes rounding :: exchange-rounding and amounts :: int  $\times$  int
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
             max-wheat-receive max-sheep-send max-sheep-receive

```

and *path-mode*: *rounding* $\neq$ *Exchange-Normal*
defines *amounts* $\equiv$
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive*
max-sheep-send max-sheep-receive rounding (*exact-receive-cap* = *False*, *sym-*
metric-exact-receive-cap = *False*) =
 (let *wheat-stays* =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive in
 let *wheat-receive* = (*word-of-int* (*fst amounts*) :: *int64*) in
 let *sheep-send* = (*word-of-int* (*snd amounts*) :: *int64*) in
 let *original* =
make-exchange-result wheat-receive sheep-send wheat-stays in
 if $\neg (0 < \text{fst } amounts \wedge 0 < \text{snd } amounts)$ then
 if *rounding* = *Exchange-Strict-Send* then
 if *sheep-send* = 0 then *Cxx-Err Cxx-Runtime-Error*
 else *Cxx-Ok original*
 else *Cxx-Ok* (*make-exchange-result* 0 0 *wheat-stays*)
 else if *price-error-bound-spec price-n price-d wheat-receive*
sheep-send *True*
 then *Cxx-Ok original*
 else *Cxx-Err Cxx-Runtime-Error*)

Proof sketch: specialize the well-formed characterization to a non-normal constructor. Positive trades are preserved exactly when the one-sided price bound holds; the remaining non-positive branch distinguishes the raw strict-send sheep-zero check from strict-receive zeroing.

lemma *word-zero-iff-sint-zero*:
 ((*word* :: *int64*) = 0) = (*sint word* = 0)

Proof sketch: the forward direction is immediate. In the reverse direction *uint ?w = take-bit LENGTH(?a) (sint ?w)* turns signed zero into unsigned zero, which uniquely identifies the zero word by (*uint ?x* = 0) = (*?x* = 0).

theorem *exchange-v10-zero-iff*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *not-strict-send*: *rounding* $\neq$ *Exchange-Strict-Send*
and *result*: *exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding (*exact-receive-cap*
 = *False*, *symmetric-exact-receive-cap* = *False*) =

Cxx-Ok exchange-result
shows (*num-wheat-received exchange-result = 0*) =
(*num-sheep-send exchange-result = 0*)

Proof sketch: a successful threshold call returns either the exact pre-threshold record or two zeros. For the first choice, signed-word faithfulness and $\llbracket \text{exchange-v10-pre } ?\text{price-n } ?\text{price-d } ?\text{max-wheat-send } ?\text{max-wheat-receive } ?\text{max-sheep-send } ?\text{max-sheep-receive}; ?\text{rounding} \neq \text{Exchange-Strict-Send} \rrbracket \implies (\text{fst } (\text{exchange-v10-amounts-int } ?\text{price-n } ?\text{price-d } ?\text{max-wheat-send } ?\text{max-wheat-receive } ?\text{max-sheep-send } ?\text{max-sheep-receive } ?\text{rounding}) = 0) = (\text{snd } (\text{exchange-v10-amounts-int } ?\text{price-n } ?\text{price-d } ?\text{max-wheat-send } ?\text{max-wheat-receive } ?\text{max-sheep-send } ?\text{max-sheep-receive } ?\text{rounding}) = 0)$ establish simultaneous zero outside strict-send mode; the second choice is immediate.

theorem *apply-price-error-thresholds-strict-send-success:*
assumes *result:*
apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays Exchange-Strict-Send = Cxx-Ok exchange-result
shows *exchange-result =*
make-exchange-result wheat-receive sheep-send wheat-stays
and *sheep-send $\neq 0$*

Proof sketch: a positive strict-send trade can only return the unchanged record, and positivity makes the sheep word nonzero. The non-positive strict-send branch also returns that record exactly when its explicit raw sheep-zero check is false.

theorem *exchange-v10-strict-send-sheep-positive:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Send ($\text{exact-receive-cap} = \text{False}$, $\text{symmetric-exact-receive-cap}$
= False) = Cxx-Ok exchange-result
shows $0 < \text{sint } (\text{num-sheep-send exchange-result})$

Proof sketch: strict-send success preserves the pre-threshold record and proves its sheep word nonzero. That word denotes a non-negative exact integer amount, so nonzero strengthens to strictly positive.

theorem *exchange-v10-positive-trade-contract:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *result: exchange-v10-with-options price-n price-d max-wheat-send*

```

    max-wheat-receive max-sheep-send max-sheep-receive rounding (exact-receive-cap
= False, symmetric-exact-receive-cap = False) =
    Cxx-Ok exchange-result
  and positive:
    0 < sint (num-wheat-received exchange-result) ∧
    0 < sint (num-sheep-send exchange-result)
  shows favored-seller-ok price-n price-d
    (num-wheat-received exchange-result)
    (num-sheep-send exchange-result)
    (result-wheat-stays exchange-result)
  and if rounding = Exchange-Normal
  then price-error-bound-spec price-n price-d
    (num-wheat-received exchange-result)
    (num-sheep-send exchange-result) False
  else price-error-bound-spec price-n price-d
    (num-wheat-received exchange-result)
    (num-sheep-send exchange-result) True

```

Proof sketch: a positive returned trade cannot be the explicit zero record, so the threshold choice theorem identifies it with the exact pre-threshold record. Apply the threshold caller's favored-seller and price-bound contracts and substitute those unchanged fields.

```

export-code big-multiply big-divide-or-throw big-divide-or-throw128
  check-price-error-bound calculate-offer-value
  calculate-offer-value-with-exact-receive-cap apply-price-error-thresholds
  exchange-v10-without-price-error-thresholds-with-options exchange-v10-with-options
  Cxx-Round-Down Cxx-Round-Up checking SML

end
theory Offer-Exchange-Adjustment
  imports Offer-Exchange-Arithmetic
begin

```

3 Offer adjustment and liabilities

This theory isolates the maker-independent arithmetic surrounding *exchange-v10-with-options*. It models offer adjustment and liability projection, then defines the offer and raw-request filters used before ledger state is consulted. Party state, ordinary pre-flight, posting, liability mutation, and crossing belong to *Offer-Exchange-Lifecycle*.

3.1 Offer adjustment and offer liabilities

definition *adjust-offer-with-options* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒
  exchange-options ⇒ int64 cxx-result
— C++: adjustOffer(Price, ...) (OfferExchange.cpp)
where
  adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
    options = do {
  result ← exchange-v10-with-options price-n price-d max-wheat-send int64-max
    int64-max max-sheep-receive Exchange-Normal options;
  Cxx-Ok (num-wheat-received result)
}

```

adjust-offer-with-options is the price-level overload of *adjustOffer* (*OfferExchange.cpp*): the wheat side of a hypothetical cross against a counterparty without limits. The final argument is an *exchange-options* record. Its two exact-cap fields model the protocol gates threaded to ledger-level call sites; the lifecycle theory supplies the state-dependent caps at those sites.

definition *adjust-offer* ::

```

uint32 ⇒ int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒
  int64 cxx-result
— C++: adjustOffer with a ledger version (OfferExchange.cpp)
where
  adjust-offer ledger-version price-n price-d max-wheat-send
    max-sheep-receive =
  adjust-offer-with-options price-n price-d max-wheat-send
    max-sheep-receive (exchange-options-at-version ledger-version)

```

adjust-offer is the protocol-facing overload of *adjustOffer* as protocol 29 leaves it: a leading ledger version, the price, the send cap, and the receive cap. Threading the version here is what carries the repaired arithmetic into posting adjustment, preventative adjustment before a cross, and post-trade remainder adjustment, since all three go through this one helper.

lemma *adjust-offer-legacy*:

```

¬ protocol-version-starts-from ledger-version protocol-version-v29 ⇒
  adjust-offer ledger-version price-n price-d max-wheat-send
    max-sheep-receive =
  adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
    legacy-exchange-options

```

lemma *adjust-offer-repaired*:

```

protocol-version-starts-from ledger-version protocol-version-v29 ⇒
  adjust-offer ledger-version price-n price-d max-wheat-send
    max-sheep-receive =
  adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
    repaired-exchange-options

```

The C++ liability helpers carry no *exactReceiveCap* flag, and the fix did not touch *TransactionUtils.cpp*: liabilities are computed against a counter-

party with a receive cap of $INT64-MAX$, where the clamp inside *calculateOfferAmountFromValue* restores the plain cap. The lemma records that agreement, so the flag-free liability definitions below match both protocol behaviors at once. Proof sketch: after the shared send-side multiplication, both sides multiply *int64-max* by the price denominator; that product is below 2^{126} and the relaxed cap adds less than 2^{64} , so the 128-bit addition cannot wrap, the relaxed receive value is at least the plain one, and the extra minimum collapses.

lemma *exact-receive-cap-collapses-at-unlimited-receive:*

calculate-offer-value-with-exact-receive-cap price-n price-d max-send
int64-max =
calculate-offer-value price-n price-d max-send int64-max

corollary *calculate-offer-amount-from-value-collapses-at-unlimited-receive:*

calculate-offer-amount-from-value price-n price-d max-send int64-max =
(calculate-offer-value price-n price-d max-send int64-max $\gg$
(λ offer-value.
big-divide-or-throw128 offer-value (scast price-n) Cxx-Round-Down))

— At an unlimited receive cap the protocol-29 helper computes the same amount as the protocol-28 calculation it replaces.

Proof sketch: the helper is the relaxed value followed by the round-down division, and at *int64-max* the retained clamp collapses the relaxed value to the plain one, so the two divisions have equal numerators.

definition *offer-selling-liabilities ::*

int32 $\Rightarrow$ int32 $\Rightarrow$ int64 $\Rightarrow$ int64 cxx-result

— C++: *getOfferSellingLiabilities (TransactionUtils.cpp)*

where

offer-selling-liabilities price-n price-d amount = do {
result $\leftarrow$ exchange-v10-without-price-error-thresholds-with-options price-n
price-d
amount int64-max int64-max int64-max Exchange-Normal
($\mid$ exact-receive-cap = False,
symmetric-exact-receive-cap = False);
Cxx-Ok (num-wheat-received result)
}

definition *offer-buying-liabilities ::*

int32 $\Rightarrow$ int32 $\Rightarrow$ int64 $\Rightarrow$ int64 cxx-result

— C++: *getOfferBuyingLiabilities (TransactionUtils.cpp)*

where

offer-buying-liabilities price-n price-d amount = do {
result $\leftarrow$ exchange-v10-without-price-error-thresholds-with-options price-n
price-d
amount int64-max int64-max int64-max Exchange-Normal
($\mid$ exact-receive-cap = False,

```

    symmetric-exact-receive-cap = False);
    Cxx-Ok (num-sheep-send result)
}

```

calculate-offer-amount-from-value-collapses-at-unlimited-receive is the reason these two definitions need no protocol parameter. Both *offer-selling-liabilities* and *offer-buying-liabilities* value a stored offer against a counterparty whose receive cap is *int64-max*, and there the two protocols agree. Protocol 29 therefore recomputes exactly the liabilities protocol 28 booked, which is what makes activation safe without rewriting any stored offer. Removing the $INT64-MAX * priceD$ clamp would invalidate this, which is why protocol 29 retains it.

3.2 Protocol compatibility of the unlimited-cap projections

lemma *pre-thresholds-options-irrelevant-at-unlimited-caps*:

```

exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send int64-max int64-max int64-max rounding options =
exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send int64-max int64-max int64-max rounding
  legacy-exchange-options

```

— With every counterparty cap at *int64-max* the exchange does not depend on the exact-cap options at all.

Proof sketch: the options are consulted in exactly two branches, and each consults them on a call whose receive cap is *int64-max*. After naming the two offer values, the retained clamp collapses each relaxed value to the plain one, so both branches compute what the legacy configuration computes and the two runs agree everywhere.

definition *offer-selling-liabilities-at-version* ::

```

uint32 ⇒ int32 ⇒ int32 ⇒ int64 ⇒
  int64 cxx-result

```

— C++: *getOfferSellingLiabilities* (*TransactionUtils.cpp*, *invariant/LiabilitiesMatchOffers.cpp*)

where

```

offer-selling-liabilities-at-version ledger-version price-n price-d
  amount = do {
    result ← exchange-v10-without-price-error-thresholds ledger-version
      price-n price-d amount int64-max int64-max int64-max Exchange-Normal;
    Cxx-Ok (num-wheat-received result)
  }

```

definition *offer-buying-liabilities-at-version* ::

```

uint32 ⇒ int32 ⇒ int32 ⇒ int64 ⇒
  int64 cxx-result

```

— C++: *getOfferBuyingLiabilities* (*TransactionUtils.cpp*, *invariant/LiabilitiesMatchOffers.cpp*)

where

```

offer-buying-liabilities-at-version ledger-version price-n price-d
  amount = do {
    result ← exchange-v10-without-price-error-thresholds ledger-version
    price-n price-d amount int64-max int64-max int64-max Exchange-Normal;
    Cxx-Ok (num-sheep-send result)
  }

```

Protocol 29 threads the ledger version through both liability helpers in *TransactionUtils.cpp* and through the copies the *LiabilitiesMatchOffers* invariant keeps. The two definitions above have those signatures. The theorems below show the version makes no difference, which is why the flag-free projections used throughout the lifecycle theory remain correct for both protocols.

theorem *offer-selling-liabilities-version-independent:*

```

offer-selling-liabilities-at-version ledger-version price-n price-d amount =
offer-selling-liabilities price-n price-d amount
— Protocol 29 recomputes exactly the selling liability protocol 28 booked.

```

Proof sketch: both project the same field out of a pre-threshold exchange whose counterparty caps are all *int64-max*, and there the exchange does not depend on the options the version selects.

theorem *offer-buying-liabilities-version-independent:*

```

offer-buying-liabilities-at-version ledger-version price-n price-d amount =
offer-buying-liabilities price-n price-d amount
— Protocol 29 recomputes exactly the buying liability protocol 28 booked.

```

Proof sketch: as for the selling liability, with the other projection.

theorem *persistent-liabilities-agree-across-activation:*

```

offer-selling-liabilities-at-version v28 price-n price-d amount =
offer-selling-liabilities-at-version v29 price-n price-d amount
offer-buying-liabilities-at-version v28 price-n price-d amount =
offer-buying-liabilities-at-version v29 price-n price-d amount
— A stored offer has the same booked liabilities before and after the protocol-29
upgrade, at any two ledger versions.

```

Proof sketch: each side is version-independent, so both reduce to the same flag-free projection.

persistent-liabilities-agree-across-activation is the hinge of migration safety. Liabilities acquired under protocol 28 are exactly the liabilities protocol 29 recomputes when it releases them, so activating protocol 29 needs no liability rewrite and no stored-offer migration pass. It depends on the retained *INT64-MAX * priceD* clamp: without that clamp the relaxed receive value

would exceed the plain one even at an unlimited cap and the two protocols would disagree here.

definition *manage-buy-selling-liabilities* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* *ctx-result*

— C++: *ManageBuyOfferOpFrame::getOfferSellingLiabilities* (*ManageBuyOfferOpFrame.cpp*)

where

manage-buy-selling-liabilities *price-n* *price-d* *buy-amount* = do {
 result $\leftarrow$ *exchange-v10-without-price-error-thresholds-with-options*
 price-d *price-n* *int64-max* *int64-max* *int64-max* *buy-amount*
 Exchange-Normal
 (*exact-receive-cap* = *False*,
 symmetric-exact-receive-cap = *False*);
 Cxx-Ok (*num-wheat-received* *result*)
 }

definition *manage-buy-buying-liabilities* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* *ctx-result*

— C++: *ManageBuyOfferOpFrame::getOfferBuyingLiabilities* (*ManageBuyOfferOpFrame.cpp*)

where

manage-buy-buying-liabilities *price-n* *price-d* *buy-amount* = do {
 result $\leftarrow$ *exchange-v10-without-price-error-thresholds-with-options*
 price-d *price-n* *int64-max* *int64-max* *int64-max* *buy-amount*
 Exchange-Normal
 (*exact-receive-cap* = *False*,
 symmetric-exact-receive-cap = *False*);
 Cxx-Ok (*num-sheep-send* *result*)
 }

definition *manage-buy-selling-liabilities-at-version* ::

uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$

int64 *ctx-result*

— C++: *ManageBuyOfferOpFrame::getOfferSellingLiabilities* (*ManageBuyOfferOpFrame.cpp*)

where

manage-buy-selling-liabilities-at-version *ledger-version* *price-n* *price-d*
buy-amount = do {
 result $\leftarrow$ *exchange-v10-without-price-error-thresholds* *ledger-version*
 price-d *price-n* *int64-max* *int64-max* *int64-max* *buy-amount*
 Exchange-Normal;
 Cxx-Ok (*num-wheat-received* *result*)
 }

definition *manage-buy-buying-liabilities-at-version* ::

uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$

int64 *ctx-result*

— C++: *ManageBuyOfferOpFrame::getOfferBuyingLiabilities* (*ManageBuyOfferOpFrame.cpp*)

where

```

manage-buy-buying-liabilities-at-version ledger-version price-n price-d
  buy-amount = do {
    result ← exchange-v10-without-price-error-thresholds ledger-version
              price-d price-n int64-max int64-max int64-max buy-amount
              Exchange-Normal;
    Cxx-Ok (num-sheep-send result)
  }

```

Protocol 29 threads the ledger version through both manage-buy request helpers, exactly as it does for the stored-offer projections. Here, however, the version genuinely matters. A stored offer is valued against a counterparty whose every cap is *int64-max*, where the retained clamp makes the two protocols agree. A manage-buy request instead places the finite submitted buy amount on the receive cap, which is precisely the cap the repaired non-staying branch consults, so *pre-thresholds-options-irrelevant-at-unlimited-caps* does not apply and the computed liabilities can move at the boundary.

lemma *manage-buy-liabilities-at-version-legacy:*

```

¬ protocol-version-starts-from ledger-version protocol-version-v29 ⇒
  manage-buy-selling-liabilities-at-version ledger-version price-n price-d
  buy-amount = manage-buy-selling-liabilities price-n price-d buy-amount
¬ protocol-version-starts-from ledger-version protocol-version-v29 ⇒
  manage-buy-buying-liabilities-at-version ledger-version price-n price-d
  buy-amount = manage-buy-buying-liabilities price-n price-d buy-amount
— Before protocol 29 the versioned helpers are the existing flag-free ones, so every
  protocol-28 result about them still applies.

```

lemma *manage-buy-liabilities-at-version-repaired:*

```

protocol-version-starts-from ledger-version protocol-version-v29 ⇒
  manage-buy-selling-liabilities-at-version ledger-version price-n price-d
  buy-amount =
    (exchange-v10-without-price-error-thresholds-with-options price-d price-n
     int64-max int64-max int64-max buy-amount Exchange-Normal
     repaired-exchange-options ≫=
     (λresult. Cxx-Ok (num-wheat-received result)))
protocol-version-starts-from ledger-version protocol-version-v29 ⇒
  manage-buy-buying-liabilities-at-version ledger-version price-n price-d
  buy-amount =
    (exchange-v10-without-price-error-thresholds-with-options price-d price-n
     int64-max int64-max int64-max buy-amount Exchange-Normal
     repaired-exchange-options ≫=
     (λresult. Cxx-Ok (num-sheep-send result)))
— From protocol 29 the versioned helpers use the repaired kernel.

```

lemma *manage-buy-request-liabilities-change-at-activation:*

```

manage-buy-selling-liabilities-at-version 28 100 101 4 = Cxx-Ok 3
manage-buy-selling-liabilities-at-version 29 100 101 4 = Cxx-Ok 4
manage-buy-buying-liabilities-at-version 28 100 101 4 = Cxx-Ok 3

```

manage-buy-buying-liabilities-at-version 29 100 101 4 = Cxx-Ok 4

— A manage-buy request for four units at price *100/101* books three units of each liability before protocol 29 and four from protocol 29 on.

manage-buy-request-liabilities-change-at-activation is the counterpart of *persistant-liabilities-agree-across-activation*, and it goes the other way. Stored-offer liabilities are the same at every ledger version, which is what makes activation safe without rewriting any offer. Manage-buy request-time liabilities are not: the same submitted request books different amounts before and after the boundary. This is an ordinary protocol-behavior change at request time, not a migration hazard, because nothing persists a request’s liabilities – only the resulting stored offer is persisted, and that is valued by the version-independent projections. The equality theorem for stored offers must therefore not be reused for this path.

definition *manage-buy-normalized-amount* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64*

— the canonical resting selling amount of a manage-buy request; no standalone C++ entry point

where

manage-buy-normalized-amount price-n price-d buy-amount =

(let offer-n = sint price-d;

offer-d = sint price-n;

wheat-value =

min (int64-max-int * offer-n)

(sint buy-amount * offer-d);

selling =

(if offer-n > offer-d

then wheat-value div offer-n

else

(let sheep = wheat-value div offer-d

in (sheep * offer-d + offer-n - 1) div offer-n))

in word-of-int selling)

offer-selling-liabilities and *offer-buying-liabilities* are *getOfferSellingLiabilities* and *getOfferBuyingLiabilities* (*TransactionUtils.cpp*): the reservation demanded by an offer is defined as the outcome of its own unlimited cross, before the price-error thresholds.

The two manage-buy liability functions model *ManageBuyOfferOpFrame::getOfferSellingLiabilities* and *getOfferBuyingLiabilities* (*ManageBuyOfferOpFrame.cpp*). They invert the submitted price, leave the selling side unlimited, and put the submitted buy amount on the receive cap. *manage-buy-normalized-amount* exposes the canonical resting selling amount computed by that exchange for the swapped price. stellar-core has no standalone entry point for it. It is deliberately a direct integer formula so request normalization remains executable without repeating it in the stateful lifecycle or the differential transport layer.

3.3 Reusable normalization and replay results

lemma *adjustment-above-floor-cap-replays-unlimited:*

assumes *pn*: $0 < \text{sint price-n}$

and *pd*: $0 < \text{sint price-d}$

and *amount-positive*: $0 < \text{sint amount}$

and *unsaturated*:

$\text{sint amount} * \text{sint price-n} \leq$

$\text{int64-max-int} * \text{sint price-d}$

and *cap-nonnegative*: $0 \leq \text{sint max-sheep-receive}$

and *above-floor*:

$(\text{sint amount} * \text{sint price-n}) \text{ div } \text{sint price-d} <$

$\text{sint max-sheep-receive}$

shows *adjust-offer-with-options price-n price-d amount max-sheep-receive*

$(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False}) =$

adjust-offer-with-options price-n price-d amount int64-max

$(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False})$

Proof sketch: Euclidean division places the product strictly below the next multiple of the positive denominator. An integral cap strictly above its floor is at least that next integer, so neither this cap nor the unlimited cap clips the wheat value. The normal-exchange characterizations then have identical integer amounts, stays flag, and threshold result.

lemma *manage-sell-request-liabilities-explicit:*

fixes *wheat-value selling buying :: int*

assumes *pn*: $0 < \text{sint price-n}$

and *pd*: $0 < \text{sint price-d}$

and *amount-positive*: $0 < \text{sint amount}$

defines *wheat-value* $\equiv$

$\min (\text{sint amount} * \text{sint price-n})$

$(\text{int64-max-int} * \text{sint price-d})$

and *selling* $\equiv$

$(\text{if } \text{sint price-n} > \text{sint price-d}$

$\text{then wheat-value div sint price-n}$

else

$(\text{let sheep} = \text{wheat-value div sint price-d}$

$\text{in } (\text{sheep} * \text{sint price-d} + \text{sint price-n} - 1) \text{ div sint price-n}))$

and *buying* $\equiv$

$(\text{if } \text{sint price-n} > \text{sint price-d}$

$\text{then } (\text{selling} * \text{sint price-n}) \text{ div sint price-d}$

$\text{else wheat-value div sint price-d})$

shows

offer-selling-liabilities price-n price-d amount =

Cxx-Ok (word-of-int selling)

offer-buying-liabilities price-n price-d amount =

Cxx-Ok (word-of-int buying)

sint (word-of-int selling :: int64) = selling

```

sint (word-of-int buying :: int64) = buying
0 ≤ selling
selling ≤ sint amount
0 ≤ buying

```

Proof sketch: characterize the request's unlimited, pre-threshold liability exchange in unbounded integers. Its wheat value is the stated minimum, the counteroffer is never the side that stays, and the normal rounding formulas are exactly the stated selling and buying amounts. The two liability helpers project those fields from the same exchange result.

```

lemma manage-buy-request-liabilities-explicit:
fixes wheat-value selling buying :: int
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and amount-positive: 0 < sint buy-amount
defines wheat-value ≡
  min (int64-max-int * sint price-d)
    (sint buy-amount * sint price-n)
and selling ≡
  (if sint price-d > sint price-n
   then wheat-value div sint price-d
   else
    (let sheep = wheat-value div sint price-n
     in (sheep * sint price-n + sint price-d - 1) div sint price-d))
and buying ≡
  (if sint price-d > sint price-n
   then (selling * sint price-d) div sint price-n
   else wheat-value div sint price-n)
shows
  manage-buy-selling-liabilities price-n price-d buy-amount =
    Cxx-Ok (word-of-int selling)
  manage-buy-buying-liabilities price-n price-d buy-amount =
    Cxx-Ok (word-of-int buying)
  manage-buy-normalized-amount price-n price-d buy-amount =
    word-of-int selling
  sint (word-of-int selling :: int64) = selling
  sint (word-of-int buying :: int64) = buying
  0 ≤ selling
  0 ≤ buying
  buying ≤ sint buy-amount

```

Proof sketch: characterize the unlimited-send exchange at the inverted raw price. The submitted buy amount is its receive cap, so its wheat value is the stated minimum. The opposing unlimited offer cannot be the smaller side, and NORMAL rounding therefore yields exactly the stated canonical selling and buying liabilities. The generated-word bounds follow from the

general integer characterization.

lemma *adjustment-at-exact-receive-value-replays-unlimited:*
assumes *pn*: $0 < \text{sint price-n}$
and *pd*: $0 < \text{sint price-d}$
and *amount-nonnegative*: $0 \leq \text{sint amount}$
and *cap-nonnegative*: $0 \leq \text{sint max-sheep-receive}$
and *unsaturated*:
 $\text{sint amount} * \text{sint price-n} \leq$
 $\text{int64-max-int} * \text{sint price-d}$
and *exact-value*:
 $\text{sint amount} * \text{sint price-n} =$
 $\text{sint max-sheep-receive} * \text{sint price-d}$
shows *adjust-offer-with-options price-n price-d amount max-sheep-receive*
 $(\text{exact-receive-cap} = \text{False},$
 $\text{symmetric-exact-receive-cap} = \text{False}) =$
adjust-offer-with-options price-n price-d amount int64-max
 $(\text{exact-receive-cap} = \text{False},$
 $\text{symmetric-exact-receive-cap} = \text{False})$

Proof sketch: at the finite cap, both arguments of the wheat-value minimum are the same exact product. Unsaturation makes the unlimited cap select that product as well. The normal exchange consequently has identical integer amounts and stay flag at the two receive caps.

lemma *adjustment-int-le-div-from-product:*
fixes *lower numerator divisor* :: *int*
assumes *divisor-positive*: $0 < \text{divisor}$
and *product-bound*: $\text{lower} * \text{divisor} \leq \text{numerator}$
shows $\text{lower} \leq \text{numerator div divisor}$

Proof sketch: the lower value is its own product's quotient, and integer division by a positive divisor is monotone in the numerator.

lemma *capped-normal-result-candidates:*
fixes *wheat-value selling buying* :: *int*
and *price-n price-d* :: *int32*
and *request-send request-receive max-send max-receive posted* :: *int64*
defines *wheat-value* $\equiv$
 $\min (\text{sint request-send} * \text{sint price-n})$
 $(\text{sint request-receive} * \text{sint price-d})$
and *selling* $\equiv \text{wheat-value div sint price-n}$
and *buying* $\equiv$
 $(\text{selling} * \text{sint price-n}) \text{ div sint price-d}$

```

assumes pn:  $0 < \text{sint price-n}$ 
and pd:  $0 < \text{sint price-d}$ 
and request-send-nonnegative:  $0 \leq \text{sint request-send}$ 
and request-receive-nonnegative:  $0 \leq \text{sint request-receive}$ 
and wheat-more:  $\text{sint price-n} > \text{sint price-d}$ 
and send-nonnegative:  $0 \leq \text{sint max-send}$ 
and receive-nonnegative:  $0 \leq \text{sint max-receive}$ 
and send-le-request:  $\text{sint max-send} \leq \text{sint request-send}$ 
and receive-le-request:  $\text{sint max-receive} \leq \text{sint request-receive}$ 
and selling-fits:  $\text{selling} \leq \text{sint max-send}$ 
and buying-fits:  $\text{buying} \leq \text{sint max-receive}$ 
and adjustment:
  adjust-offer-with-options price-n price-d max-send max-receive
  (exact-receive-cap = False,
   symmetric-exact-receive-cap = False) = Cxx-Ok posted
and posted-positive:  $0 < \text{sint posted}$ 
shows
  sint posted = selling  $\vee$ 
  (sint posted = selling - 1  $\wedge$ 
   (selling * sint price-n) mod sint price-d  $\neq 0$ )

```

Proof sketch: capacities covering the normalized selling and buying amounts sandwich the capped wheat value between the buying multiple and the raw request caps. Dividing by the more valuable wheat price confines a positive successful NORMAL result to the normalized selling amount or its immediate predecessor. Exact divisibility rules out the predecessor.

```

lemma capped-normal-manage-sell-result-candidates:
fixes wheat-value selling buying :: int
and price-n price-d :: int32
and amount max-send max-receive posted :: int64
defines wheat-value  $\equiv$ 
  min (sint amount * sint price-n)
      (int64-max-int * sint price-d)
and selling  $\equiv$  wheat-value div sint price-n
and buying  $\equiv$ 
  (selling * sint price-n) div sint price-d
assumes pn:  $0 < \text{sint price-n}$ 
and pd:  $0 < \text{sint price-d}$ 
and amount-positive:  $0 < \text{sint amount}$ 
and wheat-more:  $\text{sint price-n} > \text{sint price-d}$ 
and send-nonnegative:  $0 \leq \text{sint max-send}$ 
and receive-nonnegative:  $0 \leq \text{sint max-receive}$ 
and send-le-amount:  $\text{sint max-send} \leq \text{sint amount}$ 
and selling-fits:  $\text{selling} \leq \text{sint max-send}$ 
and buying-fits:  $\text{buying} \leq \text{sint max-receive}$ 
and adjustment:
  adjust-offer-with-options price-n price-d max-send max-receive

```

$(\text{exact-receive-cap} = \text{False},$
 $\text{symmetric-exact-receive-cap} = \text{False}) = \text{Cxx-Ok posted}$
and $\text{posted-positive}: 0 < \text{sint posted}$
shows
 $\text{sint posted} = \text{selling} \vee$
 $(\text{sint posted} = \text{selling} - 1 \wedge$
 $(\text{selling} * \text{sint price-n}) \bmod \text{sint price-d} \neq 0)$

Proof sketch: specialize the general two-request-cap confinement theorem to a ManageSell request, whose receive cap is *int64-max*.

lemma *capped-normal-manage-buy-result-candidates:*
fixes $\text{wheat-value selling buying} :: \text{int}$
and $\text{price-n price-d} :: \text{int32}$
and $\text{buy-amount max-send max-receive posted} :: \text{int64}$
defines $\text{wheat-value} \equiv$
 $\min (\text{int64-max-int} * \text{sint price-d})$
 $(\text{sint buy-amount} * \text{sint price-n})$
and $\text{selling} \equiv \text{wheat-value} \div \text{sint price-d}$
and $\text{buying} \equiv$
 $(\text{selling} * \text{sint price-d}) \div \text{sint price-n}$
assumes $\text{pn}: 0 < \text{sint price-n}$
and $\text{pd}: 0 < \text{sint price-d}$
and $\text{amount-positive}: 0 < \text{sint buy-amount}$
and $\text{wheat-more}: \text{sint price-d} > \text{sint price-n}$
and $\text{send-nonnegative}: 0 \leq \text{sint max-send}$
and $\text{receive-nonnegative}: 0 \leq \text{sint max-receive}$
and $\text{receive-le-amount}: \text{sint max-receive} \leq \text{sint buy-amount}$
and $\text{selling-fits}: \text{selling} \leq \text{sint max-send}$
and $\text{buying-fits}: \text{buying} \leq \text{sint max-receive}$
and $\text{adjustment}:$
 $\text{adjust-offer-with-options price-d price-n max-send max-receive}$
 $(\text{exact-receive-cap} = \text{False},$
 $\text{symmetric-exact-receive-cap} = \text{False}) = \text{Cxx-Ok posted}$
and $\text{posted-positive}: 0 < \text{sint posted}$
shows
 $\text{sint posted} = \text{selling} \vee$
 $(\text{sint posted} = \text{selling} - 1 \wedge$
 $(\text{selling} * \text{sint price-d}) \bmod \text{sint price-n} \neq 0)$

Proof sketch: specialize the general confinement theorem to ManageBuy's unlimited send cap and submitted receive cap, after swapping the raw price components into canonical resting orientation.

lemma *capped-normal-result-replays-unlimited:*
fixes $\text{wheat-value selling buying} :: \text{int}$
and $\text{price-n price-d} :: \text{int32}$


```

and request-send request-receive max-send max-receive posted :: int64
defines wheat-value ≡
  min (sint request-send * sint price-n)
    (sint request-receive * sint price-d)
and selling ≡ wheat-value div sint price-n
and buying ≡
  (selling * sint price-n) div sint price-d
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and request-send-nonnegative: 0 ≤ sint request-send
and request-receive-nonnegative: 0 ≤ sint request-receive
and wheat-more: sint price-n > sint price-d
and send-nonnegative: 0 ≤ sint max-send
and receive-nonnegative: 0 ≤ sint max-receive
and send-le-request: sint max-send ≤ sint request-send
and receive-le-request: sint max-receive ≤ sint request-receive
and selling-fits: selling ≤ sint max-send
and buying-fits: buying ≤ sint max-receive
and adjustment:
  adjust-offer-with-options price-n price-d max-send max-receive
    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False) = Cxx-Ok posted
and posted-positive: 0 < sint posted
and fixed:
  adjust-offer-with-options price-n price-d posted max-receive
    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False) = Cxx-Ok posted
and posted-unsaturated:
  sint posted * sint price-n ≤
    int64-max-int * sint price-d
shows
  adjust-offer-with-options price-n price-d posted int64-max
    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False) = Cxx-Ok posted

```

Proof sketch: use the confinement theorem to split on the normalized amount and its fractional predecessor. A predecessor's buying floor lies strictly below the covered receive cap, so raising that cap to unlimited replays the result. The full candidate either has the same strict-floor argument or the receive cap is exactly the normalized liability; in the latter case the successful capped exchange forces an exact price product and the exact-value replay theorem applies.

```

lemma capped-normal-manage-sell-result-replays-unlimited:
fixes wheat-value selling buying :: int
and price-n price-d :: int32
and amount max-send max-receive posted :: int64

```

```

defines wheat-value  $\equiv$ 
  min (sint amount * sint price-n)
    (int64-max-int * sint price-d)
and selling  $\equiv$  wheat-value div sint price-n
and buying  $\equiv$ 
  (selling * sint price-n) div sint price-d
assumes pn:  $0 < \textit{sint price-n}$ 
and pd:  $0 < \textit{sint price-d}$ 
and amount-positive:  $0 < \textit{sint amount}$ 
and wheat-more:  $\textit{sint price-n} > \textit{sint price-d}$ 
and send-nonnegative:  $0 \leq \textit{sint max-send}$ 
and receive-nonnegative:  $0 \leq \textit{sint max-receive}$ 
and send-le-amount:  $\textit{sint max-send} \leq \textit{sint amount}$ 
and selling-fits:  $\textit{selling} \leq \textit{sint max-send}$ 
and buying-fits:  $\textit{buying} \leq \textit{sint max-receive}$ 
and adjustment:
  adjust-offer-with-options price-n price-d max-send max-receive
    (exact-receive-cap = False,
      symmetric-exact-receive-cap = False) = Cxx-Ok posted
and posted-positive:  $0 < \textit{sint posted}$ 
and fixed:
  adjust-offer-with-options price-n price-d posted max-receive
    (exact-receive-cap = False,
      symmetric-exact-receive-cap = False) = Cxx-Ok posted
and posted-unsaturated:
   $\textit{sint posted} * \textit{sint price-n} \leq$ 
     $\textit{int64-max-int} * \textit{sint price-d}$ 
shows
  adjust-offer-with-options price-n price-d posted int64-max
    (exact-receive-cap = False,
      symmetric-exact-receive-cap = False) = Cxx-Ok posted

```

Proof sketch: specialize the two-request-cap replay theorem to an unlimited ManageSell receive cap.

```

lemma capped-normal-manage-buy-result-replays-unlimited:
fixes wheat-value selling buying :: int
and price-n price-d :: int32
and buy-amount max-send max-receive posted :: int64
defines wheat-value  $\equiv$ 
  min (int64-max-int * sint price-d)
    (sint buy-amount * sint price-n)
and selling  $\equiv$  wheat-value div sint price-d
and buying  $\equiv$ 
  (selling * sint price-d) div sint price-n
assumes pn:  $0 < \textit{sint price-n}$ 
and pd:  $0 < \textit{sint price-d}$ 
and amount-positive:  $0 < \textit{sint buy-amount}$ 

```

```

and wheat-more: sint price-d > sint price-n
and send-nonnegative:  $0 \leq \textit{sint max-send}$ 
and receive-nonnegative:  $0 \leq \textit{sint max-receive}$ 
and receive-le-amount:  $\textit{sint max-receive} \leq \textit{sint buy-amount}$ 
and selling-fits:  $\textit{selling} \leq \textit{sint max-send}$ 
and buying-fits:  $\textit{buying} \leq \textit{sint max-receive}$ 
and adjustment:
  adjust-offer-with-options price-d price-n max-send max-receive
  ( $\textit{exact-receive-cap} = \textit{False}$ ,
    $\textit{symmetric-exact-receive-cap} = \textit{False}$ ) = Cxx-Ok posted
and posted-positive:  $0 < \textit{sint posted}$ 
and fixed:
  adjust-offer-with-options price-d price-n posted max-receive
  ( $\textit{exact-receive-cap} = \textit{False}$ ,
    $\textit{symmetric-exact-receive-cap} = \textit{False}$ ) = Cxx-Ok posted
and posted-unsaturated:
   $\textit{sint posted} * \textit{sint price-d} \leq$ 
   $\textit{int64-max-int} * \textit{sint price-n}$ 
shows
  adjust-offer-with-options price-d price-n posted int64-max
  ( $\textit{exact-receive-cap} = \textit{False}$ ,
    $\textit{symmetric-exact-receive-cap} = \textit{False}$ ) = Cxx-Ok posted

```

Proof sketch: specialize the two-request-cap replay theorem to ManageBuy's unlimited send cap and submitted receive cap, in canonical price order.

```

end
theory Offer-Exchange-Posting-Refinement
  imports Offer-Exchange-Specification Offer-Exchange-Adjustment HOL.Real
begin

hide-const (open)
  Offer-Exchange-Specification.price-n
  Offer-Exchange-Specification.price-d

```

4 Minimal implementation posting slice

This theory repeats the smallest implementation-facing slice needed to state and prove the posting refinement without importing the larger lifecycle development. The record, invariants, capacities, liability reservation, pre-flight outcomes, and posting core below are copied definition-for- definition from *Offer-Exchange-Lifecycle*. They model the corresponding protocol-10 path through *ManageOfferOpFrameBase::doApply*, the *canSellAtMost* and *canBuyAtMost* helpers in *OfferExchange.cpp*, and liability acquisition in *TransactionUtils.cpp*. Keeping the definitions unchanged preserves their executable, bit-precise correspondence with the C++ code.

```

record party-state =
  sell-balance :: int64
  sell-liabilities :: int64 — Committed to sell
  buy-limit :: int64
  buy-balance :: int64
  buy-liabilities :: int64 — Committed to buy

```

A party is described by five quantities. On the side of the asset it sells, *sell-balance* is the spendable balance (net of any reserve) and *sell-liabilities* the selling liabilities already reserved for open offers. On the side of the asset it buys, *buy-limit* is the trustline limit, *buy-balance* the current balance, and *buy-liabilities* the buying liabilities already reserved. The maker sells wheat and buys sheep; the taker sells sheep and buys wheat. The selling side carries no limit because nothing in the modeled code paths ever receives into it, and the buying side carries no selling liabilities because nothing ever spends from it.

definition *party-state-wf* :: *party-state* $\Rightarrow$ *bool*

where

```

party-state-wf party  $\longleftrightarrow$ 
  0  $\leq$  sint (sell-liabilities party)  $\wedge$ 
  sint (sell-liabilities party)  $\leq$  sint (sell-balance party)  $\wedge$ 
  0  $\leq$  sint (buy-balance party)  $\wedge$ 
  0  $\leq$  sint (buy-liabilities party)  $\wedge$ 
  sint (buy-balance party) + sint (buy-liabilities party)  $\leq$ 
  sint (buy-limit party)

```

party-state-wf is the ledger invariant that operations preserve: liabilities are non-negative, selling liabilities fit inside the balance, and the balance together with the buying liabilities fits inside the limit.

definition *can-sell-at-most* :: *party-state* $\Rightarrow$ *int64*

— C++: *canSellAtMost* (*OfferExchange.cpp*)

where

```

can-sell-at-most party =
  (let available =
    sint (sell-balance party) - sint (sell-liabilities party)
  in if available  $\leq$  0 then 0 else word-of-int available)

```

definition *can-buy-at-most* :: *party-state* $\Rightarrow$ *int64*

— C++: *canBuyAtMost* (*OfferExchange.cpp*)

where

```

can-buy-at-most party =
  (let available =
    sint (buy-limit party) - sint (buy-balance party) -
    sint (buy-liabilities party)
  in if available  $\leq$  0 then 0 else word-of-int available)

```

can-sell-at-most and *can-buy-at-most* model the C++ functions of the same names (*OfferExchange.cpp*): the available balance, respectively the available

limit, clamped to zero from below.

definition *add-liability-checked* ::

int $\Rightarrow$ *int64* $\Rightarrow$ *int* $\Rightarrow$ *int64* *cxx-result*

— C++: *addBuyingLiabilities* and *addSellingLiabilities* (*TransactionUtils.cpp*)

where

add-liability-checked *cap* *current* *delta* =

(*let* *new-total* = *sint* *current* + *delta*

in if *new-total* < 0 $\vee$ *cap* < *new-total*

then *Cxx-Err* *Cxx-Runtime-Error*

else *Cxx-Ok* (*word-of-int* *new-total*))

add-liability-checked models the checked liability updates *addBuyingLiabilities* and *addSellingLiabilities* (*TransactionUtils.cpp*): adding the delta must keep the liability total between zero and the capacity supplied by the caller, and a violation is the runtime error that the C++ call sites raise.

definition *acquire-offer-liabilities* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$

party-state *cxx-result*

— C++: *acquireLiabilities* (*TransactionUtils.cpp*)

where

acquire-offer-liabilities *price-n* *price-d* *amount* *maker* = *do* {

buying $\leftarrow$ *offer-buying-liabilities* *price-n* *price-d* *amount*;

new-buying $\leftarrow$ *add-liability-checked*

(*sint* (*buy-limit* *maker*) – *sint* (*buy-balance* *maker*))

(*buy-liabilities* *maker*) (*sint* *buying*);

selling $\leftarrow$ *offer-selling-liabilities* *price-n* *price-d* *amount*;

new-selling $\leftarrow$ *add-liability-checked* (*sint* (*sell-balance* *maker*))

(*sell-liabilities* *maker*) (*sint* *selling*);

Cxx-Ok (*maker*(*buy-liabilities* := *new-buying*,

sell-liabilities := *new-selling*))

}

acquire-offer-liabilities models *acquireLiabilities* (*TransactionUtils.cpp*), the acquiring direction of *acquireOrReleaseLiabilities*. Following the C++, the buying side is updated before the selling side, each against the maker's trustline capacity.

datatype *offer-preflight-outcome* =

Preflight-Line-Full

| *Preflight-Underfunded*

| *Preflight-Ready* *int64* *int64*

datatype *post-outcome* =

Post-Malformed

| *Post-Line-Full*

| *Post-Underfunded*

| *Post-No-Offer*

| *Post-Created* *int64* *party-state*

definition *preflight-offer-core* ::

int64 cxx-result $\Rightarrow$ *int64 cxx-result* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *party-state* $\Rightarrow$ *offer-preflight-outcome cxx-result*
 — C++: *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*)

where

preflight-offer-core *buying-liability* *selling-liability* *max-send-cap*
max-receive-cap *maker* = *do* {
buying $\leftarrow$ *buying-liability*;
if *sint* (*buy-limit* *maker*) – *sint* (*buy-balance* *maker*) –
sint (*buy-liabilities* *maker*) < *sint* *buying*
then *Cxx-Ok Preflight-Line-Full*
else do {
selling $\leftarrow$ *selling-liability*;
if *sint* (*sell-balance* *maker*) – *sint* (*sell-liabilities* *maker*) <
sint *selling*
then *Cxx-Ok Preflight-Underfunded*
else
(*let* *max-sheep-send* =
signed-min64 *max-send-cap* (*can-sell-at-most* *maker*);
max-wheat-receive =
(*if* *max-receive-cap* = *int64-max*
then *can-buy-at-most* *maker*
else *signed-min64* *max-receive-cap* (*can-buy-at-most* *maker*))
in *if* *max-wheat-receive* = 0
then *Cxx-Ok Preflight-Line-Full*
else *Cxx-Ok*
(*Preflight-Ready* *max-sheep-send* *max-wheat-receive*))
}
}
}

preflight-offer-core models the protocol-10 pre-flight tests of *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*) together with the immediate check in *doApply* that reports *LINE-FULL* when *maxWheatReceive* is zero. The request-specific liabilities and operation caps arrive as parameters; the cap minima model the sell and buy instantiations of *applyOperationSpecificLimits*.

definition *post-offer-core* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *bool* $\Rightarrow$ *int64 cxx-result* $\Rightarrow$
int64 cxx-result $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
exchange-options $\Rightarrow$ *post-outcome cxx-result*

— C++: *ManageOfferOpFrameBase::doApply* (*ManageOfferOpFrameBase.cpp*)

where

post-offer-core *price-n* *price-d* *request-valid* *buying-liability*
selling-liability *max-send-cap* *max-receive-cap* *maker* *options* =
(*if* *sint* *price-n* $\leq$ 0 $\vee$ *sint* *price-d* $\leq$ 0 $\vee$ $\neg$ *request-valid*
then *Cxx-Ok Post-Malformed*
else do {
preflight $\leftarrow$ *preflight-offer-core* *buying-liability*

```

    selling-liability max-send-cap max-receive-cap maker;
  case preflight of
    Preflight-Line-Full  $\Rightarrow$  Cxx-Ok Post-Line-Full
  | Preflight-Underfunded  $\Rightarrow$  Cxx-Ok Post-Underfunded
  | Preflight-Ready max-sheep-send max-wheat-receive  $\Rightarrow$  do {
    adjusted  $\leftarrow$  adjust-offer-with-options price-n price-d
    max-sheep-send max-wheat-receive options;
    if  $0 < \text{sint adjusted}$ 
    then do {
      maker-after  $\leftarrow$  acquire-offer-liabilities price-n price-d
      adjusted maker;
      Cxx-Ok (Post-Created adjusted maker-after)
    }
    else Cxx-Ok Post-No-Offer
  }
})

```

post-offer-core models the protocol-10 posting path of *ManageOfferOpFrameBase::doApply* (*ManageOfferOpFrameBase.cpp*) for an offer that crosses nothing on entry: malformed-request rejection, pre-flight, preventative adjustment through *adjust-offer-with-options*, and liability acquisition for the created offer.

definition *post-sell-offer* ::

```

  int32  $\Rightarrow$  int32  $\Rightarrow$  int64  $\Rightarrow$  party-state  $\Rightarrow$ 
    exchange-options  $\Rightarrow$  post-outcome cxx-result
— C++: doApply for ManageSellOfferOpFrame

```

where

```

  post-sell-offer price-n price-d amount maker options =
    post-offer-core price-n price-d ( $0 < \text{sint amount}$ )
      (offer-buying-liabilities price-n price-d amount)
      (offer-selling-liabilities price-n price-d amount)
      amount int64-max maker options

```

post-sell-offer instantiates the core for a *ManageSellOffer*: the liability arguments are *ManageSellOfferOpFrame::getOfferBuyingLiabilities* and *getOfferSellingLiabilities* (*ManageSellOfferOpFrame.cpp*), and the submitted amount caps the send side exactly as *ManageSellOfferOpFrame::applyOperationSpecificLimits* does.

The copied posting slice deliberately omits balance transfer, liability release, crossing, and lifecycle predicates: none is used by the refinement theorem. The arithmetic lemmas below are included only where the repaired adjustment calculation needs them.

4.1 Posting-state invariant facts

lemma *can-sell-at-most-nonnegative*:

```

  assumes wf: party-state-wf party

```

shows $0 \leq \text{sint } (\text{can-sell-at-most party})$

Proof sketch: the well-formedness invariant makes the available selling balance non-negative and at most the signed-word maximum. The clamping definition therefore returns either zero or a word with that same non-negative signed value.

lemma *can-buy-at-most-nonnegative*:

assumes *wf*: *party-state-wf party*

shows $0 \leq \text{sint } (\text{can-buy-at-most party})$

Proof sketch: well-formedness makes the unused buying capacity non-negative and bounds it by the trustline limit. The clamping definition therefore preserves its non-negative signed value.

4.2 Repaired adjustment arithmetic

The refinement uses the two repaired receive caps that are absent from the older plain-cap integer characterization. The following word facts and repaired amount characterization are the exact dependency closure copied from *Offer-Exchange-Lifecycle*; later lifecycle maximality, crossing, and reachability results are intentionally not included.

lemma *exact-value-word-characterization*:

assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$ **and** *send*: $0 \leq \text{sint max-send}$ **and** *receive*: $0 \leq \text{sint max-receive}$

shows *calculate-offer-value-with-exact-receive-cap price-n price-d max-send max-receive* = *Cxx-Ok* $(\min (\text{word-of-int } (\text{sint max-send} * \text{sint price-n}) :: \text{uint128}) (\min (\text{word-of-int } (\text{sint max-receive} * \text{sint price-d}) :: \text{uint128}) + \text{ucast } (\text{sint } (\text{price-d} - 1) :: \text{int64})) (\text{word-of-int } (\text{sint int64-max} * \text{sint price-d}) :: \text{uint128}))$

Proof sketch: all operands are non-negative signed words, so the multiplications succeed with their integer products. Unfolding the exact cap leaves the nested minimum displayed in the conclusion.

lemma *positive-price-denominator-minus-one-cast*:

fixes *price-d* :: *int32*

assumes $0 < \text{sint price-d}$

shows $\text{sint } (\text{sint } (\text{price-d} - 1) :: \text{int64}) = \text{sint price-d} - 1$

Proof sketch: subtracting one from a positive signed 32-bit denominator does not wrap; widening that non-negative result to 64 bits preserves its signed integer value.

lemma *uint-eq-sint-nonnegative-int64*:

fixes *word* :: *int64*

assumes $0 \leq \text{sint } \text{word}$

shows $\text{uint } \text{word} = \text{sint } \text{word}$

Proof sketch: a non-negative signed 64-bit value lies in the unsigned 64-bit range, so the unsigned truncation in $\text{uint } ?w = \text{take-bit } \text{LENGTH}(?'a) (\text{sint } ?w)$ is the identity.

lemma *int-le-div-from-product*:

fixes *lower numerator divisor* :: *int*

assumes *divisor-positive*: $0 < \text{divisor}$

and *product-bound*: $\text{lower} * \text{divisor} \leq \text{numerator}$

shows $\text{lower} \leq \text{numerator div divisor}$

Proof sketch: the lower value is its own product's quotient, and integer division by a positive divisor is monotone in the numerator.

lemma *int-div-below-next-multiple*:

fixes *numerator divisor bound* :: *int*

assumes *divisor-positive*: $0 < \text{divisor}$

and *numerator-bound*: $\text{numerator} \leq \text{bound} * \text{divisor} + \text{divisor} - 1$

shows $\text{numerator div divisor} \leq \text{bound}$

Proof sketch: division monotonicity compares the numerator against the largest value below the next multiple of the divisor, whose quotient is exactly the bound.

lemma *int-div-le-imp-below-next-multiple*:

fixes *numerator divisor bound* :: *int*

assumes *divisor-positive*: $0 < \text{divisor}$

and *quotient-bound*: $\text{numerator div divisor} \leq \text{bound}$

shows $\text{numerator} \leq \text{bound} * \text{divisor} + \text{divisor} - 1$

Proof sketch: decompose the numerator into quotient multiple and remainder; the remainder is below the divisor and the quotient multiple is below the bound multiple.

lemma *exact-receive-cap-value-int*:

assumes *pn*: $0 < \text{sint } \text{price-n}$ **and** *pd*: $0 < \text{sint } \text{price-d}$

and *send-nonnegative*: $0 \leq \text{sint } \text{max-send}$

and *receive-nonnegative*: $0 \leq \text{sint } \text{max-receive}$

obtains *trade-word* :: *uint128* **where**

calculate-offer-value-with-exact-receive-cap price-n price-d max-send

$$\begin{aligned}
& \text{max-receive} = \text{Cxx-Ok trade-word} \\
& \text{uint trade-word} = \\
& \quad \min (\text{sint max-send} * \text{sint price-n}) \\
& \quad (\min (\text{sint max-receive} * \text{sint price-d} + \text{sint price-d} - 1) \\
& \quad (\text{sint int64-max} * \text{sint price-d}))
\end{aligned}$$

Proof sketch: the word-level characterization gives a nested minimum of three 128-bit values. The two products are exact by the multiplication range theorem, the relaxation addend is the denominator minus one, and the receive product is far below the 128-bit modulus, so the relaxed receive value is an exact sum and the unsigned minimum commutes with *uint*.

lemma *exchange-v10-amounts-wheat-stays-down-repaired:*
fixes *wheat-word sheep-word* :: *uint128*
and *trade sheep-send wheat-receive* :: *int*
assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$
and *sheep-more*: $\neg \text{sint price-n} > \text{sint price-d}$
and *receive-nonnegative*: $0 \leq \text{sint max-wheat-receive}$
and *send-nonnegative*: $0 \leq \text{sint max-sheep-send}$
and *symmetric*: *symmetric-exact-receive-cap options*
defines *trade* $\equiv$
 $\min (\text{sint max-sheep-send} * \text{sint price-d})$
 $(\min (\text{sint max-wheat-receive} * \text{sint price-n} + \text{sint price-n} - 1)$
 $(\text{sint int64-max} * \text{sint price-n}))$
and *sheep-send* $\equiv \text{trade div sint price-d}$
and *wheat-receive* $\equiv \text{sheep-send} * \text{sint price-d div sint price-n}$
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
Exchange-Normal options =
Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)
and *sint (word-of-int wheat-receive :: int64)* = *wheat-receive*
and *sint (word-of-int sheep-send :: int64)* = *sheep-send*

Proof sketch: the symmetric exact cap values the trade by the reversed exact receive-cap calculation. Its value is at most the sheep-send product, so the first round-down division fits signed 64 bits; the resulting sheep multiple is at most the relaxed wheat-receive component, so the second round-down division is bounded by the wheat-receive cap.

lemma *exchange-v10-amounts-sheep-stays-wheat-more-repaired:*
fixes *wheat-word sheep-word* :: *uint128*
and *trade wheat-receive sheep-send* :: *int*
assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$
and *wheat-more*: $\text{sint price-n} > \text{sint price-d}$
and *send-nonnegative*: $0 \leq \text{sint max-wheat-send}$
and *receive-nonnegative*: $0 \leq \text{sint max-sheep-receive}$

```

and exact: exact-receive-cap options
defines trade  $\equiv$ 
  min (sint max-wheat-send * sint price-n)
    (min (sint max-sheep-receive * sint price-d + sint price-d - 1)
      (sint int64-max * sint price-d))
and wheat-receive  $\equiv$  trade div sint price-n
and sheep-send  $\equiv$  wheat-receive * sint price-n div sint price-d
shows exchange-v10-amounts price-n price-d wheat-word sheep-word
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
  rounding options =
  Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)
and sint (word-of-int wheat-receive :: int64) = wheat-receive
and sint (word-of-int sheep-send :: int64) = sheep-send

```

Proof sketch: the exact receive cap values the trade by the exact calculation on the wheat offer's caps. Its value is at most the wheat-send product, so the first round-down division fits signed 64 bits; the resulting wheat multiple is at most the relaxed sheep-receive component, so the second round-down division is bounded by the sheep-receive cap.

```

definition exchange-v10-amounts-int-repaired ::
  int32  $\Rightarrow$  int32  $\Rightarrow$  int64  $\Rightarrow$  int64  $\Rightarrow$  int64  $\Rightarrow$ 
  int64  $\Rightarrow$  int  $\times$  int
where
  exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive =
  (let wheat-value =
    exchange-wheat-value-int price-n price-d max-wheat-send
      max-sheep-receive;
    sheep-value =
      exchange-sheep-value-int price-n price-d max-sheep-send
        max-wheat-receive;
    wheat-stays = wheat-value > sheep-value
  in if wheat-stays then
    if sint price-n > sint price-d then
      let wheat-receive = sheep-value div sint price-n
      in (wheat-receive,
        (wheat-receive * sint price-n + sint price-d - 1)
          div sint price-d)
    else
      let trade =
        min (sint max-sheep-send * sint price-d)
          (min (sint max-wheat-receive * sint price-n +
            sint price-n - 1)
            (sint int64-max * sint price-n));
        sheep-send = trade div sint price-d
        in (sheep-send * sint price-d div sint price-n, sheep-send)
      else if sint price-n > sint price-d then

```

```

let trade =
  min (sint max-wheat-send * sint price-n)
    (min (sint max-sheep-receive * sint price-d +
          sint price-d - 1)
        (sint int64-max * sint price-d));
wheat-receive = trade div sint price-n
in (wheat-receive,
    wheat-receive * sint price-n div sint price-d)
else
  let sheep-send = wheat-value div sint price-d
  in ((sheep-send * sint price-d + sint price-n - 1)
      div sint price-n,
      sheep-send))

```

exchange-v10-amounts-int-repaired is the normal-mode amount pair computed with both receive-cap repairs enabled, phrased in unbounded integers. The two branches selected when the staying offer is the more valuable side coincide with *exchange-v10-amounts-int*; the two corrected branches replace the plain offer value by the exact receive-cap minimum of the send-side product, the receive cap relaxed by one divisor unit, and the *Offer-Exchange-Arithmetic.int64-max* saturation ceiling.

lemma *repaired-wheat-stays-down-bounds*:

```

fixes price-n price-d imax max-wheat-send max-wheat-receive max-sheep-send
max-sheep-receive trade sheep-send wheat-receive :: int
assumes pn: 0 < price-n and pd: 0 < price-d
and sheep-more: price-n ≤ price-d
and imax-nonnegative: 0 ≤ imax
and receive-nonnegative: 0 ≤ max-wheat-receive
and send-nonnegative: 0 ≤ max-sheep-send
and wheat-stays:
  min (max-sheep-send * price-d) (max-wheat-receive * price-n) <
  min (max-wheat-send * price-n) (max-sheep-receive * price-d)
defines trade ≡
  min (max-sheep-send * price-d)
    (min (max-wheat-receive * price-n + price-n - 1) (imax * price-n))
and sheep-send ≡ trade div price-d
and wheat-receive ≡ sheep-send * price-d div price-n
shows 0 ≤ wheat-receive
and wheat-receive ≤ min max-wheat-receive max-wheat-send
and 0 ≤ sheep-send
and sheep-send ≤ min max-sheep-receive max-sheep-send

```

Proof sketch: the exact trade value exceeds the plain sheep value by at most $\text{price-n} - 1$, and the plain sheep value is strictly below the wheat value, which both opposite-side caps bound. Since the price numerator is at most the denominator, each floor quotient stays within the two caps of its output asset.

lemma *repaired-sheep-stays-wheat-more-bounds*:
fixes *price-n price-d imax max-wheat-send max-wheat-receive max-sheep-send*
max-sheep-receive trade wheat-receive sheep-send :: int
assumes *pn*: $0 < \text{price-n}$ **and** *pd*: $0 < \text{price-d}$
and *wheat-more*: $\text{price-d} < \text{price-n}$
and *imax-nonnegative*: $0 \leq \text{imax}$
and *send-nonnegative*: $0 \leq \text{max-wheat-send}$
and *receive-nonnegative*: $0 \leq \text{max-sheep-receive}$
and *sheep-stays*:
 $\min (\text{max-wheat-send} * \text{price-n}) (\text{max-sheep-receive} * \text{price-d}) \leq$
 $\min (\text{max-sheep-send} * \text{price-d}) (\text{max-wheat-receive} * \text{price-n})$
defines *trade* $\equiv$
 $\min (\text{max-wheat-send} * \text{price-n})$
 $(\min (\text{max-sheep-receive} * \text{price-d} + \text{price-d} - 1) (\text{imax} * \text{price-d}))$
and *wheat-receive* $\equiv \text{trade} \text{ div } \text{price-n}$
and *sheep-send* $\equiv \text{wheat-receive} * \text{price-n} \text{ div } \text{price-d}$
shows $0 \leq \text{wheat-receive}$
and $\text{wheat-receive} \leq \min \text{max-wheat-receive} \text{max-wheat-send}$
and $0 \leq \text{sheep-send}$
and $\text{sheep-send} \leq \min \text{max-sheep-receive} \text{max-sheep-send}$

Proof sketch: this is the mirror of the wheat-stays argument. The exact trade value exceeds the plain wheat value by at most $\text{price-d} - 1$, the plain wheat value is at most the sheep value, and the price denominator is strictly below the numerator, so each floor quotient stays within the two caps of its output asset.

lemma *exchange-v10-amounts-int-repaired-bounds*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows $0 \leq \text{fst} (\text{exchange-v10-amounts-int-repaired} \text{price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive})$
and $\text{fst} (\text{exchange-v10-amounts-int-repaired} \text{price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}) \leq$
 $\min (\text{sint max-wheat-receive}) (\text{sint max-wheat-send})$
and $0 \leq \text{snd} (\text{exchange-v10-amounts-int-repaired} \text{price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive})$
and $\text{snd} (\text{exchange-v10-amounts-int-repaired} \text{price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}) \leq$
 $\min (\text{sint max-sheep-receive}) (\text{sint max-sheep-send})$

Proof sketch: split on the offer-value comparison and the price order. The two branches whose staying offer is the more valuable side reuse the plain-cap branch bound lemmas; the two corrected branches are discharged by the repaired bound lemmas above.

theorem *exchange-v10-amounts-repaired-integer-characterization:*
fixes *wheat-word sheep-word* :: *uint128* **and** *amounts* :: *int* × *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *wheat-word* ≡
min
*(word-of-int (sint max-wheat-send * sint price-n) :: uint128)*
*(word-of-int (sint max-sheep-receive * sint price-d))*
and *sheep-word* ≡
min
*(word-of-int (sint max-sheep-send * sint price-d) :: uint128)*
*(word-of-int (sint max-wheat-receive * sint price-n))*
and *amounts* ≡
exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
(wheat-word > sheep-word) Exchange-Normal
(exact-receive-cap = True, symmetric-exact-receive-cap = True) =
Cxx-Ok
(word-of-int (fst amounts), word-of-int (snd amounts))
and *sint (word-of-int (fst amounts) :: int64) = fst amounts*
and *sint (word-of-int (snd amounts) :: int64) = snd amounts*

Proof sketch: identify both 128-bit values with their exact integers, then split on the value comparison and the price ordering. The unaffected branches use the existing plain-cap branch equations; the two corrected branches use the repaired branch equations. The repaired bounds theorem makes both re-encoded result words faithful signed-64 values.

theorem *exchange-v10-without-repaired-integer-characterization:*
fixes *amounts* :: *int* × *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
shows *exchange-v10-without-price-error-thresholds-with-options price-n price-d*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Normal
(exact-receive-cap = True, symmetric-exact-receive-cap = True) =
Cxx-Ok
(make-exchange-result
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send

$\text{max-sheep-receive} >$
 $\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 $\text{max-wheat-receive}))$

Proof sketch: both offer-value calls succeed with their exact 128-bit minima under *exchange-v10-pre*. Substitute the repaired amount characterization, then use its signed interpretations and the repaired bounds to prove that neither final runtime check is reachable.

theorem *exchange-v10-repaired-integer-characterization:*
fixes *amounts :: int × int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* $\equiv$
exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive*
max-sheep-send max-sheep-receive Exchange-Normal
 $(\text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{True}) =$
apply-price-error-thresholds price-n price-d
 $(\text{word-of-int } (\text{fst } \text{amounts})) (\text{word-of-int } (\text{snd } \text{amounts}))$
 $(\text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 $\text{max-sheep-receive} >$
 $\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 $\text{max-wheat-receive})$
Exchange-Normal

Proof sketch: the repaired pre-threshold characterization replaces the first call by its exact amount record; simplifying the record selectors leaves the threshold call shown here.

4.3 Posting refinement against the specification

For a positive integer denominator, rational floor and ceiling agree with the integer division formulas used by the executable exchange model. Proof sketch: multiply the defining floor or ceiling interval by the positive denominator. Euclidean division supplies the lower and upper endpoint inequalities.

lemma *floor-of-int-ratio:*
fixes *numerator denominator :: int*
assumes *denominator-positive: 0 < denominator*
shows
 $\lfloor (\text{of-int } \text{numerator} :: \text{rat}) / \text{of-int } \text{denominator} \rfloor =$
 $\text{numerator div denominator}$

lemma *ceiling-of-nonnegative-int-ratio:*
fixes *numerator denominator :: int*

assumes *numerator-nonnegative*: $0 \leq \text{numerator}$
and *denominator-positive*: $0 < \text{denominator}$
shows

$$\lceil (\text{of-int numerator} :: \text{rat}) / \text{of-int denominator} \rceil =$$

$$(\text{numerator} + \text{denominator} - 1) \text{ div } \text{denominator}$$

Specializing the preceding identities to a positive ledger price rewrites the specification's two rational round-trip equations into the canonical integer rounding equations of the implementation. Proof sketch: unfold the ledger-price coercion, cancel the nonzero price component, and invoke the corresponding ratio identity.

lemma *floor-times-ledger-price*:
fixes *numerator price-n price-d* :: *int*
assumes *price-d-positive*: $0 < \text{price-d}$
shows

$$\lfloor (\text{of-int numerator} :: \text{rat}) * \text{Offer-Exchange-Specification.Ledger-Price price-n price-d} \rfloor =$$

$$\text{numerator} * \text{price-n div price-d}$$

lemma *ceiling-div-ledger-price*:
fixes *numerator price-n price-d* :: *int*
assumes *numerator-nonnegative*: $0 \leq \text{numerator}$
and *price-n-positive*: $0 < \text{price-n}$
and *price-d-positive*: $0 < \text{price-d}$
shows

$$\lceil (\text{of-int numerator} :: \text{rat}) / \text{Offer-Exchange-Specification.Ledger-Price price-n price-d} \rceil =$$

$$(\text{numerator} * \text{price-d} + \text{price-n} - 1) \text{ div price-n}$$

The specification's declarative maximum unrestricted exchange is the same normalized pair computed by an unlimited implementation exchange. Proof sketch: make the feasible wheat set explicit and split on the price ordering. If wheat is more valuable, the implementation takes the largest wheat amount permitted by the request and the signed saturation bound; its rounded sheep amount round-trips and every feasible wheat amount is no larger. If sheep is at least as valuable, first take the largest sheep amount below the requested wheat value and round it back up to wheat. A zero sheep amount makes the feasible set empty; otherwise this pair is feasible and bounds every other round-tripping pair. Finiteness then identifies the specification's *Max* with the implementation amount.

lemma *specification-max-unrestricted-exchange-explicit*:
fixes *offer* :: *Offer-Exchange-Specification.sell-wheat-offer*
and *price-n price-d amount wheat-value selling buying* :: *int*
assumes *pn*: $0 < \text{price-n}$
and *pd*: $0 < \text{price-d}$


```

and amount-positive:  $0 < \text{amount}$ 
and price-n-bound:  $\text{price-n} \leq \text{int64-max-int}$ 
and price-d-bound:  $\text{price-d} \leq \text{int64-max-int}$ 
and amount-bound:  $\text{amount} \leq \text{int64-max-int}$ 
and offer-amount:  $\text{wheat-amount offer} = \text{amount}$ 
and offer-price:
  sheep-per-wheat offer =
    Offer-Exchange-Specification.Ledger-Price price-n price-d
defines wheat-value  $\equiv$ 
   $\min (\text{amount} * \text{price-n}) (\text{int64-max-int} * \text{price-d})$ 
and selling  $\equiv$ 
  (if  $\text{price-n} > \text{price-d}$ 
   then  $\text{wheat-value} \text{ div } \text{price-n}$ 
   else
    (let sheep =  $\text{wheat-value} \text{ div } \text{price-d}$ 
     in  $(\text{sheep} * \text{price-d} + \text{price-n} - 1) \text{ div } \text{price-n}$ ))
and buying  $\equiv$ 
  (if  $\text{price-n} > \text{price-d}$ 
   then  $(\text{selling} * \text{price-n}) \text{ div } \text{price-d}$ 
   else  $\text{wheat-value} \text{ div } \text{price-d}$ )
shows
  Offer-Exchange-Specification.max-unrestricted-exchange offer =
    (wheat-to-taker = selling,
     sheep-to-maker = buying)

```

Once pre-flight capacities cover the maximum unrestricted exchange, the repaired adjustment replays that exchange at the maker's actual caps. Proof sketch: the two unrestricted counterparty caps make the wheat offer the non-staying side. In the wheat-more branch, the exact receive slack covers the exchange's final division remainder; in the opposite branch, the rounded-up wheat amount covers the primary sheep amount. Thus the repaired integer amount pair is exactly the maximum unrestricted exchange. The normal price-error filter then either preserves it or returns the zero record.

lemma *repaired-adjust-offer-replays-max-unrestricted-exchange*:

```

fixes price-n price-d :: int32
and amount max-send max-receive :: int64
and wheat-value selling buying :: int
assumes pn:  $0 < \text{sint price-n}$ 
and pd:  $0 < \text{sint price-d}$ 
and amount-positive:  $0 < \text{sint amount}$ 
and wheat-value-defn: wheat-value =
   $\min (\text{sint amount} * \text{sint price-n})$ 
   $(\text{int64-max-int} * \text{sint price-d})$ 
and selling-defn: selling =
  (if  $\text{sint price-n} > \text{sint price-d}$ 
   then  $\text{wheat-value} \text{ div } \text{sint price-n}$ 
   else

```

```

      (let sheep = wheat-value div sint price-d
        in (sheep * sint price-d + sint price-n - 1) div sint price-n))
and buying-defn: buying =
  (if sint price-n > sint price-d
    then selling * sint price-n div sint price-d
    else wheat-value div sint price-d)
and selling-nonnegative:  $0 \leq \text{selling}$ 
and buying-nonnegative:  $0 \leq \text{buying}$ 
and selling-word:
  sint (word-of-int selling :: int64) = selling
and buying-word:
  sint (word-of-int buying :: int64) = buying
and zeroes:  $\text{selling} = 0 \iff \text{buying} = 0$ 
and max-send-nonnegative:  $0 \leq \text{sint max-send}$ 
and max-receive-nonnegative:  $0 \leq \text{sint max-receive}$ 
and max-send-le-amount:  $\text{sint max-send} \leq \text{sint amount}$ 
and selling-fits:  $\text{selling} \leq \text{sint max-send}$ 
and buying-fits:  $\text{buying} \leq \text{sint max-receive}$ 
shows
  adjust-offer-with-options price-n price-d max-send max-receive
    (exact-receive-cap = True,
     symmetric-exact-receive-cap = True) =
  Cxx-Ok
    (if  $0 < \text{selling} \wedge$ 
      price-error-bound-spec price-n price-d
        (word-of-int selling) (word-of-int buying) False
      then word-of-int selling else 0)

```

On a positive non-staying unrestricted exchange, the specification's rational one-percent price-error bound is exactly the implementation's scaled integer bound. Proof sketch: put the two rational prices over the common positive denominator, multiply the inequality by the positive wheat, price-denominator, and percentage factors, and use preservation of absolute value by multiplication with the positive factor.

lemma *specification-price-error-iff-implementation:*

fixes price-n price-d :: int32 **and** wheat sheep :: int

assumes pn: $0 < \text{sint price-n}$

and pd: $0 < \text{sint price-d}$

and wheat-positive: $0 < \text{wheat}$

and wheat-word:

sint (word-of-int wheat :: int64) = wheat

and sheep-word:

sint (word-of-int sheep :: int64) = sheep

shows

Offer-Exchange-Specification.price-error-within-bound

(*Offer-Exchange-Specification.Ledger-Price*

(sint price-n) (sint price-d))

(wheat-to-taker = wheat,

```

    sheep-to-maker = sheep) =
price-error-bound-spec price-n price-d
(word-of-int wheat) (word-of-int sheep) False

```

Recomputing liabilities for the positive normalized wheat amount returns the same pair. Proof sketch: for a wheat-more price, the normalized wheat product is already under the saturation bound and division reproduces its rounded sheep amount. For the other price order, the defining ceiling and floor round-trip equations reproduce both fields. The explicit liability characterization then yields the two executable helper results.

lemma *explicit-sell-wheat-liabilities-recompute*:

```

fixes price-n price-d :: int32 and amount :: int64
and wheat-value selling buying :: int
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and amount-positive: 0 < sint amount
and wheat-value-defn: wheat-value =
    min (sint amount * sint price-n)
        (int64-max-int * sint price-d)
and selling-defn: selling =
    (if sint price-n > sint price-d
     then wheat-value div sint price-n
     else
        (let sheep = wheat-value div sint price-d
         in (sheep * sint price-d + sint price-n - 1) div sint price-n))
and buying-defn: buying =
    (if sint price-n > sint price-d
     then selling * sint price-n div sint price-d
     else wheat-value div sint price-d)
and selling-positive: 0 < selling
and selling-word:
    sint (word-of-int selling :: int64) = selling
shows
    offer-selling-liabilities price-n price-d (word-of-int selling) =
        Cxx-Ok (word-of-int selling)
    offer-buying-liabilities price-n price-d (word-of-int selling) =
        Cxx-Ok (word-of-int buying)

```

The specification also recomputes the same maximum unrestricted exchange after replacing the requested amount by its positive normalized wheat amount. Proof sketch: instantiate the executable characterization at that normalized amount. The preceding fixed-point lemma identifies its two returned words with the original pair, while the characterization's signed round-trip clauses identify the underlying integers. The declarative characterization at the updated offer then yields the same record.

lemma *specification-max-unrestricted-exchange-recompute*:

```

fixes price-n price-d :: int32 and amount :: int64
and offer :: Offer-Exchange-Specification.sell-wheat-offer
and wheat-value selling buying :: int
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and amount-positive: 0 < sint amount
and wheat-value-defn: wheat-value =
  min (sint amount * sint price-n)
    (int64-max-int * sint price-d)
and selling-defn: selling =
  (if sint price-n > sint price-d
   then wheat-value div sint price-n
   else
    (let sheep = wheat-value div sint price-d
     in (sheep * sint price-d + sint price-n - 1) div sint price-n))
and buying-defn: buying =
  (if sint price-n > sint price-d
   then selling * sint price-n div sint price-d
   else wheat-value div sint price-d)
and selling-positive: 0 < selling
and selling-word:
  sint (word-of-int selling :: int64) = selling
and buying-word:
  sint (word-of-int buying :: int64) = buying
and offer-amount: wheat-amount offer = selling
and offer-price:
  sheep-per-wheat offer =
    Offer-Exchange-Specification.Ledger-Price
    (sint price-n) (sint price-d)
shows
  Offer-Exchange-Specification.max-unrestricted-exchange offer =
    (wheat-to-taker = selling,
     sheep-to-maker = buying)

```

```

lemma post-sell-offer-refines-post-sell-wheat-offer [export-audit]:
fixes price-n price-d :: int32
and amount :: int64
and passive-flag :: bool
and maker :: party-state
and offer :: Offer-Exchange-Specification.sell-wheat-offer
and abstract-maker :: Offer-Exchange-Specification.maker-state
defines
  offer ≡
    (is-passive = passive-flag,
     wheat-amount = sint amount,
     sheep-per-wheat =
       Offer-Exchange-Specification.Ledger-Price
       (sint price-n) (sint price-d))
and

```

```

abstract-maker ≡
  (wheat-balance = sint (sell-balance maker),
   sheep-balance = sint (buy-balance maker),
   wheat-selling-liabilities = sint (sell-liabilities maker),
   sheep-limit = sint (buy-limit maker),
   sheep-buying-liabilities = sint (buy-liabilities maker))
assumes
  party-state-wf maker
  Offer-Exchange-Specification.sell-wheat-offer-well-formed offer
shows
  (case post-sell-offer price-n price-d amount maker
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) of
    Cxx-Err - ⇒ False
  | Cxx-Ok Post-Malformed ⇒
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Malformed
  | Cxx-Ok Post-Line-Full ⇒
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Line-Full
  | Cxx-Ok Post-Underfunded ⇒
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Underfunded
  | Cxx-Ok Post-No-Offer ⇒
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.No-Offer
  | Cxx-Ok (Post-Created posted concrete-maker) ⇒
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Created
    (offer(wheat-amount := sint posted))
    (wheat-balance = sint (sell-balance concrete-maker),
     sheep-balance = sint (buy-balance concrete-maker),
     wheat-selling-liabilities =
       sint (sell-liabilities concrete-maker),
     sheep-limit = sint (buy-limit concrete-maker),
     sheep-buying-liabilities =
       sint (buy-liabilities concrete-maker)))

```

Proof sketch: first identify both models' maximum unrestricted exchange with the same explicit integer formulas, and identify the implementation's clamped capacities with the maker's raw specification headroom. The first two failed capacity tests then give the same line-full and underfunded outcomes. Zero receive capacity forces both exchanged amounts to zero, giving the remaining line-full case. Otherwise the repaired adjustment replays the covered exchange; its integer price test is the specification's rational test.

A zero or rejected pair gives no offer. For an accepted positive exchange, recomputation is a fixed point and both models reserve its two amounts as liabilities in the maker state, so their created offers and mapped states coincide bit-precisely.

```

end
theory Offer-Exchange-Divide-Layered
  imports Offer-Exchange-Arithmetic
begin

```

5 Layered model of the wide arithmetic helpers

This theory models the wide arithmetic helpers of *util/numeric.cpp* with one definition per C++ function, computing on 128-bit words exactly as the C++ does: *bigDivideUnsigned*, *bigDivide*, and *bigDivideOrThrow*; their 128-bit counterparts *bigDivideUnsigned128*, *bigDivide128*, and *bigDivideOrThrow128*; and *bigMultiplyUnsigned* and *bigMultiply*. Each layer is characterized over exact integers, and each top-level wrapper is proved equal to the collapsed constant of the same name in *OfferExchange.Offer-Exchange-Arithmetic*, so the integer characterizations already proved there transfer unchanged. The differential harness exercises every layer separately through the transport definitions in *Offer-Exchange-Test-Interface*.

5.1 Definitions

type-synonym *uint64* = 64 word

definition *uint64-max128* :: *uint128*
 — C++: *UINT64-MAX* after conversion to *uint128-t*
 where *uint64-max128* = $2^{64} - 1$

The definition *big-divide-unsigned* below models *bigDivideUnsigned*. The C++ function asserts that the divisor is non-zero, widens the three unsigned 64-bit operands to 128 bits, forms the rounded quotient in 128-bit arithmetic, stores its low 64 bits through the out-parameter, and returns whether the quotient fitted in 64 bits. The out-parameter and the Boolean return are modelled together as a pair.

definition *big-divide-unsigned* ::
 $\text{uint64} \Rightarrow \text{uint64} \Rightarrow \text{uint64} \Rightarrow \text{cxx-rounding} \Rightarrow$
 $(\text{bool} \times \text{uint64}) \text{ cxx-result}$
 — C++: *bigDivideUnsigned* (*util/numeric.cpp*)
 where
 $\text{big-divide-unsigned } A \ B \ C \ \text{rounding} =$
 $(\text{if } \neg (0 < \text{uint } C) \text{ then } \text{Cxx-Err } \text{Cxx-Assertion-Failed})$

```

else
  let a = (ucast A :: uint128); b = (ucast B :: uint128);
  c = (ucast C :: uint128);
  x = (if rounding = Cxx-Round-Down then (a * b) div c
       else (a * b + c - 1) div c)
  in Cxx-Ok (x ≤ uint64-max128, ucast x)

```

The definition *big-divide-layered* below models *bigDivide*. The C++ function asserts that the two multiplicands are non-negative and the divisor positive, casts all three to *uint64-t*, calls the unsigned helper, and on success additionally checks the result against *INT64-MAX*. Casting a signed 64-bit word to an unsigned one is the identity on bit patterns, so the arguments are passed through unchanged; only their interpretation switches from *sint* to *uint*. The comparison with *INT64-MAX* is unsigned in C++ because the constant is promoted, hence *uint*. On the failure path the C++ leaves the out-parameter unassigned; the model returns zero there, and its only caller never reads that value.

definition *big-divide-layered* ::
 $int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow cxx\text{-rounding} \Rightarrow$
 $(bool \times int64) \text{ cxx-result}$
 — C++: *bigDivide* (*util/numeric.cpp*)
where
big-divide-layered A B C rounding =
 (if $\neg (0 \leq sint\ A \wedge 0 \leq sint\ B \wedge 0 < sint\ C)$
 then Cxx-Err Cxx-Assertion-Failed
 else do {
 (res, r2) $\leftarrow$ *big-divide-unsigned* A B C rounding;
 if res then Cxx-Ok (uint r2 ≤ int64-max-int, r2)
 else Cxx-Ok (False, 0)
 })

The definition *big-divide-or-throw-layered* below models *bigDivideOrThrow*, which turns a false return from *bigDivide* into an overflow exception.

definition *big-divide-or-throw-layered* ::
 $int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow cxx\text{-rounding} \Rightarrow$
 $int64 \text{ cxx-result}$
 — C++: *bigDivideOrThrow* (*util/numeric.cpp*)
where
big-divide-or-throw-layered A B C rounding = do {
 (ok, res) $\leftarrow$ *big-divide-layered* A B C rounding;
 if ok then Cxx-Ok res else Cxx-Err Cxx-Overflow
}

5.1.1 Examples

lemma *big-divide-unsigned* 10 1 3 Cxx-Round-Down = Cxx-Ok (True, 3)

lemma *big-divide-unsigned* 10 1 3 Cxx-Round-Up = Cxx-Ok (True, 4)

lemma *big-divide-or-throw-layered* 10 1 3 *Cxx-Round-Up* = *Cxx-Ok* 4

lemma *big-divide-or-throw-layered* (-1) 1 3 *Cxx-Round-Up* =
Cxx-Err Cxx-Assertion-Failed

lemma *big-divide-or-throw-layered* *int64-max* *int64-max* 1 *Cxx-Round-Down* =
Cxx-Err Cxx-Overflow

5.2 The 128-bit family

definition *uint128-max* :: *uint128*
— C++: *uint128-max*()
where *uint128-max* = *word-of-int* ($2^{128} - 1$)

The definition *big-divide-unsigned128* below models *bigDivideUnsigned128*. The C++ function asserts that the divisor is non-zero, widens it to 128 bits, and, when rounding up, returns false early if the incremented numerator would wrap. Otherwise it forms the rounded quotient, stores its low 64 bits through the out-parameter, and returns whether the quotient fitted in 64 bits. On the early exit the out-parameter is left unassigned; the model returns zero there.

definition *big-divide-unsigned128* ::
uint128 $\Rightarrow$ *uint64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ (*bool* $\times$ *uint64*) *cxx-result*
— C++: *bigDivideUnsigned128* (*util/numeric.cpp*)
where
big-divide-unsigned128 *a B rounding* =
(if *B* = 0 then *Cxx-Err Cxx-Assertion-Failed*
else
let *b* = (*ucast B* :: *uint128*)
in if *rounding* = *Cxx-Round-Up* $\wedge$ *a* > *uint128-max* - (*b* - 1)
then *Cxx-Ok* (*False*, 0)
else
let *x* = (if *rounding* = *Cxx-Round-Down* then *a div b*
else (*a + b - 1 div b*)
in *Cxx-Ok* (*x* $\leq$ *uint64-max128*, *ucast x*))

The definitions *big-divide128-layered* and *big-divide-or-throw128-layered* below model *bigDivide128* and *bigDivideOrThrow128*. They mirror the signed wrapper and the throwing wrapper of the 64-bit family exactly.

definition *big-divide128-layered* ::
uint128 $\Rightarrow$ *int64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ (*bool* $\times$ *int64*) *cxx-result*
— C++: *bigDivide128* (*util/numeric.cpp*)
where
big-divide128-layered *a B rounding* =
(if $\neg 0 < \text{sint } B$ then *Cxx-Err Cxx-Assertion-Failed*
else do {


```

    (res, r2) ← big-divide-unsigned128 a B rounding;
    if res then Cxx-Ok (uint r2 ≤ int64-max-int, r2)
    else Cxx-Ok (False, 0)
  })

```

definition *big-divide-or-throw128-layered* ::
 $\text{uint128} \Rightarrow \text{int64} \Rightarrow \text{cxx-rounding} \Rightarrow \text{int64} \text{ cxx-result}$
 — C++: *bigDivideOrThrow128* (*util/numeric.cpp*)
where
big-divide-or-throw128-layered a B rounding = do {
 (ok, res) ← *big-divide128-layered* a B rounding;
 if ok then Cxx-Ok res else Cxx-Err Cxx-Overflow
}

5.2.1 Examples

lemma *big-divide-unsigned128 10 3 Cxx-Round-Up* = *Cxx-Ok (True, 4)*

lemma *big-divide-unsigned128 uint128-max 2 Cxx-Round-Up* = *Cxx-Ok (False, 0)*

With divisor one the guard cannot fire, exactly as the C++ comment argues: the incremented numerator wraps to the same value it started from, and the overflow is caught by the 64-bit range check instead.

lemma *big-divide-unsigned128 uint128-max 1 Cxx-Round-Up* =
Cxx-Ok (False, ucast uint128-max)

lemma *big-divide-unsigned128 uint128-max 1 Cxx-Round-Down* =
Cxx-Ok (False, ucast uint128-max)

lemma *big-divide-or-throw128-layered 10 3 Cxx-Round-Down* = *Cxx-Ok 3*

lemma *big-divide-or-throw128-layered 10 0 Cxx-Round-Down* =
Cxx-Err Cxx-Assertion-Failed

lemma *big-divide-or-throw128-layered uint128-max 1 Cxx-Round-Up* =
Cxx-Err Cxx-Overflow

5.3 Wide multiplication

The definitions *big-multiply-unsigned* and *big-multiply-layered* below model *bigMultiplyUnsigned* and *bigMultiply*. The unsigned helper zero-extends both operands to 128 bits and multiplies; the signed wrapper asserts that both operands are non-negative and passes them through, the cast to *uint64-t* being the identity on bit patterns.

definition *big-multiply-unsigned* :: $\text{uint64} \Rightarrow \text{uint64} \Rightarrow \text{uint128}$
 — C++: *bigMultiplyUnsigned* (*util/numeric.cpp*)

where *big-multiply-unsigned* $a\ b = (\text{ucast } a :: \text{uint128}) * \text{ucast } b$

definition *big-multiply-layered* $:: \text{int64} \Rightarrow \text{int64} \Rightarrow \text{uint128 } \text{cxx-result}$
 — C++: *bigMultiply* (*util/numeric.cpp*)
where

big-multiply-layered $a\ b =$
 (if $\neg (0 \leq \text{sint } a \wedge 0 \leq \text{sint } b)$ then *Cxx-Err Cxx-Assertion-Failed*
 else *Cxx-Ok* (*big-multiply-unsigned* $a\ b$))

5.3.1 Examples

lemma *big-multiply-unsigned* $(-1)\ (-1) = (2^{64} - 1) * (2^{64} - 1)$

lemma *big-multiply-layered* $6\ 7 = \text{Cxx-Ok } 42$

lemma *big-multiply-layered* $(-1)\ 7 = \text{Cxx-Err Cxx-Assertion-Failed}$

5.4 Reading the word arithmetic as integer arithmetic

The lemmas below let the 128-bit word operations of *big-divide-unsigned* be read as operations on unbounded integers. Zero extension preserves the unsigned value, the product of two unsigned 64-bit values fits in 128 bits, and so does the round-up numerator.

lemma *uint-ucast-uint64-uint128* [*simp*]:
 $\text{uint } (\text{ucast } (w :: \text{uint64}) :: \text{uint128}) = \text{uint } w$

Proof sketch: zero extension from 64 to 128 bits is an up-cast.

lemma *uint-uint64-max128* [*simp*]:
 $\text{uint } \text{uint64-max128} = 2^{64} - 1$

Proof sketch: the constant is a numeral below the 128-bit modulus.

lemma *uint64-product-fits-uint128*:
fixes $A\ B :: \text{uint64}$
shows $\text{uint } A * \text{uint } B < (2 :: \text{int})^{128}$

Proof sketch: both factors are non-negative and below two to the sixty-fourth power, so their product is below two to the one-hundred-twenty-eighth power.

lemma *uint64-round-up-numerator-fits-uint128*:
fixes $A\ B\ C :: \text{uint64}$

shows $\text{uint } A * \text{uint } B + \text{uint } C < (2 :: \text{int}) ^ 128$

Proof sketch: each unsigned 64-bit value is at most two to the sixty-fourth power minus one. The product of two such bounds plus a third bound is two to the one-hundred-twenty-eighth power minus two to the sixty-fourth power, which is below the modulus.

lemma *uint-product-uint64-uint128*:

fixes $A B :: \text{uint64}$

shows $\text{uint } ((\text{ucast } A :: \text{uint128}) * \text{ucast } B) = \text{uint } A * \text{uint } B$

Proof sketch: the product fits, so 128-bit multiplication is exact.

lemma *uint-round-up-numerator-uint64-uint128*:

fixes $A B C :: \text{uint64}$

assumes *divisor-positive*: $0 < \text{uint } C$

shows $\text{uint } ((\text{ucast } A :: \text{uint128}) * \text{ucast } B + \text{ucast } C - 1) = \text{uint } A * \text{uint } B + \text{uint } C - 1$

Proof sketch: the sum fits, so the 128-bit addition is exact, and it is at least one because the divisor is, so subtracting one is exact too.

lemma *ucast-uint128-uint64*:

$(\text{ucast } (x :: \text{uint128}) :: \text{uint64}) = \text{word-of-int } (\text{uint } x)$

Proof sketch: this is the general cast law at one type instance.

lemma *sint-nonnegative-eq-uint*:

fixes $w :: \text{int64}$

assumes $0 \leq \text{sint } w$

shows $\text{sint } w = \text{uint } w$

Proof sketch: a non-negative signed value lies below two to the sixty-third power, so taking its low 64 bits is the identity, and the unsigned value is exactly those low 64 bits.

5.5 Characterizations

Each layer is now characterized over exact integers. The unsigned helper is characterized for all inputs, not only for the non-negative signed ones its caller supplies.

theorem *big-divide-unsigned-characterization*:

big-divide-unsigned $A\ B\ C$ *rounding* =
 (if $C = 0$ then *Cxx-Err Cxx-Assertion-Failed*
 else *Cxx-Ok*
 (*rounded-quotient* (*uint* $A * \text{uint } B$) (*uint* C) *rounding* $\leq 2^{\wedge 64} - 1$,
word-of-int (*rounded-quotient* (*uint* $A * \text{uint } B$) (*uint* C) *rounding*)))

Proof sketch: a zero divisor fails the assertion. Otherwise split on the rounding mode; in each case the 128-bit numerator is exact by the fitting lemmas, word division is exact, the comparison against the 64-bit maximum is the unsigned comparison of exact values, and the final narrowing cast is *word-of-int* of the exact quotient.

lemma *uint-word-of-int-uint64*:
 assumes $0 \leq q$ and $q \leq (2 :: \text{int})^{\wedge 64} - 1$
 shows *uint* (*word-of-int* $q :: \text{uint64}$) = q

Proof sketch: the value is inside the unsigned 64-bit range.

theorem *big-divide-layered-characterization*:
big-divide-layered $A\ B\ C$ *rounding* =
 (if $\neg (0 \leq \text{sint } A \wedge 0 \leq \text{sint } B \wedge 0 < \text{sint } C)$
 then *Cxx-Err Cxx-Assertion-Failed*
 else *Cxx-Ok*
 (*rounded-quotient* (*sint* $A * \text{sint } B$) (*sint* C) *rounding* $\leq \text{int64-max-int}$,
 if *rounded-quotient* (*sint* $A * \text{sint } B$) (*sint* C) *rounding* $\leq 2^{\wedge 64} - 1$
 then *word-of-int* (*rounded-quotient* (*sint* $A * \text{sint } B$) (*sint* C) *rounding*)
 else 0))

Proof sketch: under the assertion the three signed values equal their unsigned readings, so the unsigned characterization applies with the same exact quotient. If that quotient fits in 64 bits the wrapper reads it back exactly and compares it against the signed maximum; otherwise the unsigned flag is already false and so is the signed comparison.

5.6 Agreement with the collapsed model

theorem *big-divide-or-throw-layered-eq* [*export-audit*]:
big-divide-or-throw-layered $A\ B\ C$ *rounding* =
big-divide-or-throw $A\ B\ C$ *rounding*

Proof sketch: rewrite the layered wrapper with the characterization of its inner layer. The assertion conditions coincide, and a quotient passes the signed range check exactly when the collapsed model does not report overflow, in which case both return the same word.

corollary *big-divide-or-throw-layered-success-iff*:
big-divide-or-throw-layered A B C rounding = Cxx-Ok result
 $\longleftrightarrow 0 \leq \text{ sint } A \wedge 0 \leq \text{ sint } B \wedge 0 < \text{ sint } C \wedge$
 $\text{ rounded-quotient } (\text{ sint } A * \text{ sint } B) (\text{ sint } C) \text{ rounding}$
 $\leq \text{ int64-max-int } \wedge$
 $\text{ result } = \text{ word-of-int }$
 $(\text{ rounded-quotient } (\text{ sint } A * \text{ sint } B) (\text{ sint } C) \text{ rounding})$

Proof sketch: transfer the existing theorem along the equality.

5.7 Wide multiplication agrees with the collapsed model

lemma *uint-big-multiply-unsigned*:
 $\text{ uint } (\text{ big-multiply-unsigned } a \ b) = \text{ uint } a * \text{ uint } b$

Proof sketch: the product of two 64-bit values fits in 128 bits.

lemma *big-multiply-unsigned-word*:
fixes $a \ b :: \text{ int64}$
assumes $0 \leq \text{ sint } a$ **and** $0 \leq \text{ sint } b$
shows $\text{ big-multiply-unsigned } a \ b = \text{ word-of-int } (\text{ sint } a * \text{ sint } b)$

Proof sketch: both words have the unsigned value $\text{ sint } a * \text{ sint } b$, the left one by the exact product lemma and the reading of non-negative signed values as unsigned ones, so they are the same word.

theorem *big-multiply-layered-eq* [*export-audit*]:
 $\text{ big-multiply-layered } a \ b = \text{ big-multiply } a \ b$

Proof sketch: the assertions coincide, and on the success path both sides return the word with unsigned value $\text{ sint } a * \text{ sint } b$.

5.8 The 128-bit family: word arithmetic as integer arithmetic

lemma *uint-uint128-max* [*simp*]:
 $\text{ uint } \text{ uint128-max } = 2^{128} - 1$

Proof sketch: the constant is the largest 128-bit numeral.

lemma *uint128-guard-value*:
fixes $B :: \text{ uint64}$
assumes $B \neq 0$
shows $\text{ uint } (\text{ uint128-max } - ((\text{ ucast } B :: \text{ uint128}) - 1)) =$

$$2^{128} - 1 - (\text{uint } B - 1)$$

Proof sketch: the divisor is at least one, so subtracting one from its zero extension is exact, and subtracting that from the all-ones word is exact as well because it is at most the all-ones value.

lemma *uint128-guard-word-iff*:

fixes $a :: \text{uint128}$ **and** $B :: \text{uint64}$

assumes $B \neq 0$

shows $(a > \text{uint128-max} - ((\text{ucast } B :: \text{uint128}) - 1)) =$
 $(\text{uint } a > 2^{128} - 1 - (\text{uint } B - 1))$

Proof sketch: unsigned word comparison compares the exact values.

lemma *uint128-round-up-numerator*:

fixes $a :: \text{uint128}$ **and** $B :: \text{uint64}$

assumes $B \neq 0$ **and** $\text{uint } a \leq 2^{128} - 1 - (\text{uint } B - 1)$

shows $\text{uint } (a + \text{ucast } B - 1) = \text{uint } a + \text{uint } B - 1$

Proof sketch: the two word operations compute the exact result modulo the 128-bit modulus, whatever the intermediate sum does. The guard says the exact result is at most the largest 128-bit value, and it is non-negative because the divisor is at least one, so the reduction is the identity. This is precisely the argument in the C++ comment.

5.9 The 128-bit family: characterizations

theorem *big-divide-unsigned128-characterization*:

big-divide-unsigned128 a B *rounding* =

(if $B = 0$ then *Cxx-Err Cxx-Assertion-Failed*

else if *rounding* = *Cxx-Round-Up* $\wedge$ $\text{uint } a > 2^{128} - 1 - (\text{uint } B - 1)$

then *Cxx-Ok* (*False*, 0)

else *Cxx-Ok* (*rounded-quotient* ($\text{uint } a$) ($\text{uint } B$) *rounding* $\leq 2^{64} - 1$,
word-of-int (*rounded-quotient* ($\text{uint } a$) ($\text{uint } B$) *rounding*)))

Proof sketch: a zero divisor fails the assertion. Otherwise the word guard is the exact integer guard. When it fires the function returns false without touching the out-parameter. When it does not, split on the rounding mode: word division is exact, the round-up numerator is exact by the previous lemma, the 64-bit comparison compares exact values, and the narrowing cast is *word-of-int* of the exact quotient.

theorem *big-divide128-layered-characterization*:

big-divide128-layered a B *rounding* =

```

(if  $\neg 0 < \text{sint } B$  then Cxx-Err Cxx-Assertion-Failed
 else if  $\text{rounding} = \text{Cxx-Round-Up} \wedge \text{uint } a > 2^{128} - 1 - (\text{sint } B - 1)$ 
 then Cxx-Ok (False, 0)
 else Cxx-Ok
  (rounded-quotient (uint a) (sint B) rounding  $\leq \text{int64-max-int}$ ,
   if  $\text{rounded-quotient} (\text{uint } a) (\text{sint } B) \text{ rounding} \leq 2^{64} - 1$ 
    then word-of-int (rounded-quotient (uint a) (sint B) rounding)
    else 0))

```

Proof sketch: under the assertion the signed divisor equals its unsigned reading, so the unsigned characterization applies. If the guard fires, the false flag is passed through. Otherwise, if the quotient fits in 64 bits the wrapper reads it back exactly and compares it against the signed maximum, and if it does not, both flags are false.

5.10 The 128-bit family agrees with the collapsed model

theorem *big-divide-or-throw128-layered-eq* [*export-audit*]:

big-divide-or-throw128-layered a B rounding =
big-divide-or-throw128 a B rounding

Proof sketch: rewrite the layered wrapper with the characterization of its inner layer. The assertion and the guard coincide with the collapsed model's first two branches, and a quotient passes the signed range check exactly when the collapsed model does not report overflow.

corollary *big-divide-or-throw128-layered-success-iff*:

big-divide-or-throw128-layered a B rounding = Cxx-Ok result
 $\iff 0 < \text{sint } B \wedge$
 $\neg (\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{uint } a > (2 :: \text{int})^{128} - 1 - (\text{sint } B - 1)) \wedge$
 $\text{rounded-quotient} (\text{uint } a) (\text{sint } B) \text{ rounding}$
 $\leq \text{int64-max-int} \wedge$
 $\text{result} = \text{word-of-int}$
 $(\text{rounded-quotient} (\text{uint } a) (\text{sint } B) \text{ rounding})$

Proof sketch: transfer the existing theorem along the equality.

end